Mansoura University

Faculty of Computer & Information Sciences Department of

Computer Science

Phirus

A Detection tool for phishing mails enhanced with ML

Team Members

Ammar Yasser Gomaa

Abdallah Gomaa Taha

Ali Osama Elhefnawy

Mohamed Ali

Noureldein Hamdy

Shrief Magdy Mahmoud

Nourhan Mahmoud

Kawther Walid

Raneem Sameh

Aya Mosaad

Supervised By: -

Dr / Nehal Sakr

Eng / Karam Mohamed

Mansoura University, Egypt (2024 / 2025)

Abstract

Phishing emails remain one of the most persistent, adaptive, and damaging forms of cyberattacks, exploiting human psychology and technical vulnerabilities to compromise systems, steal credentials, and gain unauthorized access to sensitive data. As cyber threats continue to evolve in complexity, phishing techniques have also become more sophisticated, often bypassing traditional security filters and deceiving even the most vigilant users. This challenge is particularly pronounced in enterprise environments where email serves as a primary vector for communication and potential exploitation.

In response to this pressing issue, this graduation project presents the development of a comprehensive and intelligent phishing email detection tool designed to serve both Security Operations Center (SOC) analysts and general end users. The proposed solution diverges from conventional approaches by emphasizing detection and diagnostic clarity over prevention and control. The tool is engineered to analyze and evaluate emails post-delivery, providing detailed insights into their safety and legitimacy, thus equipping users with the knowledge needed to respond appropriately.

At its core, the system integrates advanced Artificial Intelligence (AI) and Machine Learning (ML) algorithms to identify malicious patterns and behavior that may not be evident through static rule-based methods. These AI components are capable of learning from vast datasets and evolving over time to detect emerging phishing techniques. The system's intelligent analysis includes the behavioural scrutiny of embedded URLs, linguistic examination of message content, evaluation of header anomalies, and analysis of file attachments all aimed at uncovering subtle indicators of malicious activity.

To further strengthen its detection capabilities, the system is connected to a network of external Application Programming Interfaces (APIs) provided by trusted cybersecurity platforms. These APIs supply real-time threat intelligence, enabling the tool to cross-reference URLs, domains, and IP addresses with global threat databases and blacklists. Additionally, the system incorporates a robust IP reputation checking mechanism to identify and flag potentially compromised or suspicious servers involved in the delivery of phishing emails.

A unique differentiator of this tool lies in its layered and synergistic approach, which combines automated AI analysis with external threat intelligence to maximize accuracy and minimize false positives. By offering detailed, transparent reporting rather than automatic filtering or deletion, the system supports forensic investigation and decision-making, making it especially valuable in SOC environments where analysts require full visibility into potential threats.

Furthermore, the system is designed with usability and accessibility in mind. The user interface presents findings in a clear and intuitive format, making the tool approachable for non-technical users while still offering granular insights for cybersecurity professionals. Through this hybrid design, the tool bridges a critical gap between technical capability and user empowerment a gap often overlooked by traditional security solutions.

This project aspires to contribute meaningfully to the field of cybersecurity by providing a scalable, intelligent, and user-friendly solution for phishing detection. By merging advanced technologies such as AI, real-time API integrations, and IP reputation systems into a unified platform, the proposed tool aims to enhance organizational resilience and individual awareness against phishing attacks. Ultimately, it positions itself as a vital asset in the modern cybersecurity toolkit, delivering both strategic and operational value in the ongoing battle against digital deception.

Acknowledgement

The success of our project would not have been possible without the contributions of numerous individuals and organizations. On behalf of our team, we would like to express our sincere gratitude to everyone who played a part in making our project a reality.

We would like to express our deepest gratitude to everyone who contributed to the successful completion of this project.

First and foremost, we thank our academic advisor Dr. Nehal Sakr, for their invaluable guidance, insightful feedback, and unwavering support throughout this journey. Their expertise and encouragement have been instrumental in shaping the direction and scope of this project.

We extend our sincere appreciation to Faculty of Computer & Information Science – Mansoura University, for providing the necessary resources and an environment conducive to research and innovation.

Special thanks to our colleagues and peers for their collaborative spirit and constructive discussions, which enriched our understanding and improved the quality of our work.

We also acknowledge the various platforms and communities that provided the APIs, datasets, and technical tools essential for building and testing our phishing detection system. Their contributions have significantly enhanced the functionality and reliability of our project.

Finally, we express heartfelt gratitude to our families and friends for their constant support, patience, and motivation throughout the duration of this project.

This project is a testament to the collective effort and commitment of everyone involved, and we are deeply thankful for their contributions.

Table of Contents.

List of figures

List of Tables.

List of Abbreviations

Full Form	Abbreviation
Antivirus	AV
Application Programming Interface	API
Artificial Intelligence	AI
Comma-Separated Values	CSV
Data Loss Prevention	DLP
Domain Name System	DNS
Domain-based Message Authentication, Reporting and Conformance	DMARC
DomainKeys Identified Mail	DKIM
Fully Qualified Domain Name	FQDN
Hypertext Transfer Protocol / Secure	HTTP / HTTPS
Internet Protocol	IP
IP Database (like AbuseIPDB)	IPDB
JSON Web Token	JWT
Machine Learning	ML
One Time Password	OTP
Portable Document Format	PDF
Security Operations Center	SOC

Full Form	Abbreviation
Sender Policy Framework	SPF
Secure Sockets Layer	SSL
Simple Mail Transfer Protocol	SMTP
Transport Layer Security	TLS
Unified Modelling Language	UML
Uniform Resource Locator	URL
User Experience	UX
User Interface	UI

Chapter (1)

Chapter 1: Introduction

1.1 Overview

Phishing is one of the most prevalent forms of cyberattack in our world, which targeting individuals and organizations worldwide. It involves the use of deceptive tactics to manipulate users into accessing sensitive information, such as login credentials, financial information, or personal details. Attackers achieve this by masquerading as legitimate entities—such as banks, popular brands, or trusted contacts—to gain the victim's trust and prompt them to act without suspicion. The ongoing rise in phishing attacks, fueled by increasingly sophisticated techniques, has positioned phishing as a critical issue in the realm of cybersecurity.

Phishing has evolved considerably over the years. Originally, phishing emails were relatively easy to spot, often riddled with spelling errors and suspicious links. However, attackers have become much more advanced, utilizing social engineering tactics and psychological triggers to trick recipients. Modern phishing emails can appear almost identical to legitimate emails, using brand logos, professional formatting, and carefully crafted language that lures users into a false sense of security. In addition to basic phishing, there are now more targeted variants, such as spear phishing, which tailors' messages to specific individuals, and whaling, which targets high-ranking executives. These types of phishing pose even greater risks, as they are harder to detect and often yield higher rewards for attackers.

Phishing by emails can be done with phishing (URLs, Headers, Steganography, Attached files like pdf)

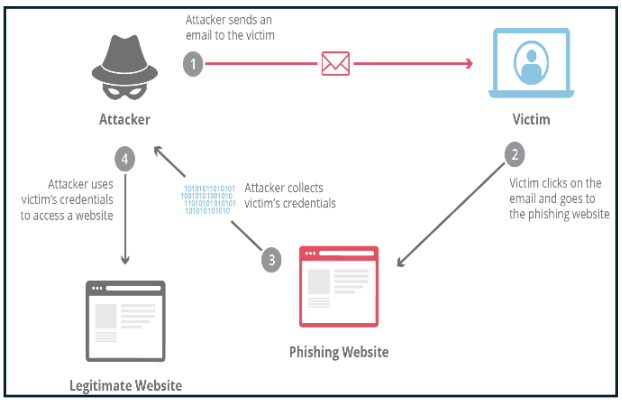


Figure 1 : Phishing

During 2022, phishing emails increased by 61% compared to previous years, with attackers continually adapting to evade detection methods. This means that not only are more people at risk, but they're also facing increasingly sophisticated phishing attempts.

Despite ongoing efforts to combat phishing, traditional security solutions like spam filters and antivirus programs have proven insufficient to fully address the problem. Many of these solutions rely on predefined rules or known patterns, which limits their effectiveness against newly devised phishing tactics. Consequently, phishing emails continue to evade these security measures, resulting in data breaches, financial loss, and compromised privacy. According to recent cybersecurity reports, phishing attacks account for a significant proportion of data breaches, costing businesses millions in recovery and prevention efforts.

Given this backdrop, this graduation project is highly relevant as it aims to tackle phishing detection by developing a robust tool capable of identifying phishing emails with greater accuracy. By incorporating advanced detection techniques, such as analyzing the content, headers, and URLs of emails, this project seeks to contribute to the field of cybersecurity with a more effective solution to phishing. This tool has the potential to protect users from increasingly complex phishing threats, making email communication safer for individuals and organizations alike.

1.2 Motivations and Problem Statement

Motivations:

The motivation behind this project stems from the widespread and evolving nature of phishing attacks, which have become one of the leading causes of data breaches worldwide. Phishing not only poses a significant financial threat but also impacts personal privacy, corporate reputation, and national security. The increasing sophistication of phishing techniques, such as highly targeted spear-phishing and whaling, makes them harder to detect and more damaging when successful. Additionally, traditional detection methods are often inadequate, as they rely on static rules or past data, which attackers can easily circumvent. Therefore, there is a strong need for advanced detection systems that can adapt to these evolving tactics and protect users more effectively.

Beyond the technical need, there is a societal motivation as well. As more people depend on digital communication, from personal emails to work correspondence, phishing continues to endanger anyone with an email account. The project aims to address this urgent need by

developing an innovative tool that provides more reliable detection of phishing emails. This tool could protect countless individuals and organizations, making digital spaces safer and fostering greater trust in online interactions.

Problem Statement:

Despite advancements in cybersecurity, phishing emails remain a persistent challenge for individuals, organizations, and security providers. Current detection mechanisms often fall short due to their reliance on predefined rules or historical data, which do not account for new and adaptive phishing tactics. As a result, phishing attacks can evade traditional filters and reach users, leaving them vulnerable to deception and exploitation.

The problem addressed in this project is the need for a more effective and adaptable phishing detection tool that can accurately identify phishing emails, regardless of changing techniques or formats. By analyzing various aspects of phishing emails—including content, headers, and embedded URLs—this project aims to develop a solution that is more resilient and capable of detecting phishing with higher accuracy. The objective is to create a tool that minimizes both false positives and false negatives, providing users and organizations with a dependable defense against phishing threats.

1.3 Objectives

The primary objective of this project is to develop a highly accurate tool for detecting phishing emails. The tool aims to improve detection accuracy, reduce false positives and negatives, and adapt to the evolving tactics used in phishing attacks.

- Develop an Automated Phishing Detection Tool.
- Provide full analysis for mail body and head by several ways.
- Offers detailed reports.
- Integration with threat intelligence platforms.
- Provide product as application.
- Automated phishing detection solution with ML.

1.4 Contributions

- **AI-Powered URL Detection:** We add the usage of machine learning algorithms to analyze and classify suspicious URLs, enhancing the system's ability to detect phishing attempts that rely on deceptive links.

- **Multi-Layered Phishing Analysis**: The system combines content-based analysis, header, and attachment with advanced URL and IP reputation checks to improve detection accuracy.
- **User-Oriented Interface**: The platform provides a simple and accessible interface for both SOC analysts and general users, making phishing detection more accessible and practical in real-world scenarios.
- **Dataset Enhancement and Annotation**: A phishing dataset was enhanced and used to train and validate the model, contributing to the research community by showing practical application and evaluation.
- **Cross-Platform Availability:** A web application and a mobile application were developed to make the system widely accessible and easy to use on different platforms, enhancing user convenience and usability.

1.5 Project Outlines

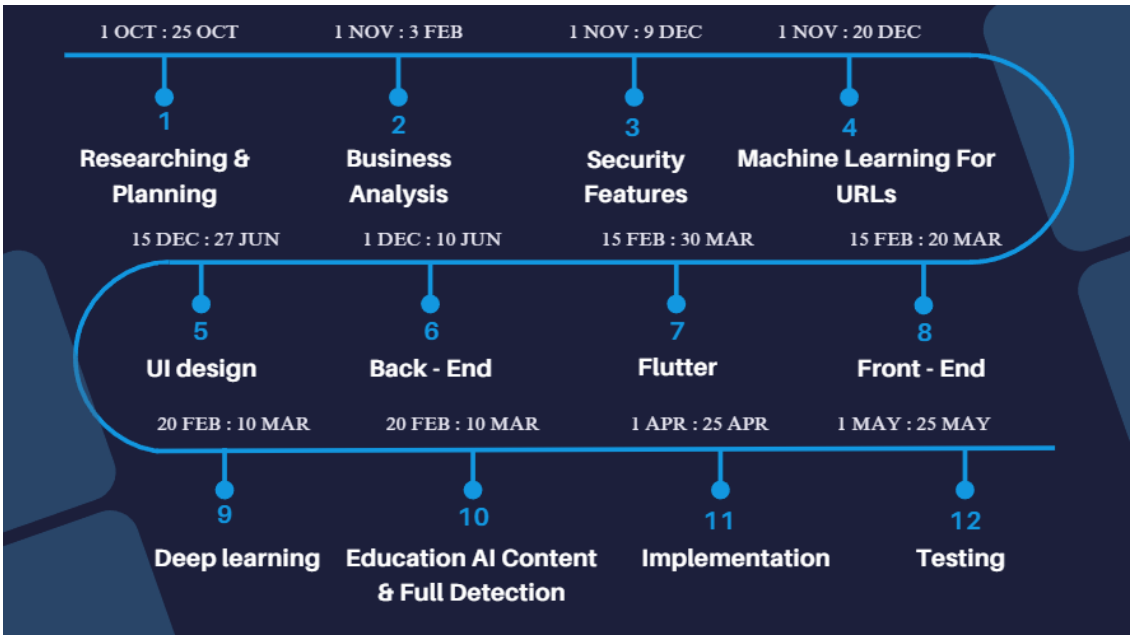


Figure 2 : Timeline

Chapter (2)

Chapter 2: Literature Review

2.1 Introduction

In this chapter, we provide background on the tools and techniques needed to build our system. We also review previous work related to our system and common technologies used. At the end of the chapter, we present a comparison between the system we are going to build and related systems.

2.2 Background

Background chapter provides a comprehensive overview of the project's key technologies and terminologies. It aims to establish a solid foundation of knowledge for readers to understand the project's scope and significance. The chapter explores fundamental technologies, known for their specific features and advantages. This exploration sets the stage for subsequent implementation, methodology, and results chapters. The Background chapter's emphasis on these technologies and terminologies allows readers to grasp the project's context, challenges, and potential implications. It provides a meaningful framework for decision-making and contributions within the project's domain. By familiarizing themselves with these foundational concepts, readers can navigate the project's objectives and strategies effectively. Ultimately, the Background chapter serves as a crucial steppingstone toward a comprehensive understanding of the project.

2.3 Related Work

2.3.1 Outlook

Outlook is an email client and personal information manager from Microsoft. It allows users to send and receive emails, manage calendars, tasks, contacts, and notes. It's widely used in both personal and professional settings.

Features

- Microsoft uses AI and machine learning to identify suspicious or potentially dangerous emails based on patterns commonly found in phishing attacks.
- Offers more robust DLP options in enterprise environments
- Often used in enterprise environments that integrate with Microsoft Office and other third-party applications.
- Outlook flags suspicious senders, especially those attempting to impersonate trusted entities, by highlighting unverified senders and showing warnings.

- Suspicious Sender Identification: Outlook flags suspicious senders, especially those attempting to impersonate trusted entities, by highlighting unverified senders and showing warnings.

Advantages

- Integrated Email Management: Combines email, calendar, tasks, and contacts in one application, making it easy to manage communication and scheduling.
- Robust Calendar Features: Allows users to schedule meetings, send invites, set reminders, and view shared calendars.
- Task Management: Users can create, assign, and track tasks, helping with organization and productivity.
- Search Functionality: Powerful search tools make it easy to find emails, contacts, or calendar events quickly.

Disadvantages

- Users may still fall victim to phishing attacks, where attackers impersonate legitimate organizations or contacts. Even with security features, human error can lead to successful phishing attempts.
- Outlook can be a vector for malware and ransomware if users open infected attachments or click on malicious links. While Outlook has protections like Safe Attachments, these measures may not catch every threat.
- Outlook can be a vector for malware and ransomware if users open infected attachments or click on malicious links. While Outlook has protections like Safe Attachments, these measures may not catch every threat.
- The evolving nature of phishing attacks means that new techniques may emerge before security measures are updated. Outlook's protections may not immediately recognize newly developed phishing schemes.
- Some phishing emails can bypass Outlook’s filters by using deceptive filenames or disguised file types. Users might open these attachments, believing they are safe.

2.3.2 Gmail

Gmail is a free email service developed by Google. It allows users to send and receive emails, organize messages into folders, and use features like chat, video calls, and integrated Google

services (like Google Drive and Google Calendar). Gmail offers a user-friendly interface, powerful spam filtering, and substantial storage space for emails. It can be accessed via web browsers, as well as through mobile apps for smartphones and tablets.

Features

- Integrates seamlessly with Google Workspace applications.
- A confidential mode that will expire email messages after a set period of time and remove the option for recipients to forward, copy, download, or print the message from Gmail.
- All Gmail messages are encrypted in transit and at rest. Messages sent to third-party providers are encrypted with transport layer security (TLS) when possible or required by configuration.
- It integrates with Google's Safe Browsing technology, which warns users if they click on links in emails that may lead to malicious websites.
- Gmail warns you before you download an attachment that could put your security at risk.
- Many malware and phishing attacks start with an email. Gmail blocks more than 99.9% of spam, phishing attempts, and malware from Reaching

Advantages

- Gmail uses machine learning and AI-powered algorithms to detect and filter out phishing and malware attempts. These algorithms are continuously updated with Google's threat intelligence to help block sophisticated phishing campaigns and malicious attachments.
- Gmail scans links within emails to identify and warn users about potentially harmful or suspicious websites. This helps protect users from phishing attempts where attackers try to trick them into clicking malicious links.
- Gmail supports BIMI, which allows verified companies to display their logos next to their emails in users' inboxes. This helps users recognize legitimate companies and reduce the risk of falling for impersonation scams.
- Gmail's powerful spam filter is highly effective at identifying unwanted and potentially harmful emails, reducing the risk of users clicking on spam or phishing messages. Google's AI continuously learns from spam patterns to improve detection accuracy.

Disadvantages

- Google security measures protect user data, and the company officially stopped scanning user emails for ad targeting in 2017, the potential for privacy breaches and unauthorized access remains a concern
- Phishing Scams: Fraudulent emails can trick users into sharing personal information or downloading malware. Even with Gmail's filters, some phishing emails still slip through.
- Evasion of Filters: Despite Gmail's spam and phishing filters, some phishing emails manage to bypass detection, reaching users' inboxes.
- Fake Login Pages: Phishing emails often include links to fake Gmail login pages, tricking users into entering their login credentials, which attackers can then steal.
- Impersonation: Phishers may impersonate trusted contacts, like colleagues or family members, using familiar names or similar email addresses to deceive users.
- Malicious Links and Attachments: Phishing emails often contain links or attachments that install malware or redirect users to harmful websites when clicked.
- Google Workspace provides DLP features, but users must actively configure and manage these settings. If DLP is not correctly set up, sensitive data can be inadvertently shared.

2.3.3 Phish Tool

Phish Tool is a cybersecurity platform designed for phishing email analysis and incident response. It helps organizations and security teams to efficiently investigate, analyze, and respond to phishing threats reported by users. PhishTool specializes in examining suspicious emails and identifying potential phishing attempts by providing contextual intelligence and facilitating collaboration among security teams.

Features

- PhishTool provides a centralized platform where security analysts can collect and analyze reported phishing emails. It helps in categorizing, visualizing, and correlating phishing-related data. The centralized dashboard aggregates phishing incidents, allowing security teams to focus on high-priority threats quickly.
- PhishTool integrates with multiple threat intelligence sources, such as VirusTotal, AbuseIPDB, Spamhaus, and PhishTank. This integration allows security analysts to cross-reference URLs, email addresses, and attachments with known threat indicators, enhancing the accuracy of identifying phishing attacks.
- The platform offers comprehensive email analysis, including the examination of headers, metadata, and email content. Analysts can identify signs of email spoofing,

suspicious attachments, and links. The tool assists in uncovering key indicators of compromise (IOCs) within the email body or headers.

- PhishTool enables domain and URL reputation checks, which help identify malicious links. It employs domain reputation analysis and domain similarity checks to flag URLs that closely resemble legitimate sites (to catch typosquatting attempts).

Advantages

- Improved Incident Response: Faster and more efficient analysis of phishing emails.
- Streamlined Workflow: Reduces manual tasks and complexity for analysts.
- Enhanced Threat Context: Better decision-making with integrated threat intelligence.
- Centralized Collaboration: Facilitates teamwork and communication within security teams.
- Comprehensive Reporting: Detailed documentation and compliance support.

Disadvantages

- PhishTool relies on integrations with external threat intelligence feeds, its effectiveness depends on the quality and coverage of these sources. If a new phishing technique is not yet recognized in the external databases, the tool might miss it.
- PhishTool relies on integrations with external threat intelligence feeds, its effectiveness depends on the quality and coverage of these sources. If a new phishing technique is not yet recognized in the external databases, the tool might miss it.

2.3.4 Virus Total

VirusTotal (VT) is a widely used scanning service for researchers and practitioners to label malicious entities and predict new security threats. Unfortunately, it is little known to the end-users how VT URL scanners decide on the maliciousness of entities and the attack types they are involved in (e.g., phishing or malware-hosting websites). In this paper, we conduct a systematic comparative study on VT URL scanners' behavior for different attack types of malicious URLs, in terms of 1) detection specialties, 2) stability, 3) correlations between scanners, and 4) lead/lag behaviors. Our findings highlight that the VT scanners commonly disagree with each other on their detection and attack type classification, leading to challenges in ascertaining the

maliciousness of a URL and taking prompt mitigation actions according to different attack types. This motivates us to present a new highly accurate classifier that helps correctly identify the attack types of malicious URLs at the early stage. This in turn assists practitioners in performing better threat aggregation and choosing proper mitigation actions for different attack types.

Features

- Upload a file for scanning: analysis your file with 70+ antivirus products, 10+ dynamic analysis sandboxes and a myriad of other security tools to produce a threat score and relevant context to understand it.
- Get a file report by hash: given a {md5, sha1, sha256} hash, retrieves the pertinent analysis report including threat reputation and context produced by 70+ antivirus products, 10+ dynamic analysis sandboxes and a myriad of other security tools and datasets.
- Scan URL: analysis your URL with 70+ antivirus products/blocklists and a myriad of other security tools to produce a threat score and relevant context to understand it.
- Get a URL analysis report: given a URL, retrieves the pertinent analysis report including threat reputation and context produced by 70+ antivirus products/blocklists and a myriad of other security tools and datasets.
- Get a domain report: given a domain, retrieves the pertinent analysis report including threat reputation and context produced by 70+ antivirus products/blocklists and a myriad of other security tools and datasets.

Advantages

- Free Access: The basic version of VirusTotal is free to use, making it accessible for individuals and small organizations.
- Threat Intelligence: VirusTotal offers insights into the behavior of files and URLs, helping users understand the nature of potential threats.
- Community Feedback: Users can contribute and view feedback from the community, which can provide additional context on files or URLs.
- File and URL Analysis: It can analyze a variety of file types and URLs, helping users identify malware, phishing attempts, and other security risks.
- Security Reports: Detailed reports generated by VirusTotal help users understand the specifics of any detected threats.

Disadvantages

- False Positives and Negatives: The reliance on multiple antivirus engines can lead to false positives (legitimate files flagged as threats) and false negatives (malicious files not detected), which may mislead users.
- Privacy Concerns: Uploading files to VirusTotal may raise privacy issues, especially if sensitive or proprietary data is involved, as files are shared with multiple antivirus vendors.
- API Rate Limits: For users relying on the API for integration, there are rate limits and restrictions that can hinder extensive automated analysis.
- File Size Limitations: There are restrictions on the maximum file size that can be uploaded, which may be limiting for users needing to analyze larger files.
- Lack of Context: The results may not provide enough context for users to make informed decisions about whether to trust a file or URL.

2.3.5 Talos Intelligence

Talos Intelligence is a cybersecurity research and intelligence website within Cisco, providing threat intelligence and security tools to help protect users, organizations, and networks from cyber threats. Talos operates by analyzing vast amounts of internet traffic, identifying threats, and disseminating actionable threat intelligence, which contributes to Cisco's security products like Cisco Firepower, Cisco Umbrella, and other threat-prevention tools.

Features

- Threat Detection and Analysis: Provides real-time information on malware, phishing, and other cyber threats.
- Global Threat Intelligence: Uses a worldwide network to collect and analyze data from various sources, offering up-to-date insights on emerging threats.
- Reputation Analysis: Assesses IP addresses, URLs, and domains for malicious activity and provides a reputation score to help block harmful content.
- Malware Research: Conducts in-depth analysis of new malware, ransomware, and exploits to identify and block them before they cause harm.
- Incident Response: Assists organizations in handling and investigating incidents through detailed reports and response recommendations.

Advantages

- Large Data Pool: By leveraging Cisco's extensive network, Talos has access to a massive amount of data, providing better insights into global threat trends.
- Automated Threat Blocking: Integrates with Cisco security solutions to automatically block threats at the network and endpoint levels.
- Real-time Updates: Constantly updates its database with new threats, which helps organizations stay protected against the latest cyberattacks.

- Community Support and Collaboration: Shares insights and research openly with the broader cybersecurity community to enhance collective defense efforts.
- Diverse Security Coverage: Protects against a wide range of threats, including malware, phishing, botnets, and other advanced persistent threats.
- Difference Resources: it deals with multiple software to detect and search user's input.

Disadvantages

- Cisco Dependency: Talos is most effective when integrated with Cisco products, limiting its utility for non-Cisco users.
- High Cost for Full Access: Access to certain Talos features may require subscriptions to Cisco products, which can be costly.
- Complexity for Small Businesses: The sophistication of Talos can be overwhelming for smaller businesses that lack a dedicated security team.
- Limited Customization: Some users might find Talos' features less flexible, as it primarily aligns with Cisco's security ecosystem.
- Hardness of using: It might be hard to deal with unaware people who don't have a security background.
- Lack of information: I think the information that website gives it to me is little about URL compared to other websites like virustotal and abuseip.

2.3.6 Spam Assassin

It is an open-source, rule-based spam filtering tool that helps identify and manage spam emails on email servers. Developed by the Apache Software Foundation, it's widely used by both organizations and individuals to reduce unwanted or malicious email. SpamAssassin assigns a numerical score for every email attribute that it checks, and in the end, all the scores are added

up. Usually, positive scores indicate probable spam, while negative ones indicate that it is unlikely to be spam.

Features

- Receiving and Scanning Emails: When an email reaches the mail server, SpamAssassin grabs it and starts its spam-checking process.
- Applying Rules and Tests: SpamAssassin uses a combination of rules, lists, and statistical techniques to analyze the email.
- Scoring System: For each rule or condition that's met, SpamAssassin assigns a certain number of points. The email's total score is the sum of all the individual rule scores. If the score exceeds a specified threshold, the email is flagged as spam. For example, a message scoring over 5 points might be considered spam.
- Customization and Whitelists/Blacklists: Users can customize rules and thresholds, adding:
- Whitelist: Specific email addresses or domains that are always allowed.
- Blacklist: Addresses or domains that are always blocked.
- Handling Marked Spam: Once SpamAssassin labels an email as spam, it can take actions based on the server's setup.

Advantages

- Free and Open Source: SpamAssassin is completely free and open source, meaning you don't pay to use it and can customize it as needed. This flexibility is ideal for companies or developers who want a powerful spam filter without a subscription fee.
- Comprehensive Spam Detection: It combines multiple techniques to identify spam, including keyword detection, blacklists, and statistical analysis. This layered approach makes it highly effective, significantly reducing the chances of spam emails reaching users.
- Integration-Friendly: It integrates well with various mail servers and systems, making it easy to set up in existing email environments. This means you can add spam protection to your existing setup without a complex overhaul.
- Community and Regular Updates: As an open-source tool, SpamAssassin benefits from a community of developers who contribute updates, rules, and improvements, ensuring it keeps up with new types of spam.

Disadvantages

- Complex Setup and Configuration: Setting up and configuring SpamAssassin can be complex, especially for those without much experience in server management or email filtering. Fine-tuning its rules and thresholds to achieve optimal spam detection requires some technical expertise.
- Resource Intensive: SpamAssassin can be demanding on system resources (like CPU and memory), especially on high-traffic mail servers. Processing a large volume of emails with detailed rule analysis may slow down the server or require additional hardware.
- . Frequent Rule Updates Needed: Spam trends evolve, and SpamAssassin relies on rule sets to detect spam accurately. Therefore, these rules need regular updates to stay effective, which can be time-consuming and require ongoing management.
- Limited Out-of-the-Box Machine Learning: SpamAssassin primarily uses rule-based filtering with limited Bayesian learning. While it has a Bayesian filter, its out-of-the-box machine learning capabilities are less sophisticated compared to some newer, AI-driven spam filters, which may provide better performance in detecting complex spam patterns.

2.3.7 MX Toolbox

is a comprehensive online platform designed for email and domain management, specializing in diagnosing and troubleshooting email delivery issues. It assists businesses and IT professionals in analyzing the health and performance of their email systems. MXToolbox offers a range of tools for monitoring DNS records, checking blacklists, and assessing email server configuration to ensure reliable email delivery.

Features

- Email Blacklist Check: Allows users to check if their domain or IP address is listed on any major blacklists, helping to maintain a positive email reputation.
- DNS Lookup: Provides detailed DNS record lookups, including MX, A, CNAME, SPF, and DKIM records, essential for diagnosing email delivery issues.
- SMTP Diagnostics: Enables users to perform SMTP tests to verify email server configurations and identify potential issues affecting email flow.
- Domain Health Monitoring: Offers ongoing monitoring of domain health, alerting users to issues that may impact email deliverability.
- Email Header Analyzer: Analyzes email headers to identify routing issues and understand how emails are processed through the mail servers.

Advantages

- Enhanced Email Deliverability: By diagnosing and resolving issues related to email configuration, MXToolbox helps improve the chances of successful email delivery.
- Proactive Monitoring: Continuous monitoring of domain health and blacklist status allows organizations to take proactive measures to maintain their email reputation.
- Simplified Troubleshooting: Provides IT professionals with easy-to use tools for identifying and resolving complex email delivery problems.
- Comprehensive Reporting: Offers detailed reports that aid in compliance and documentation, ensuring that organizations maintain transparency in their email practices.

Disadvantages

- Reliance on External Databases MXToolbox heavily relies on external databases for DNS information and blacklists, drawing data on IP reputation from these sources. If these databases are outdated or contain inaccurate data, the results may be misleading or inaccurate.
- Lack of Advanced Content Analysis MXToolbox focuses primarily on inspecting system configurations and email infrastructure (like DNS records and SMTP settings) rather than analyzing the actual content of email messages.
- Absence of Predictive Alerts MXToolbox does not generally include features that predict problems before they occur, as it focuses on diagnosing existing issues with email.
- Limited Integration with Other Systems MXToolbox may be limited in terms of integration with other systems in an organization's infrastructure, reducing its effectiveness in complex environments where coordination across systems is required.
- Limited Scalability and Flexibility MXToolbox may lack sufficient scalability or customization options, making it less suitable for large organizations or those with specific requirements.
- Pricing varies based on the number of domains or IP addresses you want to monitor. Typically, subscriptions start from $10 to $50 per month, depending on the level of service and features required.

2.3.8 Whois

Whois is a query and response protocol used for querying databases that store registered users or assignees of an Internet resource, such as domain names, IP addresses, or autonomous systems. It provides information like the domain owner, contact details, and registration dates

Features

- Domain Ownership Information: Provides the name, email, and contact information of domain owners.
- Historical Data: Some services offer historical information about past owners or registration changes.
- DNS Details: Provides nameservers and related DNS information

Advantages

- Helps identify the owner of a domain for business or legal purposes.
- Useful for cybersecurity investigations to detect malicious domains Assists in domain name management and monitoring.
- Enables tracking of registration expiration dates.

Disadvantages

- Data Privacy Concerns: Information about domain owners is publicly accessible, raising privacy issues.
- Inaccurate Information: The data may not always be up-to-date or accurate.
- Not Standardized: Different registrars may provide varied levels of detail.

2.3.9 McAfee

McAfee SiteAdvisor is a web safety tool designed to help users avoid dangerous websites by providing safety ratings based on the site's reputation, the presence of malware, and potential phishing attempts. It integrates with your web browser to offer real-time protection while browsing the internet.

Features

- Real-time website ratings: Displays safety ratings for websites while you browse.
- Malware protection: Alerts users about websites that may host malicious software.
- Phishing detection: Identifies potential phishing sites and warns users before they can access them.
- Comprehensive database: Utilizes a large database of known unsafe websites to provide accurate ratings.
- User reviews: Allows users to contribute their experiences and reviews of websites.

Advantages

- Enhances online safety by identifying harmful sites.
- User-friendly interface that integrates easily with browsers.
- Regular updates ensure that the database remains current.
- Community-driven feedback adds an extra layer of protection.

Disadvantages

- May generate false positives, flagging safe sites as dangerous.
- Limited effectiveness against newly created malicious sites.
- Requires an internet connection to access its database.
- Some users report slower browsing speeds when the tool is active.

2.4 Our Work

Detection Tool for Phishing Email

Our Product is a Cyber Security Platform designed specifically for detection phishing mails. It offers a variety of features to help security analyst to detection phishing mails through full analyzing for header ang body of mail.

2.4.1 Features

- Detect Malicious URLs: Our product will identify and analyze URLs embedded within emails that enhanced with Ml. By cross-referencing URLs with known malicious domain databases and examining for URL obfuscation techniques (like shortened or redirected links), the tool will accurately flag phishing links and warn users before they click on potentially harmful websites.
- Analyze Suspicious Attachments: Phishing emails often include malicious attachments.

  This tool will scan attachments for malware signatures, suspicious file types (such as .exe, .js, or macro-enabled documents), and other risky behaviors. By integrates with multiple threat intelligence sources that have Sandbox, such as VirusTotal. This integration allows us to detect attachments before they can harm the system.
- Check Authentication Signatures (SPF, DKIM, DMARC): Our product will validate key email authentication protocols, including SPF (Sender Policy Framework), DKIM (DomainKeys Identified Mail), and DMARC (Domain-based Message Authentication, Reporting, and Conformance). These protocols are essential for verifying if an email

originated from an authorized source. Failure in these checks will signal the possibility of a phishing attempt.

- WHOIS Lookup for Sender Domain: Perform WHOIS lookups on a sender domain to get registration details or records, which can be used to assess the legitimacy of a domain.
- Blacklisting Service Integration: Check if the domain or embedded URL in mail body is blacklisted by major anti-virus services or threat intelligence platforms.
- Check SSL certificate for url.

2.4.2 Advantages

- Provide full analysis for mail body and header.
- Offers detailed reports that aid in compliance and documentation, ensuring that organizations maintain transparency in their email practices.
- Provide Difference Resources that our product deals with multiple software to detect and search user's input.
- Provide product as website and application.
- Steganography for (video, photos).

Table 2.4 Comparison

	Outlook	Gmail	Virus Total	Talos Intelligence	Spam Assassin	MX Toolbox	McAfee	Phish Tool	Our product
Provide full analysis for mail body and head	✓	✓	✗	✗	✗	✓	✗	✓	✓
Offers detailed reports	✓	✗	✓	✗	✗	✗	✓	✗	✓
Integration with threat intelligence platforms	✗	✓	✓	✓	✓	✓	✓	✓	✓
Provide product as application	✓	✓	✓	✗	✗	✗	✗	✗	✓
Automated phishing detection solution with ML	✓	✗	✗	✓	✓	✗	✓	✓	✓
Predictive Alerts	✗	✓	✗	✗	✗	✗	✗	✗	✗

Table 1: Comparison

Chapter (3)

Chapter 3: System Analysis and Design

3.1 Introduction

In this chapter we will provide the system analysis and design process that included in our system that include functional and non-functional requirement and UML diagrams.

3.2 Development methodology

Unified Modelling language (UML) is a standardized mode ling language enabling developers to specify, visualize, construct and document artifacts of a software system. Thus, UML makes these artifacts scalable, secure and robust in execution. UML is an important aspect involved in object-oriented software development. It uses graphic notation to create visual models of software systems.

3.3 Functional requirements

Table 3.1 Functional requirements:

Functional requirements	
Email Analysis	Extracts email details (headers, body, attachments).
URL Scanner	Analyze and classifies URLs.
Attachment Scanner	Scans attachments for malicious content.
IP Checker	Checks sender IP against Databases.
Phishing Detector	Combines all analysis to determine if an email is phishing.
Report Generator	Creates reports based on detection results.
API Integrator	Connects with external services like VirusTotal, PhishTank.

Table 2: Functional requirements

3.4 Non-Functional requirements

Non-Functional requirements	
Security	Ensure end-to-end encryption for all communication within the application.
Usability	The application interface should be user-friendly
Performance	Data transfer between devices should be completed in real-time.
Portability	The system should be accessible across devices, including mobile, desktop, and tablets.
Maintainability	The system should allow easy updates for new phishing detection patterns and evolving threats.
Compliance	Must adhere to GDPR or other regional data protection laws to safeguard user data.
Scalability	The application should be designed to accommodate a growing number of users and data.

Table 3: Non-Functional requirements

3.5 Use Case Diagram

A use case diagram provides a high-level view of how the system interacts with users and external entities. In this scenario, we can identify two primary actors:

1. User (e.g., individuals or businesses using the system).
2. Admin (e.g., system administrator managing the application).

1. User Use Case Diagram

This section represents the activities that the User can perform in the system:

1. **Register**: Allows the user to create a new account in the application.
2. **Login**: Used for authentication to enter the system.
3. **Write IP**: Allows the user to input an IP address for analysis.
4. **Give URL**: Enables the user to submit URLs (links) to check for phishing activity.
5. **Upload Email File**: Users can upload email files for phishing detection.
6. **Give Header**: Users provide email headers for analysis to detect phishing patterns.

7. **Make Review**: Allows users to provide feedback or reviews on the system's performance or results.
8. **Input domain**: user can write the domain of sender and app detect it for user.

---

2. Admin Use Case Diagram

This section represents the activities that the admin can perform in the system:

1. **Login**: Used for authentication to access the admin panel.
2. **Add Admin**: Allows the admin to add new administrators to the system.
3. **Remove Admin**: Enables the admin to remove an existing administrator.
4. **Edit Feature**: Allows the admin to update or modify system features.
5. **Edit Algorithm**: Admins can adjust or improve the algorithms used for detecting phishing emails.

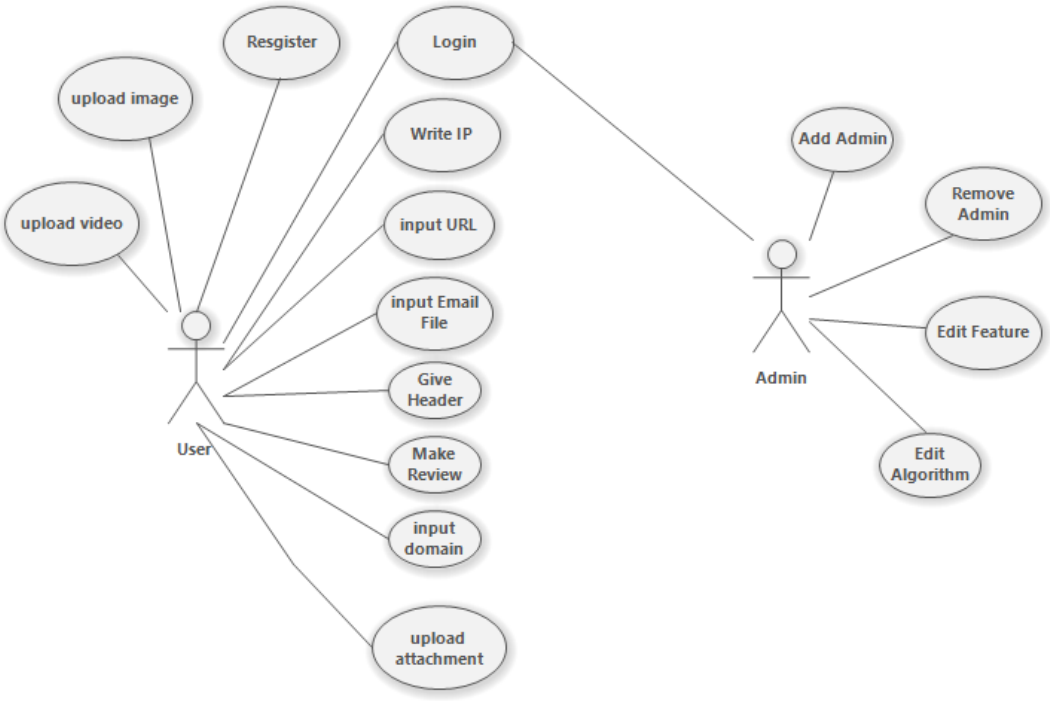


Figure 3 : Use case Diagram

3.6 Sequence Diagram

Sequence diagrams play a pivotal role in our software development process, serving as a robust visual aid that offers a comprehensive depiction of the dynamic behaviour and intricate interactions among objects or components within our system. These diagrams act as a window into the chronological sequence of messages exchanged between objects, showcasing their collaborative efforts to achieve specific functionalities. By presenting a clear visualization of the flow of control, sequence diagrams empower us to delve into the intricate details of method invocations, parameter passing, and the precise order of events.

One of the key advantages of sequence diagrams lies in their ability to illuminate the interactions between different elements of our system. Through their graphical representation, sequence diagrams facilitate the identification of potential bottlenecks, ensuring that performance issues and inefficiencies can be promptly addressed. Moreover, they provide invaluable insights into system behaviour, allowing us to gain a deeper understanding of how various components interconnect and function together.

In addition to their analytical capabilities, sequence diagrams also serve as an indispensable communication and documentation tool throughout the software development life cycle. Developers, designers, and stakeholders alike can leverage these diagrams to effectively collaborate and cultivate a shared understanding of system behaviour. By showcasing the temporal ordering of activities and the exchange of information between objects, sequence diagrams foster clarity and facilitate effective communication among team members.

User Login and Phishing Detection: This sequence diagram illustrates the login process and the phishing detection flow after the user accesses the system.

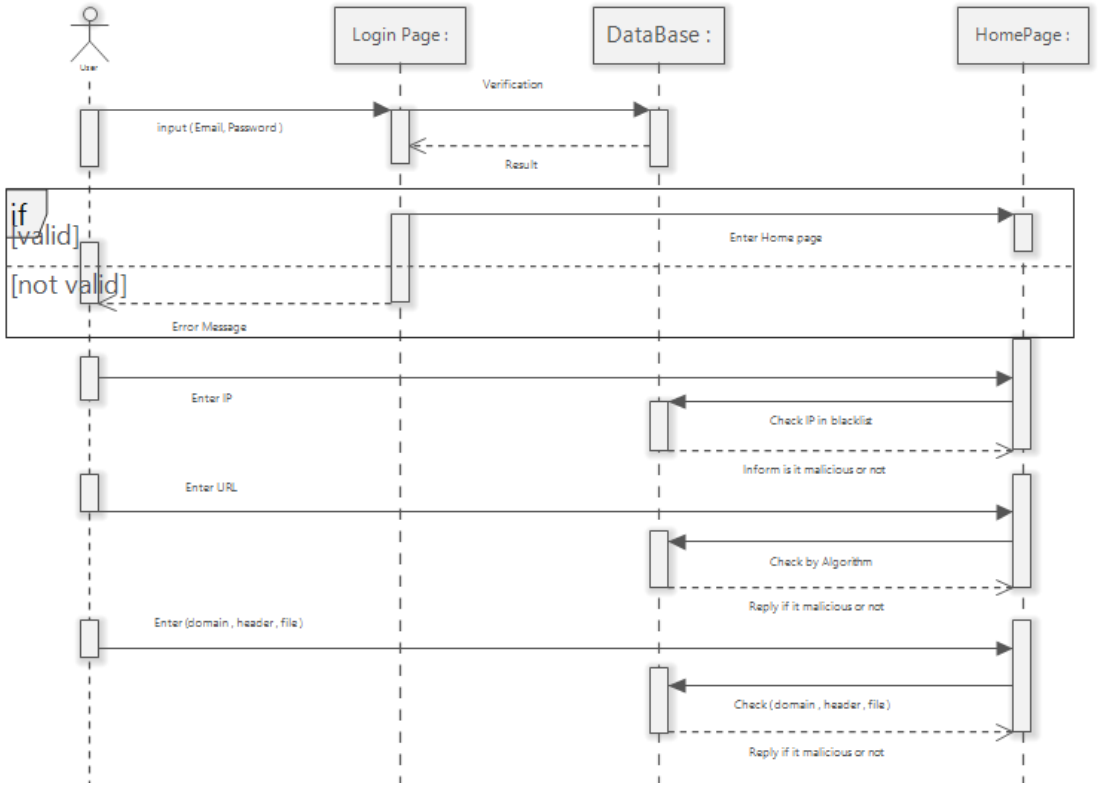


Figure 4 : Login Sequence Diagram

User Sign-Up Process: This diagram represents the user sign-up process and highlights how the system validates new users.

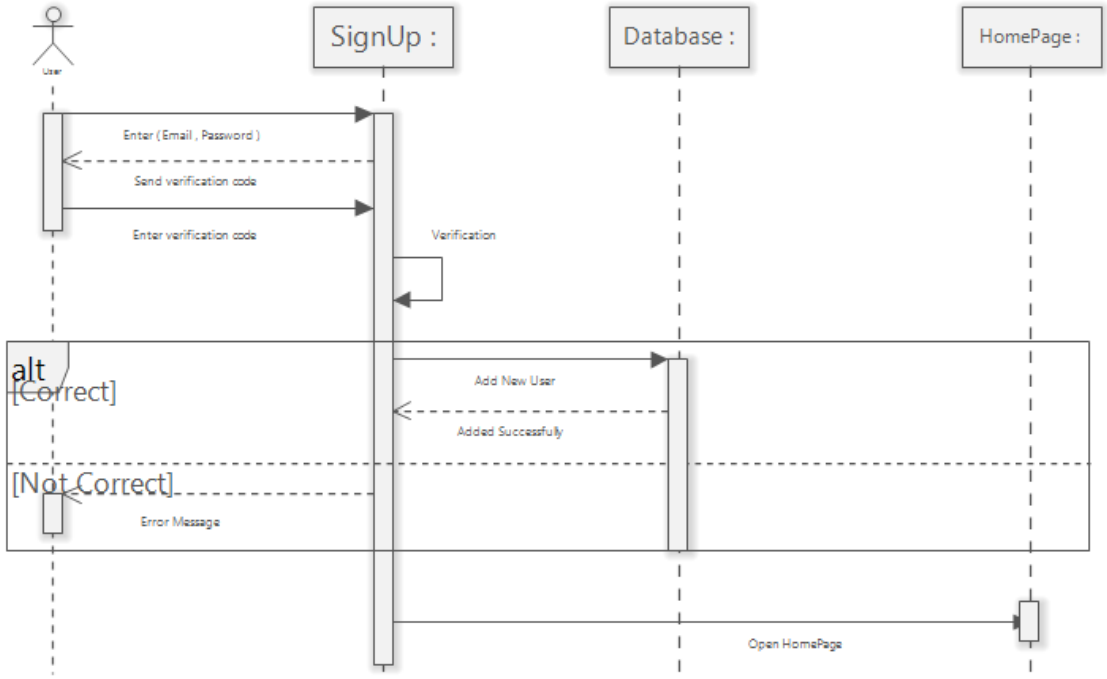


Figure 5 : Signup Sequence Diagram

3.7 Class Diagram

Class design phase is a fundamental step in our software development process, focusing on the detailed implementation of individual classes within our system. During this phase, we define the internal structure and behaviour of each class, including attributes, methods, visibility, and relationships. By meticulously considering the requirements and specifications, we make informed decisions about data types, algorithms, and design patterns that best align with our system's objectives. The Actual Class design stage bridges the gap between the high-level conceptual model and the actual coding and implementation. It involves translating abstract representations, such as class diagrams, into concrete and executable code. Through the Actual Class design, we ensure that our classes are optimized, modular, and maintainable while adhering to coding standards and best practices. This phase plays a crucial role in transforming our software architecture into a functional and reliable system, providing developers with clear guidelines for coding and promoting consistency and scalability across the project.

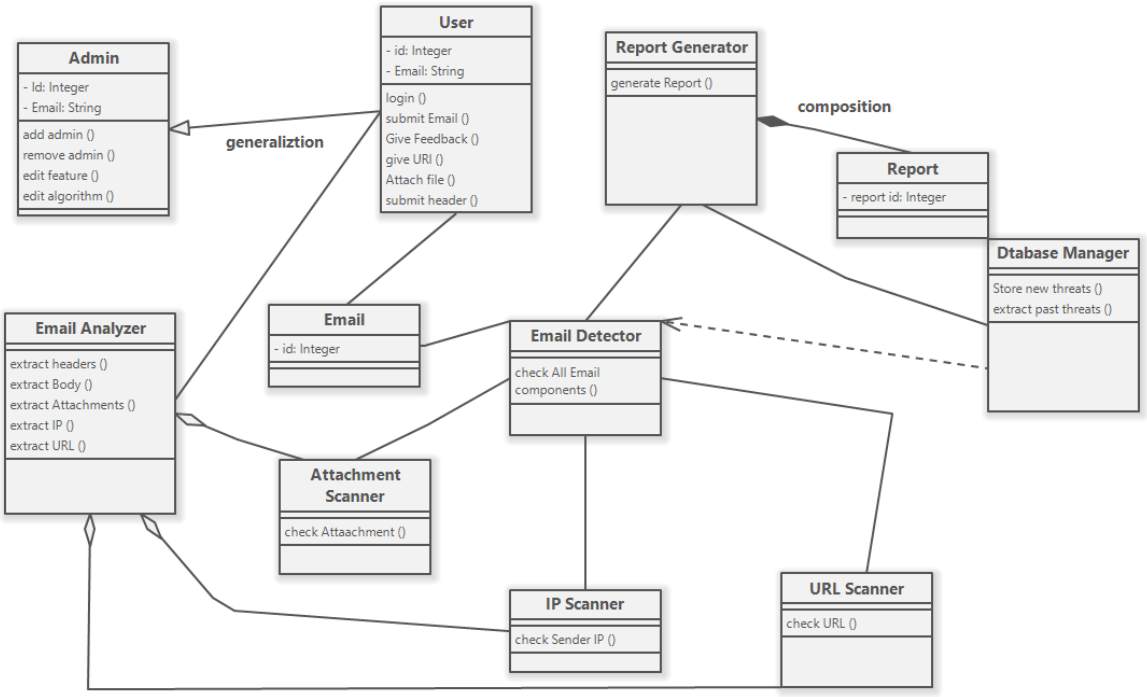


Figure 6 : Class Diagram

3.8 Activity Diagram

The activity diagram illustrates the phishing email detection process. It begins with the user accessing the system's homepage. The user must log in or register if they are new to the system. After logging in, the user uploads an email or provides its content, such as headers, attachments, or URLs. The system then analyzes the input using its detection algorithm. If the email is flagged as phishing, the system informs the user with detailed results. If it is safe, the system displays a "no threat detected" message. This diagram effectively captures the sequential steps and decision points involved in detecting phishing emails.

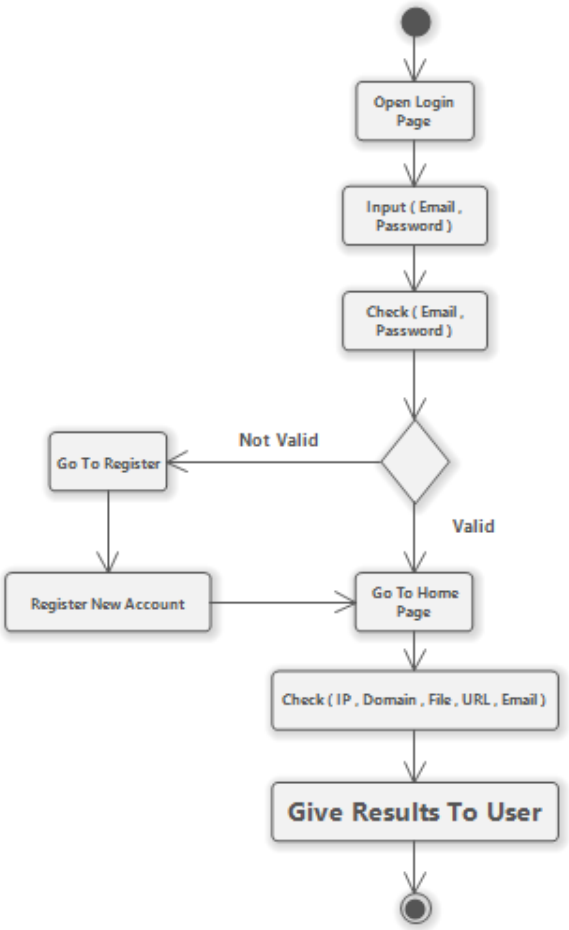


Figure 7 : Activity Diagram

3.9 Database schema

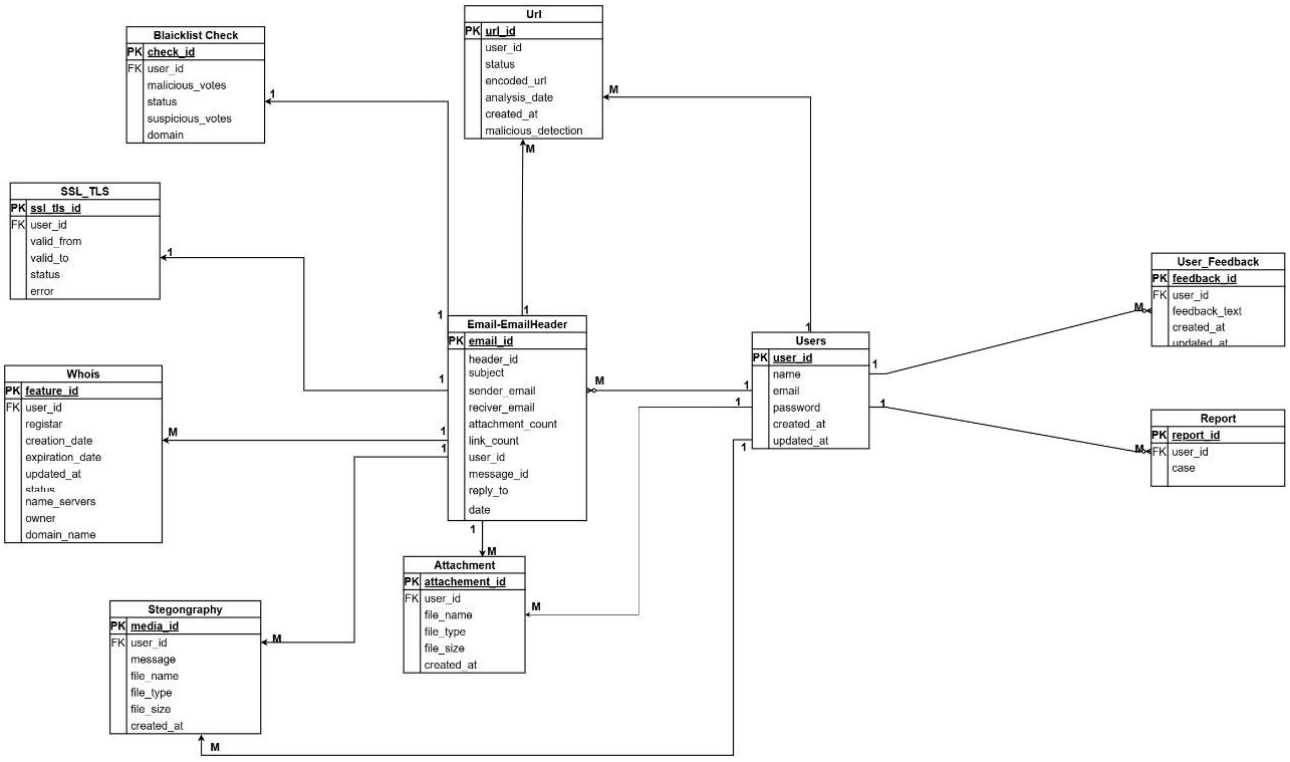


Figure 8 : Database Schema

3.10 Business Model

A well-designed and robust business model serves as the foundation for sustainable growth and success in today's competitive marketplace. It outlines the strategic framework and operational structure that a company employs to create, deliver, and capture value. A professional business model integrates various elements, including customer segments, value propositions, channels, customer relationships, revenue streams, key activities, resources, and partnerships. By meticulously analyzing customer needs and market trends, a business model enables organizations to identify and target lucrative customer segments effectively. It delineates the unique value propositions that differentiate a company from its competitors, demonstrating how it solves customer problems or fulfils their desires. Furthermore, a business model delineates the most effective channels and customer relationships through which a company delivers its products or services to its target audience. It encompasses revenue streams, depicting the different ways a business generates income, such as product sales, subscriptions, licensing, or advertising. The key activities and resources required to operate the business are identified, along with the strategic partnerships that enhance capabilities or extend reach. A professional

business model is dynamic, adaptable, and aligned with the company's overall vision, enabling it to navigate changing market conditions and seize new opportunities. By leveraging a well-crafted business model, organizations can optimize their operations, drive innovation, and foster long-term profitability while consistently delivering value to their stakeholders.

Our system is a phishing email detection tool designed to analyze and identify suspicious content such as headers, URLs, attachments, and other elements within emails. It serves as an analytical solution, offering users clear and accurate results on whether an email is malicious or legitimate. Unlike protective tools, we do not block or mitigate threats directly; instead, we empower users with actionable insights to make informed decisions, ensuring clarity and transparency in threat assessment.

Figure 9 : Business Model

Chapter (4)

Chapter 4: System Design

In this chapter, we present a comprehensive analysis and design of the system, focusing on its architecture, functionality, and user interface. The system aims to detect phishing emails by analyzing various elements such as email headers, body content, attachments, and URLs. This chapter provides an in-depth exploration of the system's components, workflow, and design choices, ensuring a seamless user experience. The user interface (UI) is designed with simplicity and efficiency in mind, allowing users to navigate and utilize the system with ease. Through this analysis, we establish a solid foundation for the system's implementation and future enhancements.

4.1 System Architecture

Our system analyzes phishing emails using multiple security modules. Users can upload emails, attachments, URLs, images, and videos. The system checks for hidden threats, malware, and suspicious domains.

Key components include:

- **Steganography Check:** Detects hidden data in images and videos.
- **URL & Attachment Scanner:** Identifies malicious links and files.
- **SSL & Domain Check:** Verifies SSL certificates and domain legitimacy.
- **Email Header Analyzer:** Extracts and inspects sender info.
- **Blacklist & DNS Check:** Ensures domain is not blacklisted and validates SPF, DKIM, and DMARC.
- **Detailed Report:** Summarizes all findings for user review.

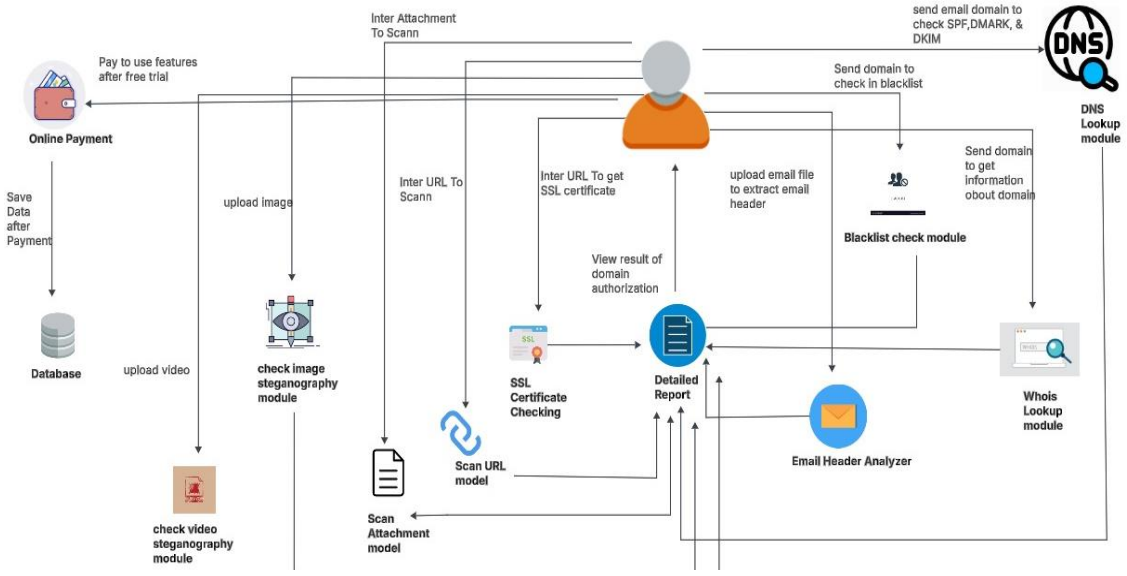


Figure 10 : System Architecture

4.2 Block Diagrams

At this section we will know and learn the process and the flow of each feature in our system

4.2.1 The process of scanning URLs

Scan with ML: This feature uses machine learning to evaluate the risk of URLs contained in emails. It applies a Random Forest algorithm trained on features such as URL structure, length, domain reputation, and lexical patterns to classify links as safe or potentially malicious. In our phishing detection project, this helps proactively identify suspicious URLs that may not yet be listed in threat databases.

Scan URL with VirusTotal: Queries VirusTotal API; delivers VT Reputation Score and multi-engine scan results.

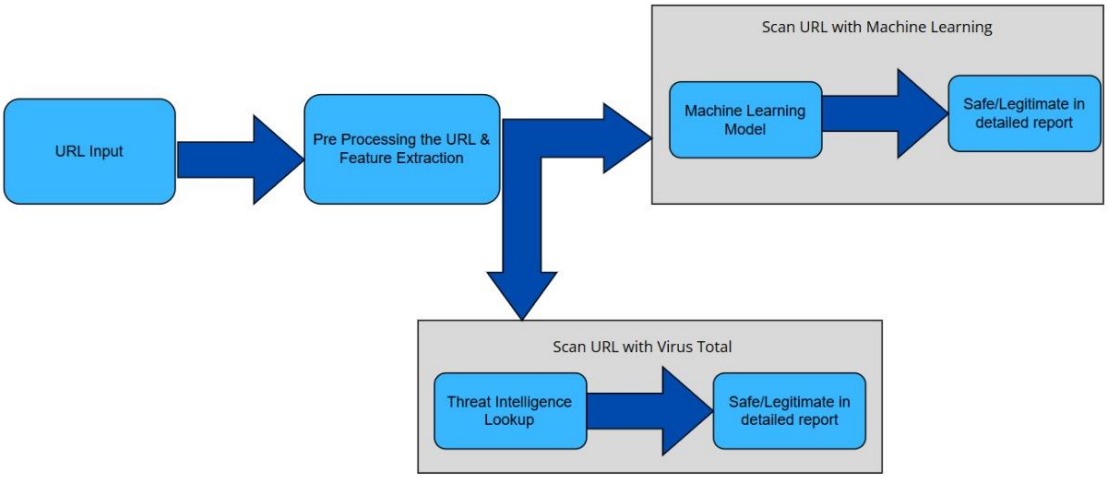


Figure 11 : process of scan url

4.2.2 The process of scanning Header

This feature extracts key metadata from the email headers, including fields like From, To, Reply-To, and Subject. These fields are essential for analyzing the sender's identity and identifying mismatches or anomalies. In our phishing detection system, we use this data to detect spoofed

addresses or suspicious reply paths that may indicate phishing. For example, if the Reply-To address is different from the From address, it could be a sign of a phishing attempt.

Figure 12 : process of extract email header

4.2.3 Attachment Scanning Feature

This feature automatically analyzes email attachments by scanning them with multiple antivirus engines and reputation databases to detect malware or harmful content. In our phishing detection project, this helps identify malicious files that could compromise the recipient's system. If an attachment is flagged as suspicious or infected, the email is classified as high risk.

Figure 13 : process of scanning attachment

4.2.4 SSL Certificate Scanning Feature

This feature checks whether the URLs included in an email use SSL/TLS encryption (i.e., start with HTTPS). SSL (Secure Sockets Layer) and TLS (Transport Layer Security) are protocols that secure data transmission between the user and the website. In our phishing detection project, this helps assess whether links point to secure, legitimate sites. If URLs do not use SSL/TLS (HTTP only) or have invalid certificates, it increases the risk that the link is malicious or part of a phishing attack.

Figure 14 : process of SSL certificate

4.2.5 Check Domain in Blacklist

This feature checks the reputation of URLs included in emails by scanning them against multiple threat intelligence and malware detection engines. It helps identify links that may lead to phishing websites, malware downloads, or other malicious activities. In our phishing detection project, if a URL is flagged as suspicious or associated with known threats, the email is marked as potentially dangerous.

Figure 15 : process of check domain in blacklist

4.2.6 Whois Lookup for Domain

This feature performs a WHOIS query to retrieve registration information about the domain included in an email. It shows details such as the domain owner, creation date, and expiration date. In our phishing detection project, this helps evaluate whether a domain is newly registered, suspicious, or linked to known malicious activity. If the WHOIS data is missing, hidden, or indicates a very recent registration, it can be a strong indicator that the email is part of a phishing attempt.

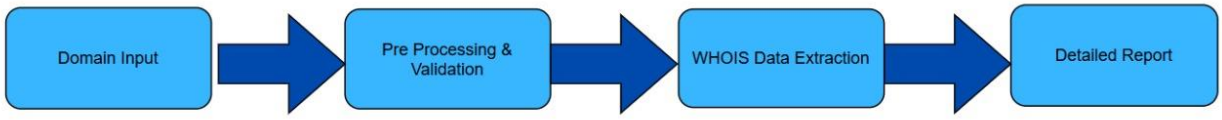


Figure 16 : process of whois lookup

4.2.7 SPF & DMARC & DKIM Analysis

This feature verifies whether an incoming email passes SPF (Sender Policy Framework), DKIM (DomainKeys Identified Mail), and DMARC (Domain-based Message Authentication, Reporting, and Conformance) checks. These protocols are essential to confirm the legitimacy of the sender's domain and prevent email spoofing. In our phishing detection system, this helps identify forged emails pretending to come from trusted domains. If these records are missing or invalid, it increases the likelihood that the email is fake or part of a phishing attempt.

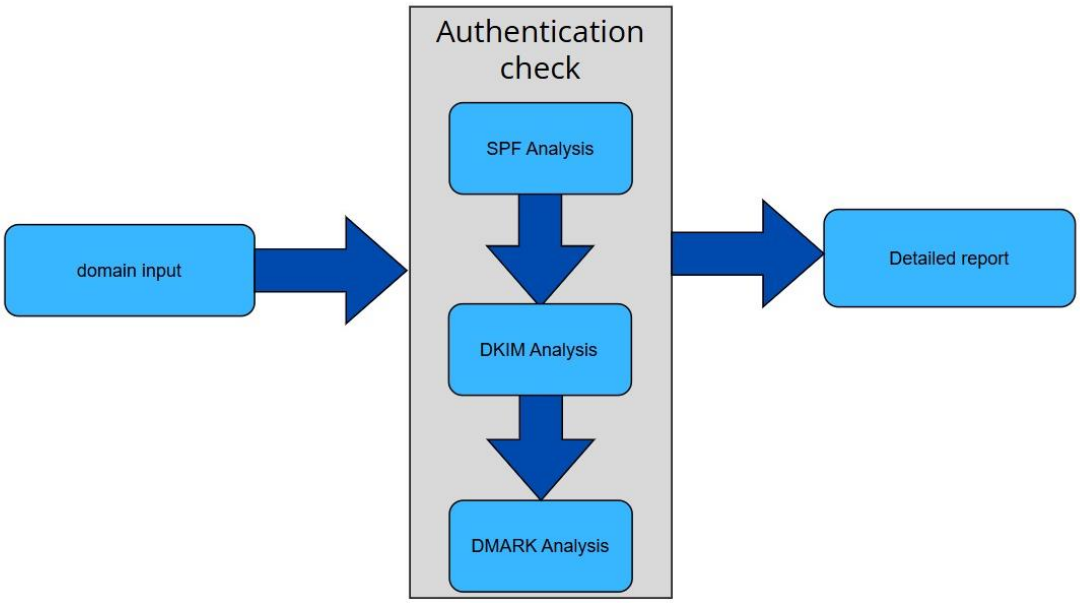


Figure 17 : process of SPF, DMARC, DKIM

4.2.8 Image & Video steganography

This feature uses a steganography analysis library to scan images and videos attached to emails for hidden data. Steganography techniques embed malicious content or secret messages within multimedia files without visibly altering them. In our phishing detection project, this helps identify files that may conceal harmful payloads or covert communication channels. If hidden

data is detected, the system flags the email as suspicious or potentially malicious.

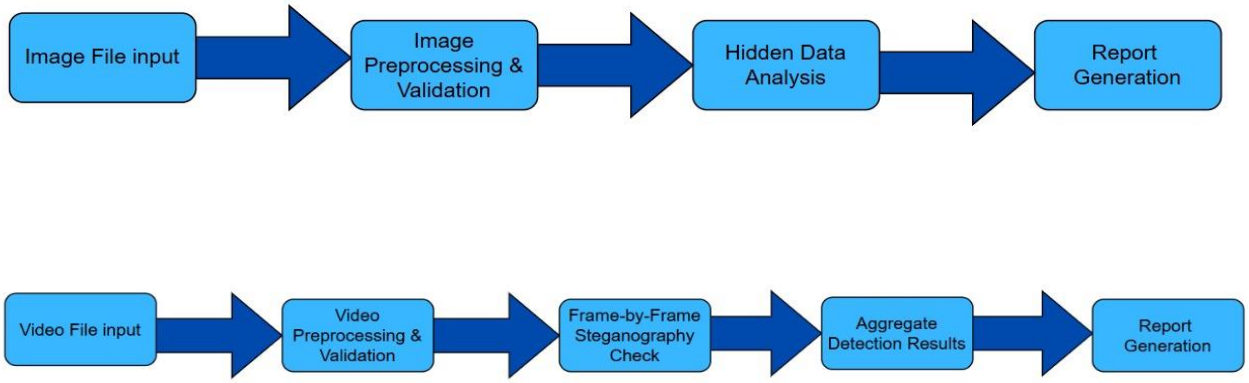


Figure 18 : image & video steganography

4.2.9 Full Email analysis

DKIM, and DMARC validation, scans attachments and URLs for threats, analyzes URL reputation with AI, inspects headers, and detects steganography in media files. In our phishing detection project, this provides an in-depth assessment of each email's risk level. Access to the full analysis requires a premium subscription.

It helps security teams quickly identify high-risk emails and take preventive action. This feature is designed to improve detection accuracy and reduce the chance of successful phishing attacks.

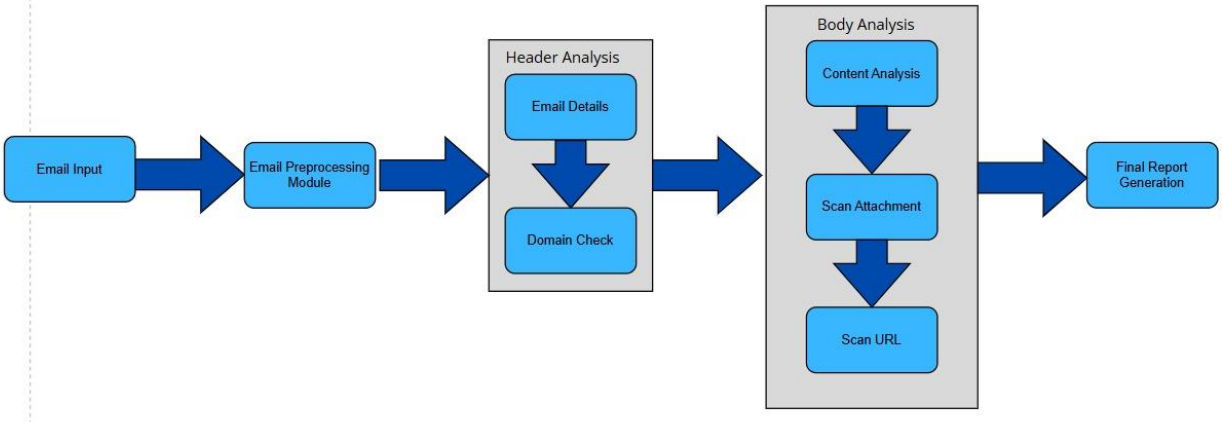


Figure 19 : process of full email analysis

4.3 Software application design

Software Application Design involves gathering and analyzing the customer business functions, then designing an application solution to meet the prioritized business requirements. The first step in this service is to establish a charter, which defines the scope and responsibilities for this service. The final step is a presentation of options for meeting customer business. The first step in this service is approval to proceed with website and mobile application so we try to focus on the look and feel of both. The design stage encompasses several different aspects, including user interface design (UI), usability (UX), content production, and graphic design. Within this post, we focused mainly on UI and UX design. The best application is chosen based on design, Usability, and Creativity.

4.4 User Interface & User Experience Design

UI stands for User Interface. UI is the part of the mobile app which a user interacts with. In simple terms, it's everything you see and touch, such as buttons, colours, fonts, navigation, etc. UX stands for User Experience. UX focuses on how the users feel towards the application, and their experience using it. Was the web site or mobile application hard to use, was it slow, was the user disappointed when using it? How will users interact with screens and view? These are the types of questions a UX designer will focus on when reviewing and working on a website and mobile application.

4.4.1 Phirus Logo

Figure 20 : Phirus Logo

4.4.2 User Interface screens (mobile app)

Splash screen

Phirus's logo will appear after opening the app for a few seconds then start with the application.

Figure 21 : Splash Screens

Page view

It's information for the user to explain the app's goals and services that can also include:

- Diagnose with image classification and survey.
- 2D and 3D activities that help a child to improve his skills.
- Advice and doctor's information for parents.

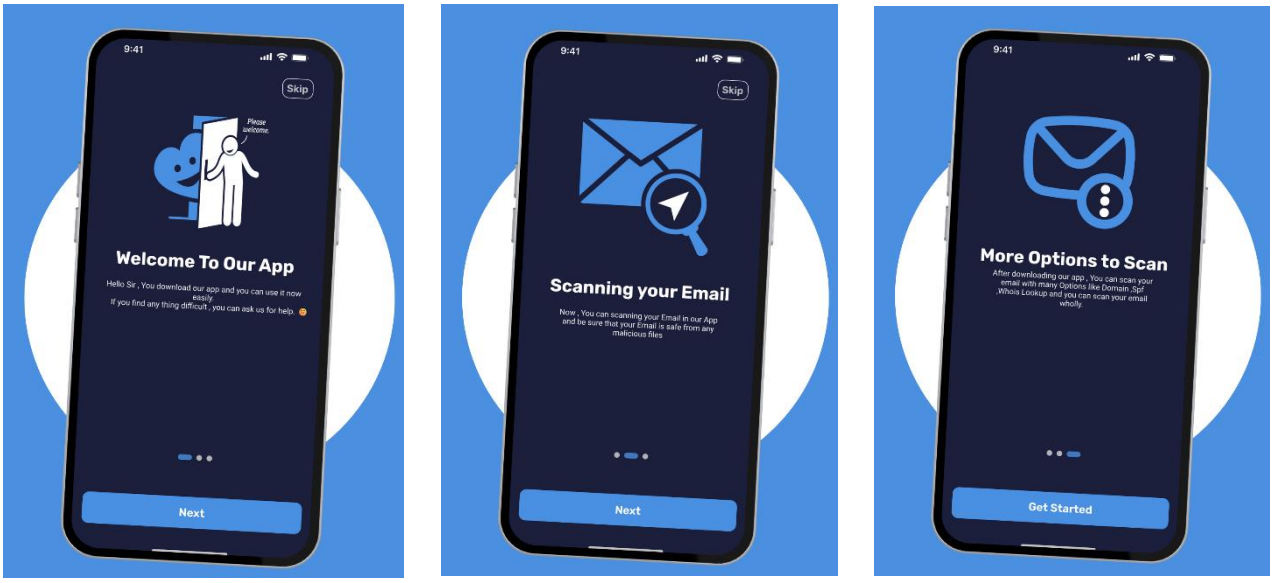


Figure 22 : Page View

Log in or Sign-up page

After the guide screens, the user will choose to sign up or sign in if they already have an account. Or they can skip and go to the main page.

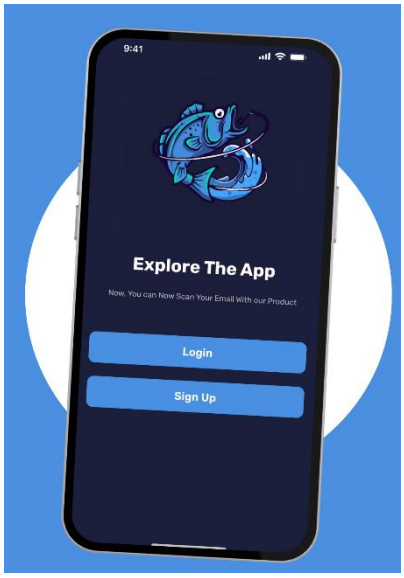


Figure 23 : Login or Signup

Sign-up page

Users can sign up in the application by entering their name, email, and password.

After creating Account user receives a verification code to ensure his account.

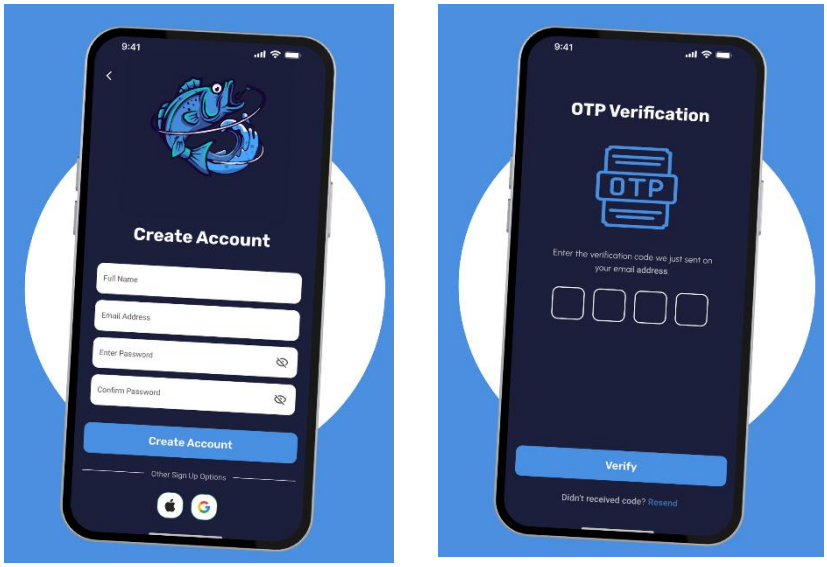


Figure 24 : Signup

Log in page:

If the user has an account, he can sign in by entering email and password.

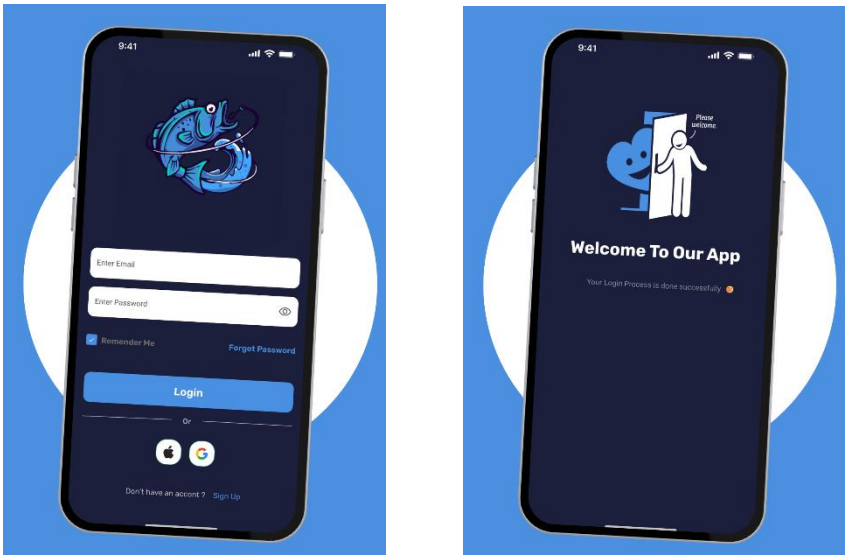


Figure 25 : Login page

Forget password:

- If user forgets the password, he can reset his password by using his email.
- A verification code will be sent to your chosen contact information.

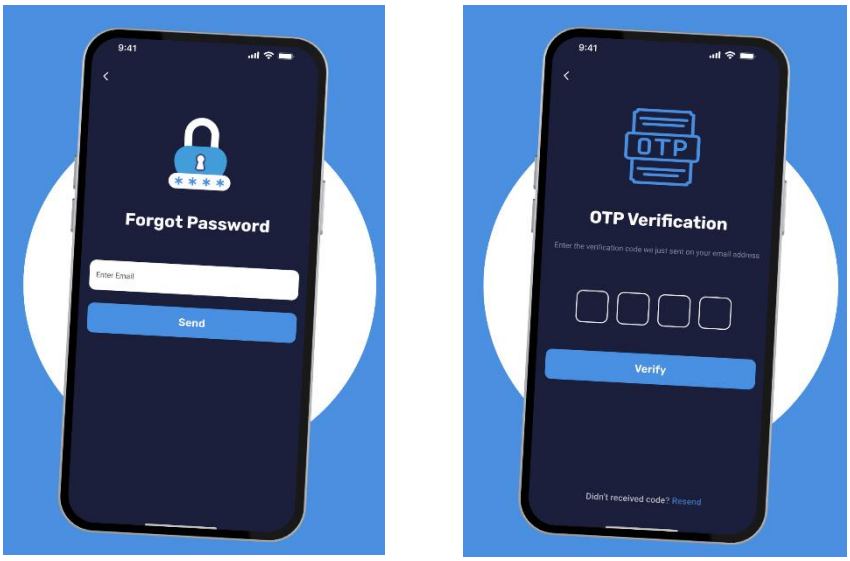


Figure 26 : Forgot Password

Create new password:

here user inputs new password and get changed successfully.

Create a new, strong password and confirm it to complete the process.

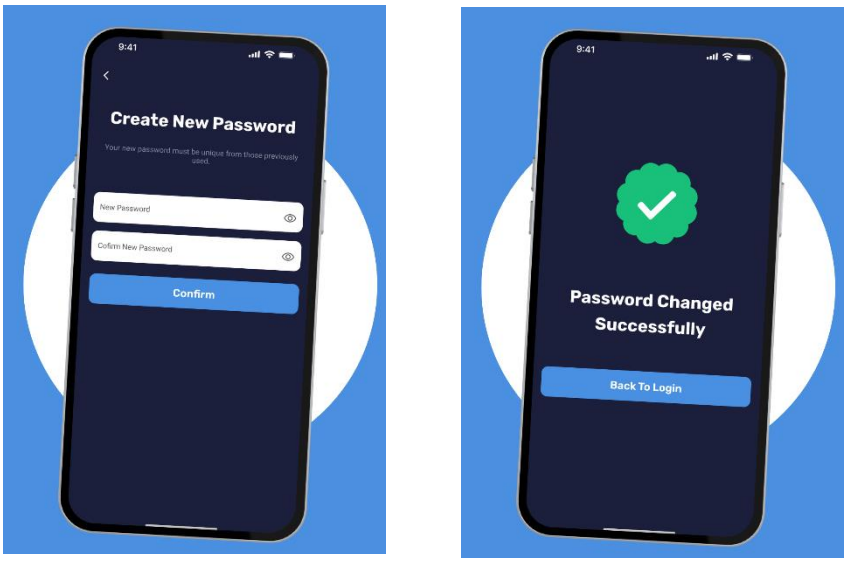


Figure 27 : Create new password

Home Page:

After successfully sign in or sign up he faces now home page.

Here user can see Body and Header checking and chooses what he wants.

Figure 28 : Home Page

Header page:

In this section, if user wants to check email header, he can click on header button, and he will see multi options in the app: Whois lookup, SPF&DMARK&DKM check, Check domain in blacklist, Extract header information.

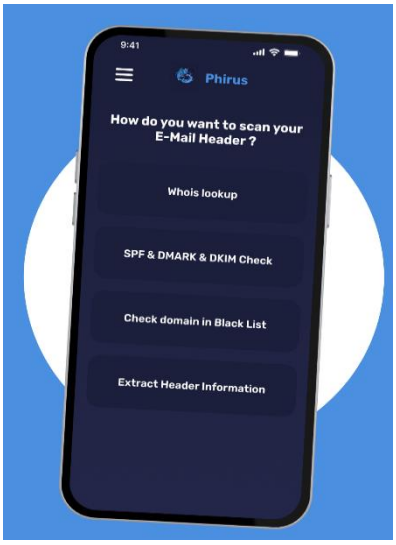


Figure 29 : Header Page

1. Whois Lookup:

If user chooses this option, he can enter domain he wants to check then can receive result Around domain information related to whois lookup.

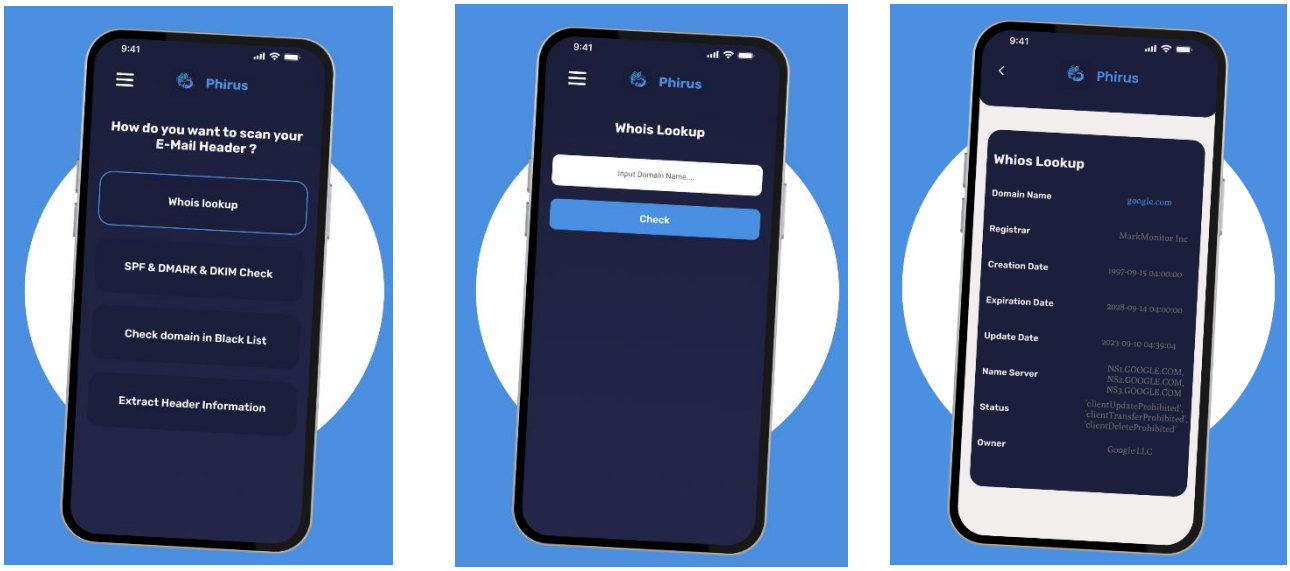


Figure 30 : Whois Lookup

2. SPF & DMARK & DKM Check:

If user chooses this option, he can enter domain he wants to check then can receive result Around domain information related to SPF & DMARK & DKM Check.

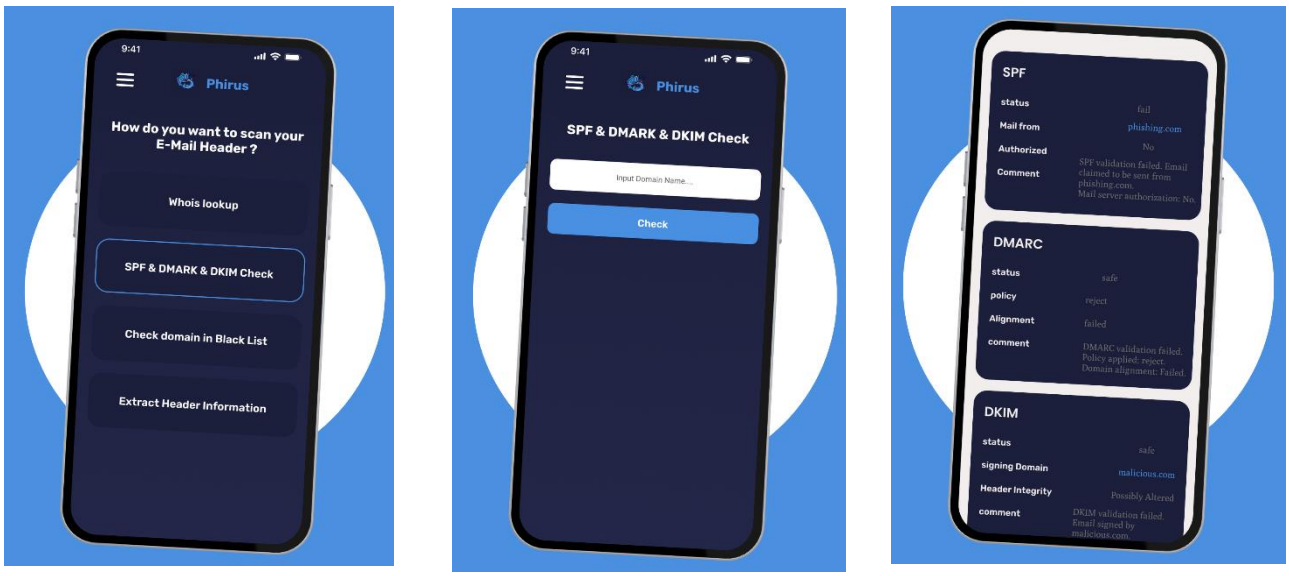


Figure 31 : SPF, DMARK, DKIM

3. Check domain in blacklist:

If user chooses this option, he can enter domain he wants to check then can receive result Around domain information related to if domain is in blacklist or no.

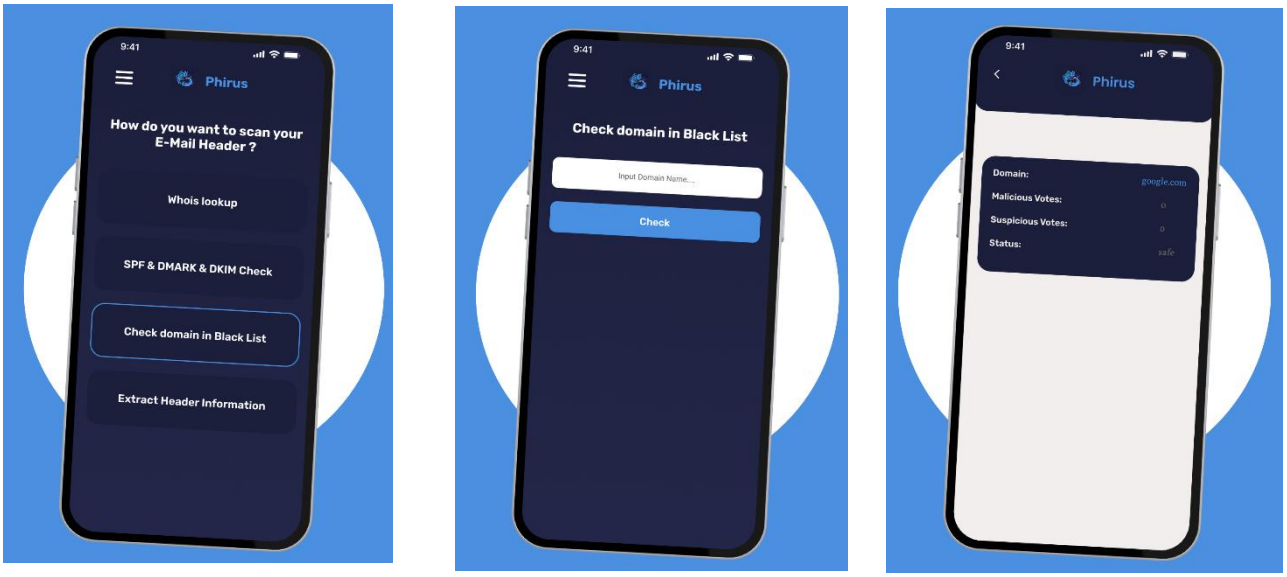


Figure 32 : Check domain in blacklist

4. Extract Header Information:

If user chooses this option, he can enter domain he wants to check then can receive result Around domain information related to extract header information.

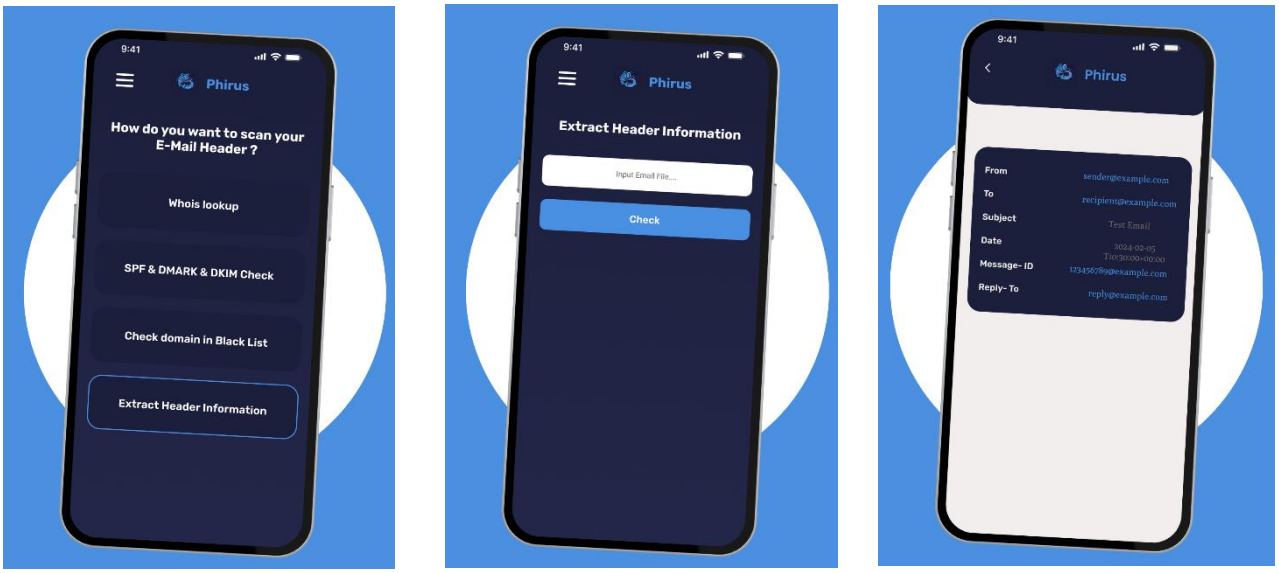


Figure 33 : Extract header information

Body Page:

In this section, if user wants to check email body, he can click on email button, and he will see multi options in the app: Check Attachment, Check URL, Check Steganography.

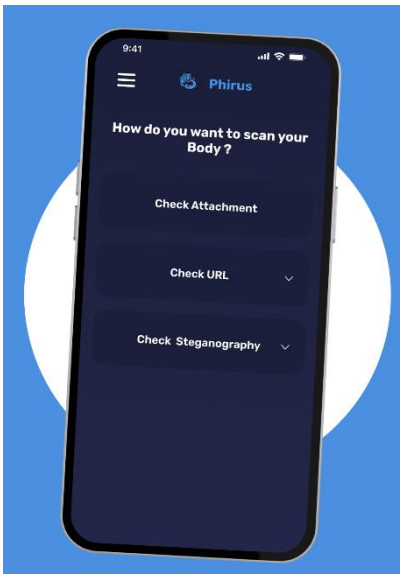


Figure 34 : Body Page

1. Check Attachment:

if user chooses this option, he can upload attachment he wants to check, then he can receive results about check attachment.

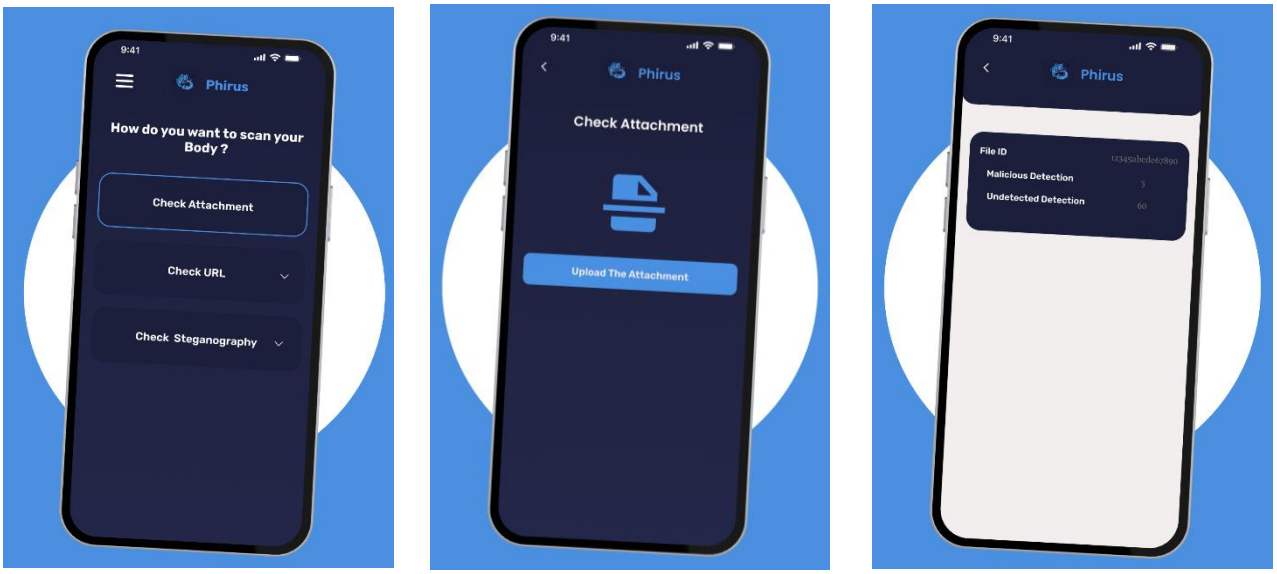


Figure 35 : Check Attachment

2. Check URL:

This option contains 2 options (Scan URL, Check SSl Certificate).

Scan URL:

Here, User input URL he wants to check, then receive results about this scanned URL.

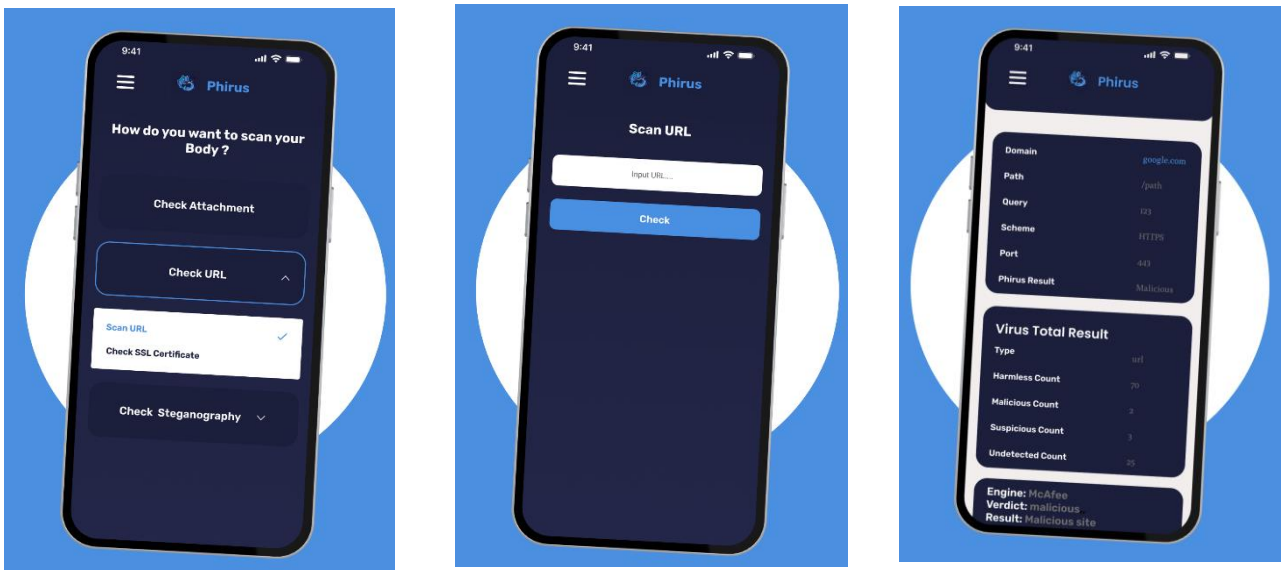


Figure 36 : Scan URL

Check SSL Certificate:

Here, User input URL he wants to check its SSL Certificate, then receive results about this scanned URL.

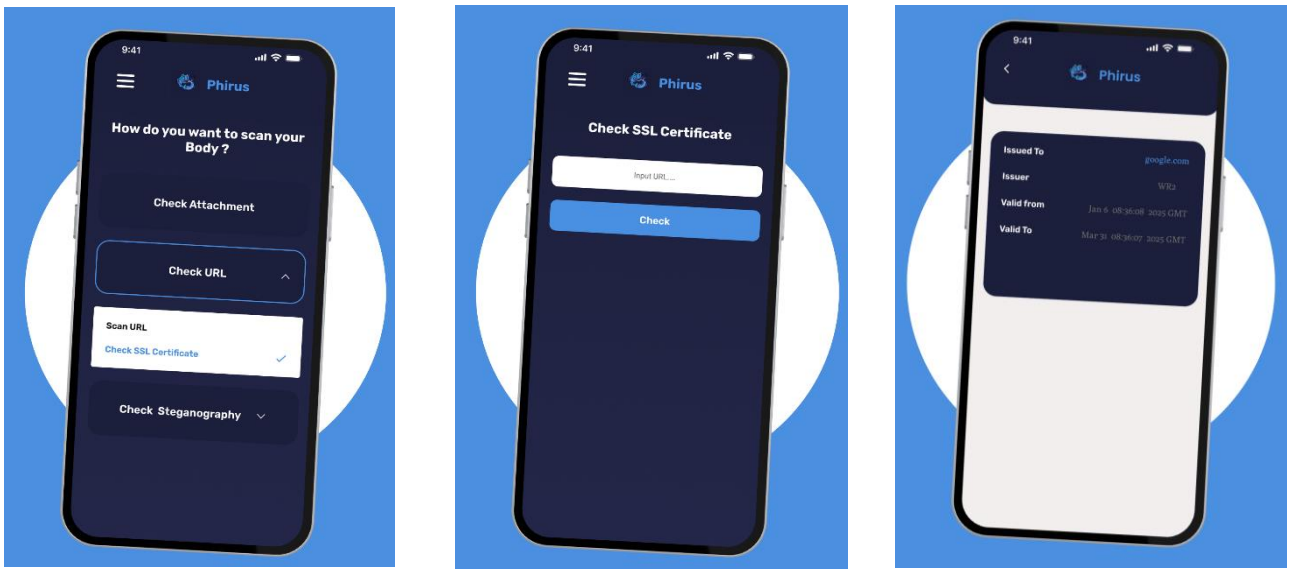


Figure 37 : Check SSL Certificate

2. Check Steganography:

This option also contains 2 options (Image Steganography, Video Steganography).

Image Steganography:

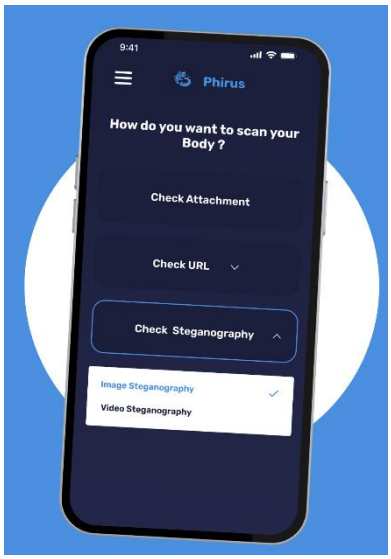


Figure 38 : Image Steganography-1

Here user can upload the image he wants to check and the result shows if there is hidden message in image or not.

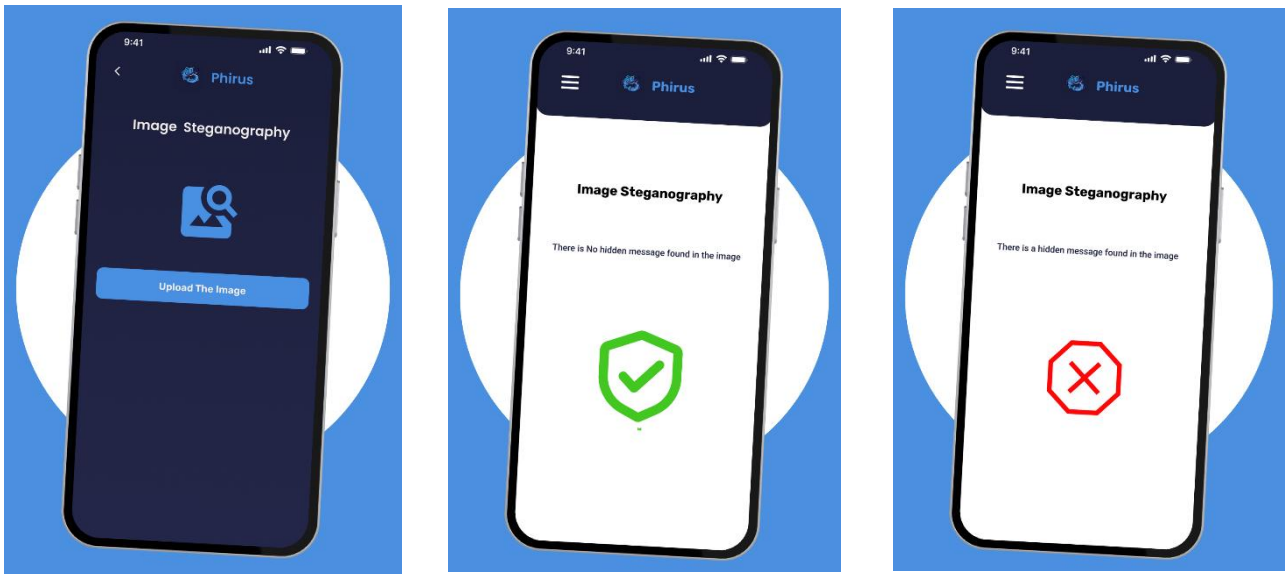


Figure 39 : Image Steganography-2

The second option is

Video Steganography:

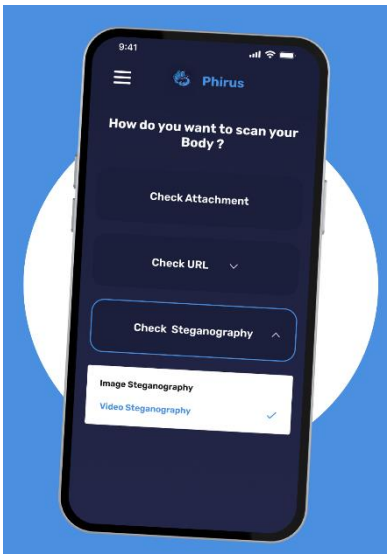


Figure 40 : Video Steganography-1

Here user can upload the video he wants to check and the result shows if there is hidden message in video or not.

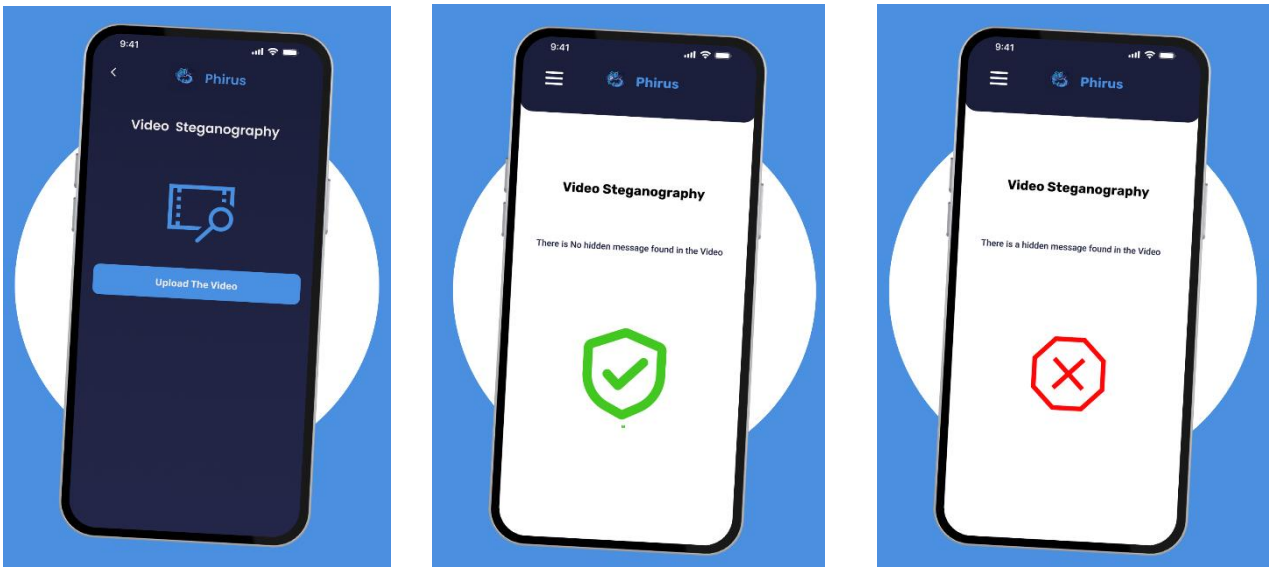


Figure 41 : Video Steganography-2

The Last Part of our Application is the Educational Content for learning:

In this part the user can watch courses related to cyber security field which may be videos, articles or research. at the end he can give the feedback for course and program as whole

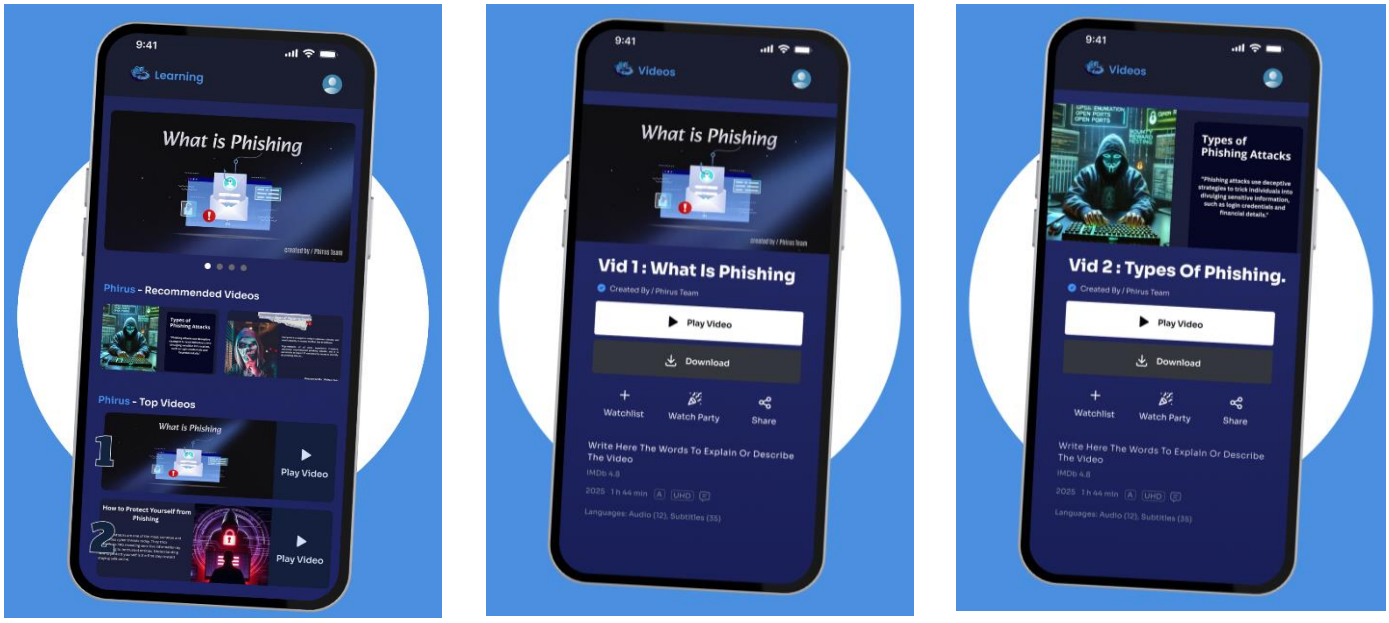


Figure 42 : Learning-1

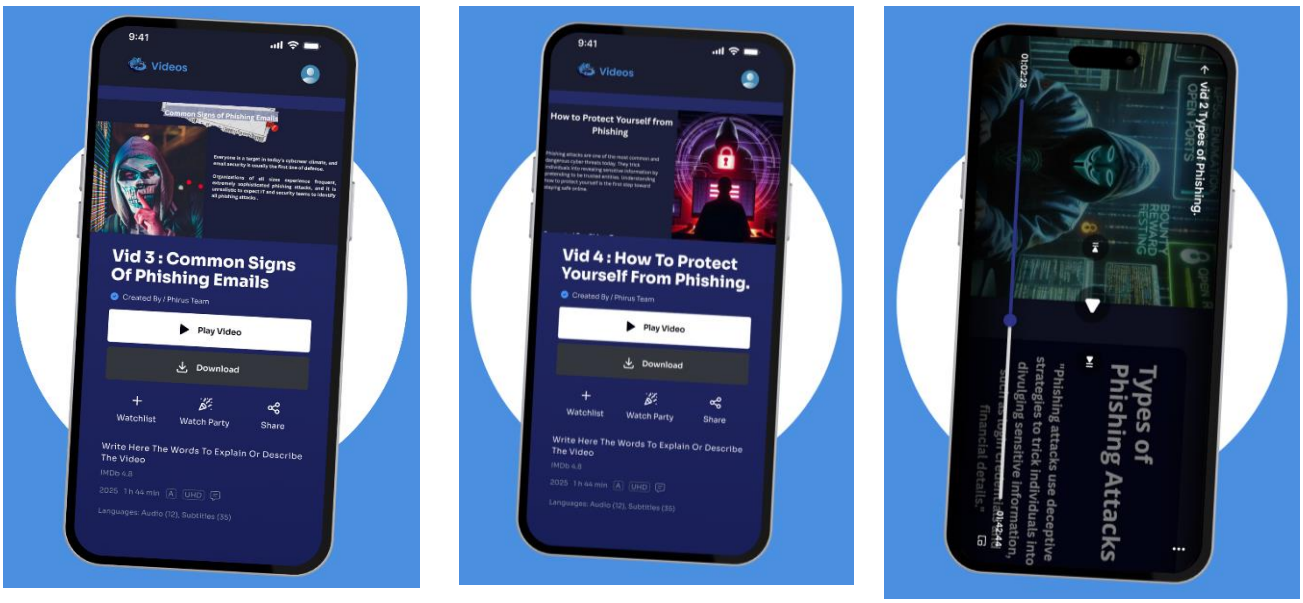


Figure 43 : Learning-2

Chapter (5)

Chapter 5: System Implementation

In previous chapters, we have demonstrated the potential requirements and limitations of our system. In this chapter, we will map our design into implementation.

5.1 UI / UX

When it comes to designing UI for a project, Figma can be particularly useful. With its easy-to-use interface, designers can quickly create wireframes, prototypes, and high-fidelity designs that accurately reflect their vision. The software also offers a range of tools for collaborating with team members and stakeholders, making it an excellent choice for group projects.

We carefully selected colors that were well-suited to the nature of our project and provided optimal visual comfort for users. By choosing colors that complemented the project's theme and purpose, we were able to create a cohesive and visually pleasing design. Additionally, we ensured that the colors we used were easy on the eyes, promoting a comfortable user experience that encourages continued engagement with the product. This figure places the basic colors used in the design.

These are the colors we have choosed:

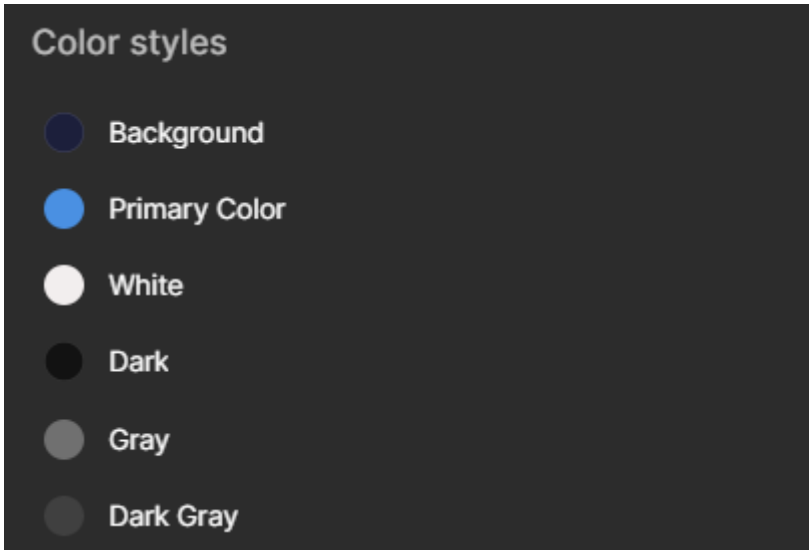


Figure 44 : Used Colors

5.1.1 Front-end

We can say it's like Mobile App but without the educational content.

1. **Splash & onboarding:**

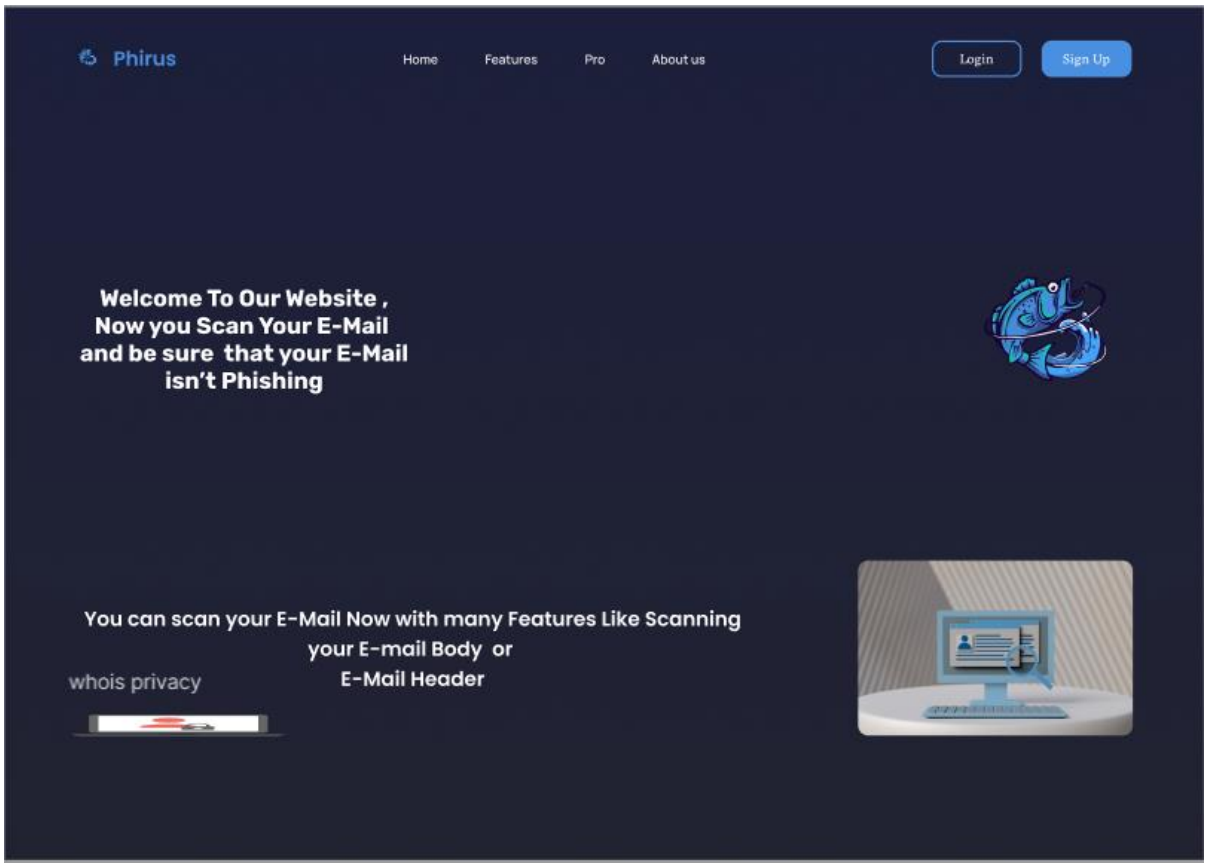


Figure 45 : Front-End 1

2. **Login, forget password and create account:**

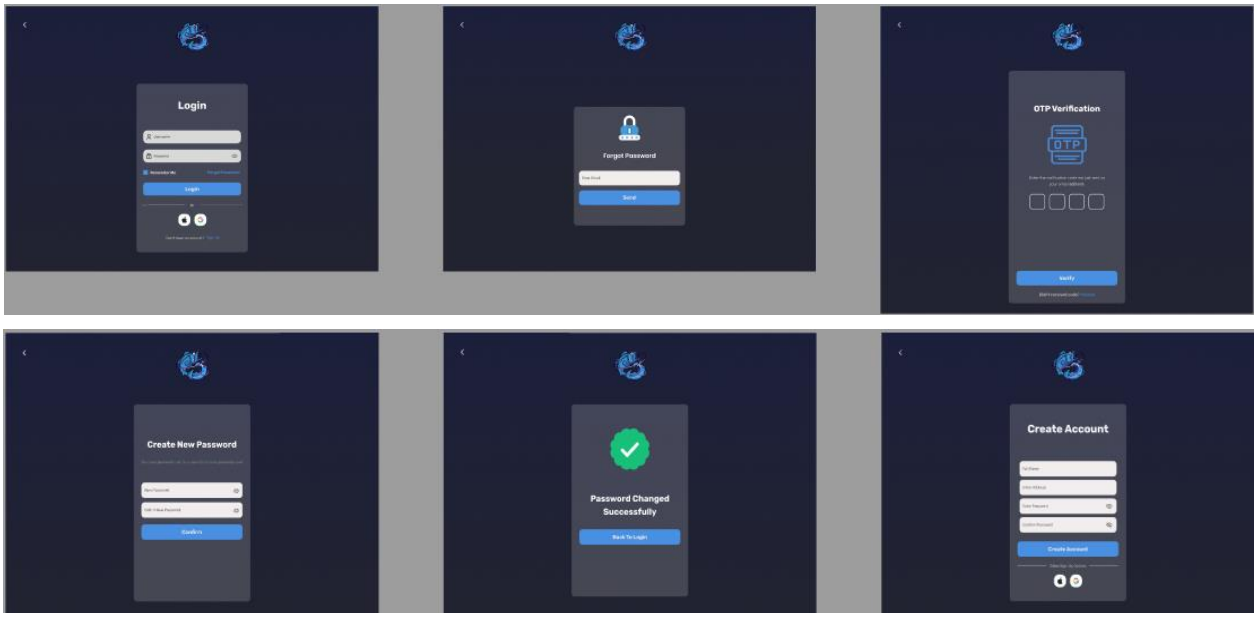


Figure 46 : Front-End 2

3. Header Checking:

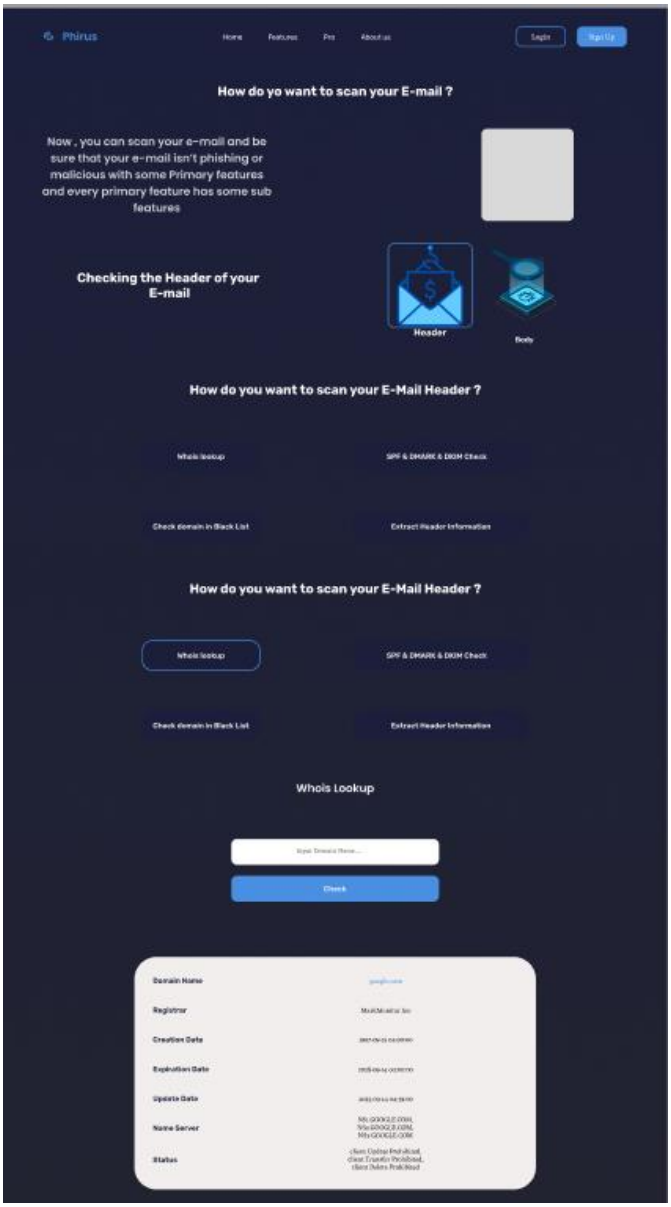


Figure 47 : Front-End 3

4. Body Checking

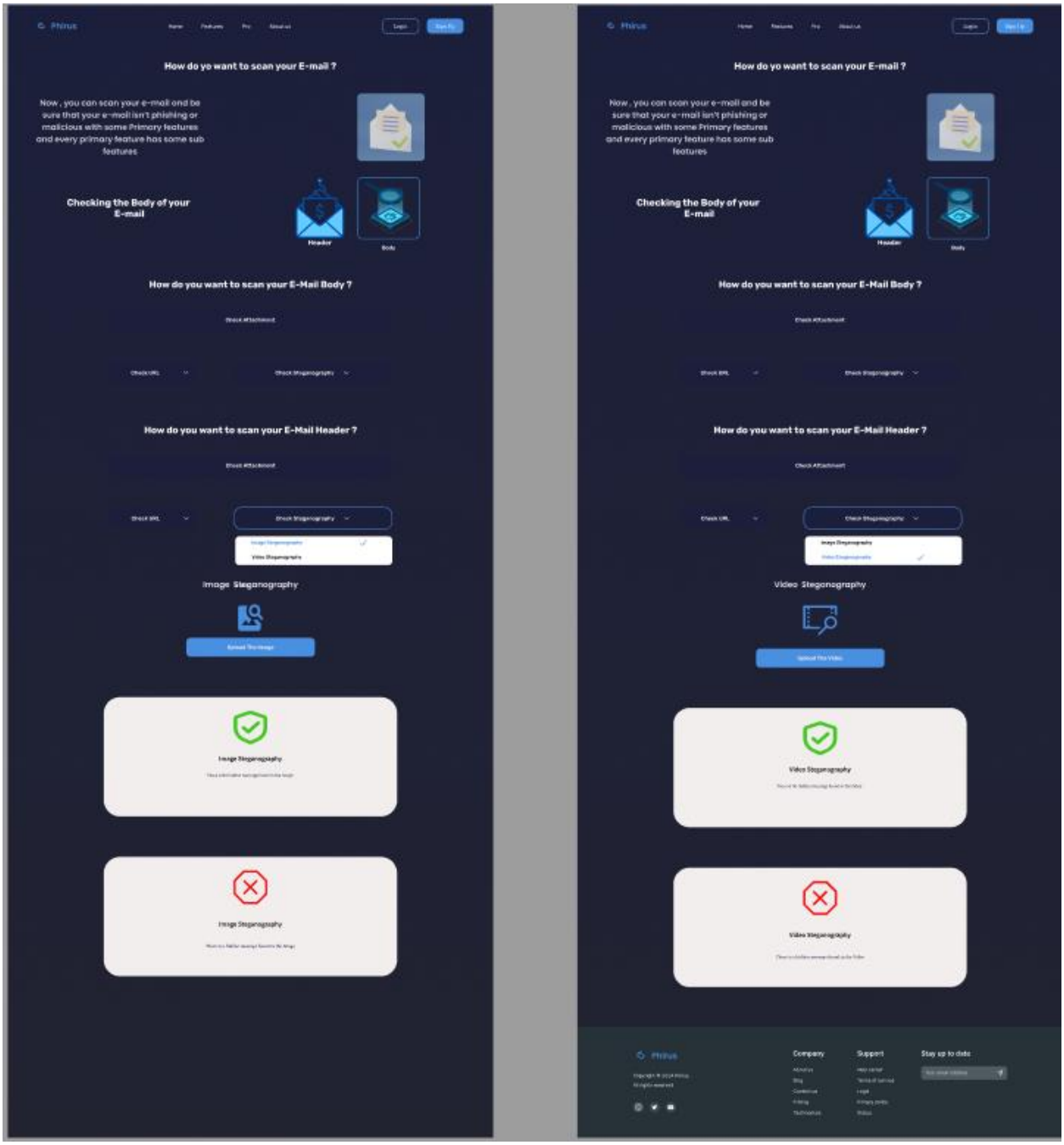


Figure 48 : Front-End 4

5.2 Back-end

5.2.1. Register

It receives the user data and checks if the password is correct, and if the passwords do not match, it redirects the user to the registration page with an error message. If there is a missing field, it also redirects them back with an error message. If everything is fine, it saves the new user's data in the database using the register() function in the User class. If the registration is successful, it takes them to the home page (home.php), and if there is a problem with the registration, it redirects them back to the registration page with the error message.

```php
<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json");
header("Access-Control-Allow-Methods: POST");

include_once "../config/Database.php";
include_once "../model/User.php";

$database = new Database();
$db = $database->connect();
$user = new User($db);

// Check if form data exists
if (isset($_POST['full_name'], $_POST['email'], $_POST['password'], $_POST['confirm_password'])) {
    $full_name = $_POST['full_name'];
    $email = $_POST['email'];
    $password = $_POST['password'];
    $confirm_password = $_POST['confirm_password'];

    // Check if passwords match
    if ($password !== $confirm_password) {
        header("Location: ../signup.php?error=Passwords do not match");
        exit;
    }

    // Assign values
    $user->username = $full_name;
    $user->email = $email;
    $user->password = $password;

    if ($user->register()) {
        header("Location: ../home.php?success=User registered successfully");
    } else {
        header("Location: ../signup.php?error=Registration failed. Try again");
    }
} else {
    header("Location: ../signup.php?error=Incomplete form submission");
}
```

Figure 49 : Back-end: register

5.2.2. Login

A session starts to save the user's data in a list of users stored in the code: user1 and user2 with their passwords. When the user enters their details and clicks submit, it checks if the username and password are correct. If correct, it creates a token (JWT) that holds the user's information and its validity period, placing the token in a cookie for the user, redirecting them to home.php. If incorrect, it displays the error message 'Wrong email or password.'

```php
<?php
session_start();
require_once './helpers/jwt_helper.php';

$users = [
    'user1' => 'password123',
    'user2' => 'mypassword'
];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = $_POST['username'] ?? '';
    $password = $_POST['password'] ?? '';

    if (isset($users[$username]) && $users[$username] === $password) {

        $payload = [
            'username' => $username,
            'iat' => time(),
            'exp' => time() + 3600
        ];

        $token = JWTHandler::generateToken($payload);

        setcookie('token', $token, time() + 3600, '/');

        header("Location: home.php");
        exit;
    } else {
        $error = "Wrong email or password";
    }
}
```

```
32  ?>
33
34  <!DOCTYPE html>
35  <html lang="en">
36
37  <head>
38      <meta charset="UTF-8">
39      <link rel="icon" href="./img/logo_nav.png">
40      <meta http-equiv="refresh" content="" />
41      <meta name="viewport" content="width=device-width, initial-scale=1.0">
42      <link rel="stylesheet" href="./css/login.css">
43      <title>Phirus</title>
44
45       <script src="https://accounts.google.com/gsi/client" async defer></script>
46  </head>
47
48  <body>
49
50  <?php if (!empty($error)): ?>
51      <p style="color:red;"><?php echo htmlspecialchars($error); ?></p>
52  <?php endif; ?>
53
54      <div class="Back_Arrow">
55          <a href="index.php"><img class="icon" src="./img/Back-Arrow.svg" alt="icon_img"></a>
56      </div>
57
58      <div class="logo">
59          <img class="logo_img" src="./img/logo_home.png" alt="logo_img">
60      </div>
61      <div class="container">
```

```
62
63          <div class="login_box">
64              <h2>Login</h2>
65              <form method="POST" action="api/login.php">
66                  <div class="input_group">
67                      <img class="imge" src="./img/Frame.svg">
68                      <input name="email" required type="email" placeholder="Email Address">
69
70                  </div>
71                  <div class="input_group">
72                      <img class="imge" src="./img/Frame (1).svg">
73                      <input name="password" required type="password" placeholder="Enter Password">
74
75                  </div>
76                  <div class="options">
77                      <label><input type="checkbox">Remender Me</label>
78                      <a id="Forgot_Password" class="Forgot_Password" href="forget.php">Forgot Password</a>
79                  </div>
80                  <button type="submit">Login</button>
81                  <div class="or">
82                      <hr>
83                      or
84                      <hr>
85                  </div>
86                  <div class="social_login">
87                      <a href="https://appleid.apple.com/login" class="social_btn_apple">
88                          <img src="./img/IOS.svg">
89                      </a>
ccounts
91                      <a href="https://accounts.google.com/login" class="social_btn_google">
92                          <img src="./img/Google.svg">
                        </a>
```

```
                </div>
                <div class="sign_up">
                    <p>Don't have an accont ?</p>
                    <a href="signup.php" class="sign">Sign Up</a>
                </div>
            </form>
        </div>
    </div>


</body>

</html>
```

Figure 50 : Back-end: Login

5.2.3. Profile

The token is received by taking it from the request header and verifying the token. If there is no token, it returns the message 'Token required'. If the token is incorrect or expired, it returns the message 'Invalid token'. If the token is correct, it extracts the ID from the token and uses that ID to fetch the user's data from the database, returning only the username in JSON format.

```php
<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json");

include_once "../config/Database.php";
include_once "../model/User.php";
include_once "../helpers/jwt_helper.php";

$headers = getallheaders();
$token = isset($headers["Authorization"]) ? str_replace("Bearer ", "", $headers["Authorization"]) : "";

if ($token) {
    $decoded = JWTHandler::verifyToken($token);

    if ($decoded) {
        $database = new Database();
        $db = $database->connect();
        $user = new User($db);
        $user->id = $decoded->id;
        $userData = $user->getProfile();

        // Return only the username
        echo json_encode(["username" => $userData['username']]);
    } else {
        echo json_encode(["message" => "Invalid token."]);
    }
} else {
    echo json_encode(["message" => "Token required."]);
}
```

Figure 51 : Back-end: profile

5.2.4. Reset Password

It receives the token that is sent to the user when they request to reset their password, as well as the new password and the confirmation of the new password. It checks the data; if the new password and its confirmation do not match, it returns an error message. If any field is missing, it returns a message stating that all fields are required. If everything is in order, it first verifies

the token to ensure it's valid and not expired. If the token is valid, it encrypts the new password to ensure security and saves the new password in the database. If the update is successful, it returns a success message; if any issue occurs, it returns an error message.

```php
<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json");

include_once "../config/Database.php";
include_once "../model/User.php";
$database = new Database();
$db = $database->connect();
$user = new User($db);

$data = json_decode(file_get_contents("php://input"));

if (!empty($data->token) && !empty($data->new_password) && !empty($data->confirm_password)) {
    if ($data->new_password !== $data->confirm_password) {
        echo json_encode(["message" => "Passwords do not match."]);
        exit;
    }
    if ($user->verifyToken($data->token)) {
        $hashedPassword = password_hash($data->new_password, PASSWORD_BCRYPT);
        if ($user->updatePassword($hashedPassword)) {
            echo json_encode(["message" => "Password updated successfully."]);
        } else {
            echo json_encode(["message" => "Failed to update password."]);
        }
    } else {
        echo json_encode(["message" => "Invalid or expired token."]);
    }
} else {
    echo json_encode(["message" => "All fields are required."]);
}
?>
```

Figure 52 : Back-end: reset password

5.2.5. Request Reset

The email can be received through JSON data for use with APIs or through a regular POST form. It checks the email, and if the email is not found, it redirects the user to the forgot password page with an error message. If the email exists, it generates a verification code consisting of 4 random digits, sets an expiration time for the code, and saves the code and its expiration time in the database. After completing these steps, it sends the email to the user with the code. The email will direct the user to the code entry page log.otp.html, and if there is a problem, it redirects them back to the forgot password page with an error message. It uses the mail_helper.php file, which should contain functions for sending emails, and the content of the email is very simple; it only sends the code without any additional details.

```php
<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json");

include_once "../config/Database.php";
include_once "../model/User.php";
include_once "../helpers/mail_helper.php";

$database = new Database();
$db = $database->connect();
$user = new User($db);
// Support both JSON and form data
$data = json_decode(file_get_contents("php://input"));
$email = $data->email ?? $_POST['email'] ?? null;
if (!empty($email)) {
    $user->email = $email;
    $verification_code = rand(1000, 9999);

    $expiry_time = date("Y-m-d H:i:s", time() + 30);

    if ($user->storeResetToken($verification_code, $expiry_time)) {
        sendEmail($user->email, "Password Reset Code", "Your verification code is: $verification_code");

        header("Location: ../log.otp.html?success=Verification code sent");
    } else {
        header("Location: ../forget.php?Failed=failed to generate");
    }
} else {
    header("Location: ../forget.php?Enter the Email");
}
?>
```

Figure 53 : Back-end: request reset

5.2.6. Mail Header

It prepares the basic settings using the PHPMailer library to send these emails better than the regular mail() function and sets up SMTP for Gmail, but you can change it to any other email. To send an email, it uses three things: $to (the email to be sent to), $subject (the subject of the email), and $body (the content of the email itself). The Gmail settings are: server smtp.gmail.com, port 587, and the type of protection that requires a username and password, which are the Gmail login credentials.

```php
<?php
use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Exception;

require '../vendor/autoload.php';

function sendEmail($to, $subject, $body) {
    $mail = new PHPMailer(true);

    try {
        // SMTP Configuration
        $mail->isSMTP();
        $mail->Host       = 'smtp.gmail.com';
        $mail->SMTPAuth   = true;
        $mail->Username   = 'shefo993@gmail.com'; // Replace with your email
        $mail->Password   = 'hello';   // Replace with your app password
        $mail->SMTPSecure = 'tls';
        $mail->Port       = 587;

        // Email content
        $mail->setFrom('shefo993@gmail.com', 'Your App Name');
        $mail->addAddress($to);
        $mail->Subject = $subject;
        $mail->Body    = $body;

        return $mail->send(); // Returns true if email sent
    } catch (Exception $e) {
        return false;
    }
}
?>
```

Figure 54 : Back-end: mail header

5.2.7. JWP Helper

The Firebase JWT library is a powerful and tested library for handling tokens and uses strong encryption algorithms. The generateToken() function generates a new token, while validateToken() checks the validity of the token and decrypts it. There is a secret key that must be strong and hidden, and the HS256 algorithm is used for encryption, which is one of the best algorithms. When the user logs in, the site communicates with the JWTHandler to create a new token. The token is generated and sent to the client. When the user attempts to access a protected page, the site communicates with the JWTHandler to verify the token. If the token is valid, access is granted.

```php
<?php
require_once __DIR__ . '/../vendor/autoload.php';

use Firebase\JWT\JWT;
use Firebase\JWT\Key;

class JWTHandler
{
    private static $secret_key = 'your_secret_key';
    private static $algo = 'HS256';


    public static function generateToken($payload)
    {
        return JWT::encode($payload, self::$secret_key, self::$algo);
    }


    public static function validateToken($token)
    {
        try {
            $decoded = JWT::decode($token, new Key(self::$secret_key, self::$algo));
            return (array) $decoded;
        } catch (Exception $e) {
            return false;
        }
    }
}
```

Figure 55 : Back-end: JWP helper

5.2.8. Forget

The user writes their email and presses the Send button. The data is sent to the request_reset.php file, which we will discuss later. The form has required fields to ensure it is not left empty, and the email field type is specified to ensure the email is in the correct format.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <link rel="icon" href="./img/logo_nav.png">
    <meta http-equiv="refresh" content="" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="./css/forget.css">
    <title>Phirus</title>
</head>
<body>
    <div class="Back_Arrow">
        <a href="login.php">
            <img class="icon" src="./img/Back-Arrow.svg"></a>
    </div>
    <div class="logo">
        <img class="logo_img" src="./img/logo_home.png" alt="logo_img">
    </div>
    <div class="container">
        <div class="card">
            <form action="api/request_reset.php" method="POST">
                <img class="lock_box" src="./img/Password.svg">
                <h1>Forgot Password</h1>
                <input type="email" name="email" required placeholder="Enter Email" autofocus>
                <a href="log.otp.html"><p><button type="submit">Send</button></p></a>
            </form>
        </div>
    </div>
</body>
</html>
```

Figure 56 : Back-end: forget

5.2.9. Logout

It deletes the user's token by clearing the cookie named token by setting its value to an empty string and setting its expiration time to the past. This causes an immediate expiration of the cookie and redirects the user to the login page. After deleting the token, it redirects to login.php and adds a success message 'Logged out' to let the user know that they have been logged out.

```php
<?php
setcookie("token", "", time() - 3600, "/", "", false, true);
header("Location: login.php?success=Logged out");
exit;
?>
```

Figure 57 : Back-end: logout

5.2.10. Signup

It checks the data that I have entered; if there is something wrong, it gives a message in red indicating that there is something wrong with the data. If the data is entered correctly, it gives a message in green indicating that everything is fine and the account has been created.

```php
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <link rel="icon" href="./img/logo_nav.png">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="./css/signup.css">
    <title>Phirus</title>
</head>

<body>
<div class="Back_Arrow">
    <a href="index.php">
        <img class="icon" src="./img/Back-Arrow.svg"></a>
</div>
<div class="logo">
    <img class="logo_img" src="./img/logo_home.png" alt="logo_img">
</div>
<div class="container">
    <div class="card">
        <form action="api/register.php" method="POST">
            <h1>Create Account</h1>

            <?php
                if (isset($_GET['error'])) {
                    echo '<p style="color:red;">' . htmlspecialchars($_GET['error']) . '</p>';
            }
            if (isset($_GET['success'])) {
            echo '<p style="color:green;">' . htmlspecialchars($_GET['success']) . '</p>';
            }
```

```
            ?>

            <div class="input_group">
                <input name="full_name" required type="text" placeholder="Full Name" autofocus>
                <p><input name="email" required type="email" placeholder="Email Address"></p>
                <p><input name="password" required type="password" placeholder="Enter Password"></p>
                <p><input name="confirm_password" required type="password" placeholder="Confirm Password"></p>
            </div>

            <button type="submit">Create Account</button>

            <div class="hr">
                <hr>
                Other Sign Up Options
                <hr>
            </div>

            <div class="social_sign_up">
                <a href="https://appleid.apple.com/login" class="social_btn_apple">
                    <img src="./img/IOS.svg">
                </a>
                <a href="https://accounts.google.com/login" class="social_btn_google">
                    <img src="./img/Google.svg">
                </a>
            </div>
        </form>
    </div>
</div>
</body>
</html>
```

Figure 58 : Back-end: signup

5.2.11. Home

This ensures that the page will only open for registered users and uses an encrypted JWT token to verify the user's identity. If anything is not correct, it redirects the user back to the login page with an error message.

```
<?php
session_start();
require_once './helpers/jwt_helper.php';

$user = null;
if (isset($_COOKIE['token'])) {
    $token = $_COOKIE['token'];
    $payload = JWTHandler::validateToken($token);

    if ($payload) {
        $user = $payload;
    } else {
        header("Location: login.php?error=Session expired");
        exit;
    }
} else {
    header("Location: login.php?error=Please login first");
    exit;
}
?>

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <link rel="icon" href="./img/logo_nav.png">
    <meta http-equiv="refresh" content="" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="./css/home.css">
    <title>Phirus</title>
```

```
</head>

<body>
    <nav>
        <div class="logo_navbar_home">
            <a href="home.php">
                <img class="logo_img" src="./img/logo_nav.png" alt="logo_icon"></a>
            <a class="logo_text" href="home.php">Phirus</a>
        </div>
        <ul class="links">
            <a class="li" href="#">
                <li>Home</li>
            </a>
            <a class="li" href="features.html">
                <li>Features</li>
            </a>
            <a class="li" href="Email.Details.html">
                <li>Pro</li>
            </a>
            <a class="li" href="#footer">
                <li>About us</li>
            </a>
        </ul>
        <div class="links_navbar">
            <div class="links_navbar">
   <?php if ($user && isset($user['username'])): ?>
    <span class="username_nav"><?php echo htmlspecialchars($user['username']); ?></span>
    <a class="logout_nav" href="logout.php">Logout</a>
<?php else: ?>
    <a class="login_nav" href="login.php">Login</a>
    <a class="signup_nav" href="signup.php">Sign up</a>
```

```
<?php endif; ?>

</div>

        </div>
    </nav>

    <!-- Hero_section -->
    <div class="Hero_section">
        <h1 class="text_lift_section">
            Welcome To Our Website ,
            Now you Scan <br>Your E-Mail<br>
            and be sure that your E-Mail<br>
            isn't Phishing
        </h1>
        <img class="img_right_section" src="./img/logo_home.png" alt="image">
    </div>

    <!--section_2-->
    <div class="Hero_section">
        <span class="text_section_2">
            You can scan your E-Mail Now with many Features <br>
            Like Scanning your E-mail Body or <br>
            E-Mail Header
        </span>
        <img class="img_right_section_2" src="./img/logo_home_2.svg" alt="image">
    </div>

    <!--section_3-->
    <div class="Hero_section">
        <span class="text_section_2">
            E-Mail header feature contains many sub features to<br>
            be sure that the E-Mail header isn't Phishing or <br>
            Malicious .<br>
            It contains many features like Whois lookup , SPF &<br>
            DMARC & DKIM , Check domain in black list and<br>
            Extract Header information
        </span>
        <img class="img_right_section_2" src="./img/logo_home_3.svg" alt="image">
    </div>

    <!--section_4-->
    <div class="Hero_section">
        <span class="text_section_2">
            E-Mail Body feature contains many sub features to be <br>
            sure that the E-Mail body isn't Phishing or Malicious . <br>
            It contains many features like Checking attachment , <br>
            Checking URL and Checking Steganography
        </span>
        <img class="img_right_section_2" src="./img/logo_home_4.svg" alt="image">
    </div>
</body>
</html>
```

Figure 59 : Back-end: home

5.2.12. User

He is responsible for everything related to the users on the site, from registration to password recovery.

```php
<?php
class User {
    private $conn;
    private $table = "users";
    private $table2 = "password_reset_tokens";
    public $id;
    public $username;
    public $email;
    public $password;

    public function __construct($db) {
        $this->conn = $db;
    }

    public function register() {
        $query = "INSERT INTO " . $this->table . " (username, email, password) VALUES (:username, :email, :password)";
        $stmt = $this->conn->prepare($query);

        $this->username = htmlspecialchars(strip_tags($this->username));
        $this->email = htmlspecialchars(strip_tags($this->email));
        $this->password = password_hash($this->password, PASSWORD_BCRYPT);

        $stmt->bindParam(':username', $this->username);
        $stmt->bindParam(':email', $this->email);
        $stmt->bindParam(':password', $this->password);
        $stmt->execute();
        return $stmt;
    }

    public function login() {
     $query = "SELECT * FROM users WHERE email = :email LIMIT 1";
     $stmt = $this->conn->prepare($query);
     $stmt->bindParam(':email', $this->email);
     $stmt->execute();
     return $stmt->fetch(PDO::FETCH_ASSOC);
}
     public function getProfile()
     {
         $query = 'SELECT username FROM '.$this->table.' WHERE id = :id';
         $stmt = $this->conn->prepare($query);
         $stmt->bindParam(':id', $this->id);
         $stmt->execute();
         return $stmt->fetch(PDO::FETCH_ASSOC);
     }
     public function storeResetToken($token)
     {
         $query = 'INSERT INTO '.$this->table2.' (email, token) VALUES (:email, :token)';
         $stmt = $this->conn->prepare($query);
         $stmt->bindParam(':email', $this->email);
         $stmt->bindParam(':token', $token);
         $stmt->execute();
         return $stmt;
     }
     public function verifyToken($token): bool
     {
         $query = "SELECT email FROM password_reset_tokens WHERE token = :token AND created_at >= NOW() - INTERVAL 1 HOUR";

         $stmt = $this->conn->prepare($query);
         $stmt->bindParam(":token", $token);
         $stmt->execute();
         $result = $stmt->fetch(PDO::FETCH_ASSOC);
         if ($result) {
             $this->email = $result["email"];
             return true;
         }
         return false;
     }
     public function updatePassword($newPassword) {
         $query = "UPDATE users SET password = :password WHERE email = :email";
         $stmt = $this->conn->prepare($query);
         $stmt->bindParam(":password", $newPassword);
         $stmt->bindParam(":email", $this->email);
         return $stmt->execute();
     }
}
?>
```

Figure 60 : Back-end: user

5.2.13. Database

This is a database connection mechanism, and any other class that wants to work with the database, like User.php, needs to call connect from it and uses PDO to protect the site from SQL Injection attacks.

```php
<?php
class Database {
    // DB Params
    private $host = 'localhost';
    private $db_name = 'authentication';
    private $username = 'root';
    private $password = '';
    private $conn;

    // DB Connect
    public function connect() {
        $this->conn = null;

        try {
            $this->conn = new PDO('mysql:host=' . $this->host . ';dbname=' . $this->db_name, $this->username, $this->password);
            $this->conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        } catch(PDOException $e) {
            echo 'Connection Error: ' . $e->getMessage();
        }

        return $this->conn;
    }
}
```

Figure 61 : Back-end: database

5.3 Flutter

Here are some of flutter code samples of pages from ui

1. On Boarding screen

```dart
onboarding_screen.dart ×  video_steganograph_body.dart  item_uploaded_body.dart
lib > features > onboarding > presentation > views > onboarding_screen.dart > ...
class _OnboardingScreenState extends State<OnboardingScreen> {
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: ColorsManager.backgroundColor,
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.fromLTRB(16, 16, 16, 35),
          child: Column(
            children: [
              BuildHeder(
                pageIndex: pageIndex,
                onboardingData: onboardingData,
              ), // BuildHeder
              const SizedBox(
                height: 100,
              ), // SizedBox
              BuildBody(
                image: onboardingData[pageIndex].image,
                pageIndex: pageIndex,
                onboardingData: onboardingData,
              ), // BuildBody
              const Spacer(
                flex: 1,
              ), // Spacer
              BuildFooter(
                pageIndex: pageIndex,
                onboardingData: onboardingData,
                onTap: () => incrementPageIndex(),
              ), // BuildFooter
            ],
                                                    Ln 1, Col 1  Spaces: 2
```

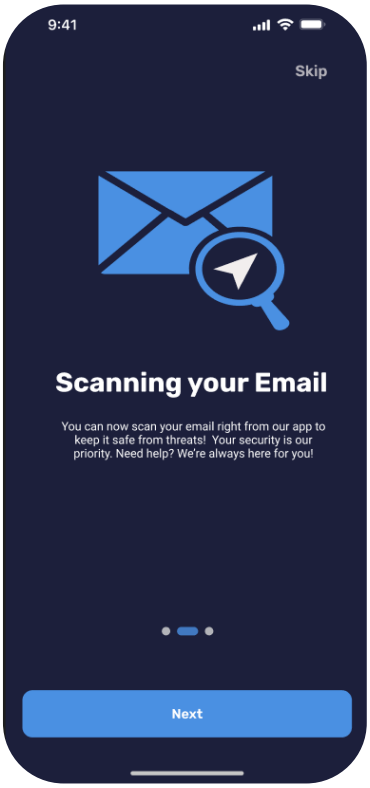


Figure 62 : Flutter: On Boarding

2. Splash screen

splash_screen.dart | video_steganograph_body.dart | item_uploaded_body.dart | video_steg

lib > features > splash > presentaion > views > splash_screen.dart > ...

```dart
class _SplashScreenState extends State<SplashScreen> {
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: ColorsManager.backgroundColor,
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            bodySplash(screenWidth),
            const SizedBox(height: 20),
            headerSplash(screenWidth),
          ],
        ), // Column
      ), // Center
    ); // Scaffold
  }
}

Widget headerSplash(double screenWidth) {
  return TweenAnimationBuilder<double>(
    tween: Tween<double>(begin: screenWidth / 2, end: 0),
    duration: const Duration(seconds: 2),
    curve: Curves.easeInOut,
    builder: (context, value, child) {
      return Transform.translate(
        offset: Offset(value, 0),
        child: child!,
      ); // Transform.translate
    },
    child: Text(
```

Figure 63 : Flutter: Splash

3. Login screen

phirus

lient.dart | dkim_model.dart | spfdmarc_model.dart | whios_model.dart | heder_model.dart

lib > features > auth > presentaions > widgets > login_body.dart > LoginBody > build

```dart
class LoginBody extends StatelessWidget {
  final TextEditingController email = TextEditingController();
  final TextEditingController password = TextEditingController();
  final GlobalKey<FormState> formKey = GlobalKey();

  final bool isLoading = false;

  LoginBody({super.key});

  @override
  Widget build(BuildContext context) {
    return SafeArea(
      child: SingleChildScrollView(
        child: Center(
          child: ModalProgressHUD(
            inAsyncCall: isLoading,
            child: Form(
              key: formKey,
              child: Column(
                children: [
                  SvgPicture.asset(
                    width: 150.w,
                    height: 150.h,
                    Assets.imagesLogo,
                  ), // SvgPicture.asset
                  Padding(
                    padding: const EdgeInsets.only(
                      bottom: 16,
                    ), // EdgeInsets.only
```

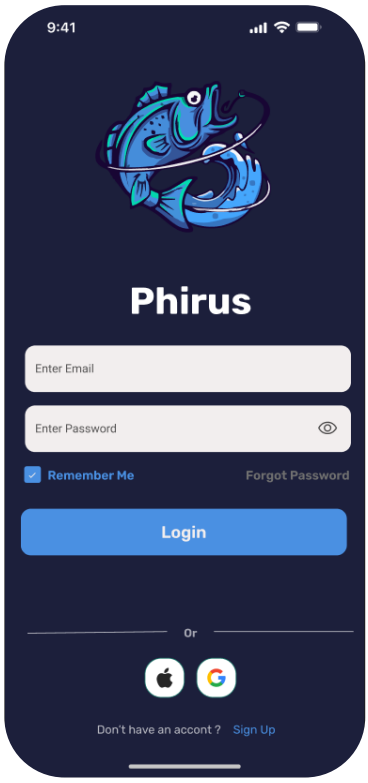


Figure 64 : Flutter: Login

4. Attachment scan screen

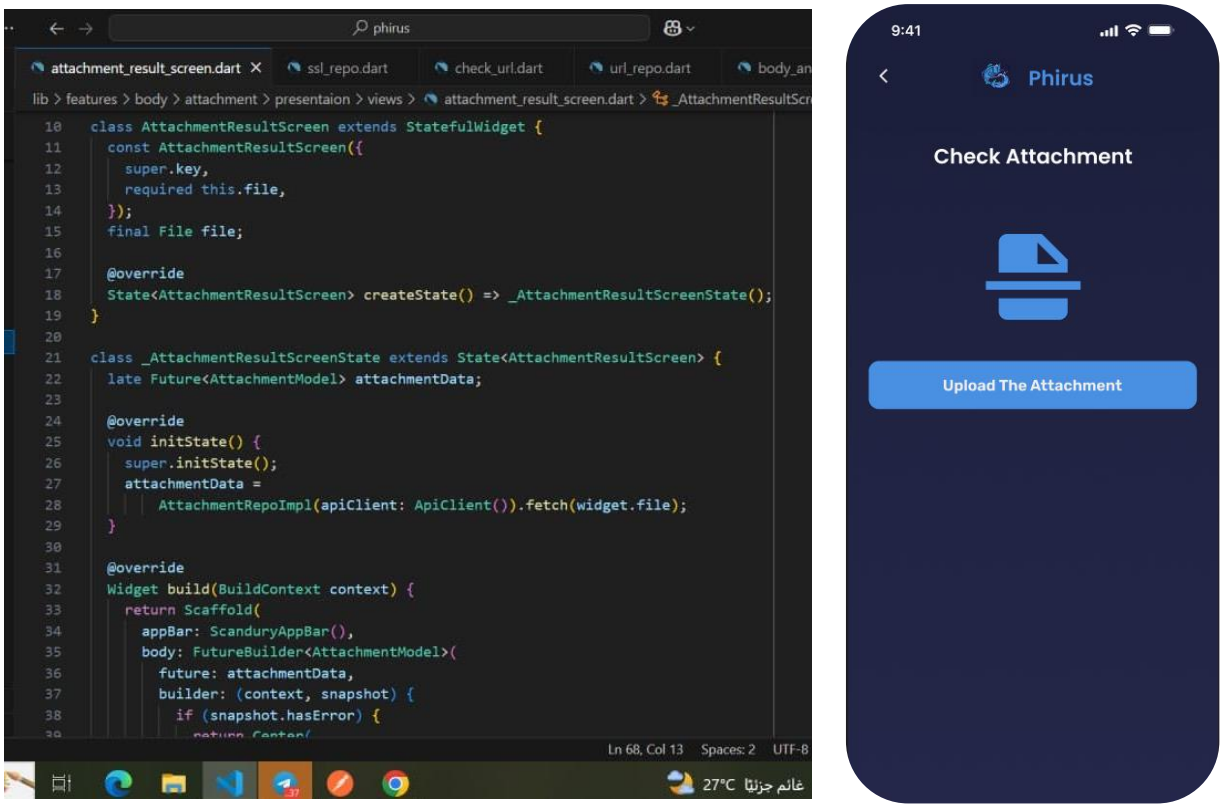


Figure 65 : Flutter: attachment scan

5. Domain check screen

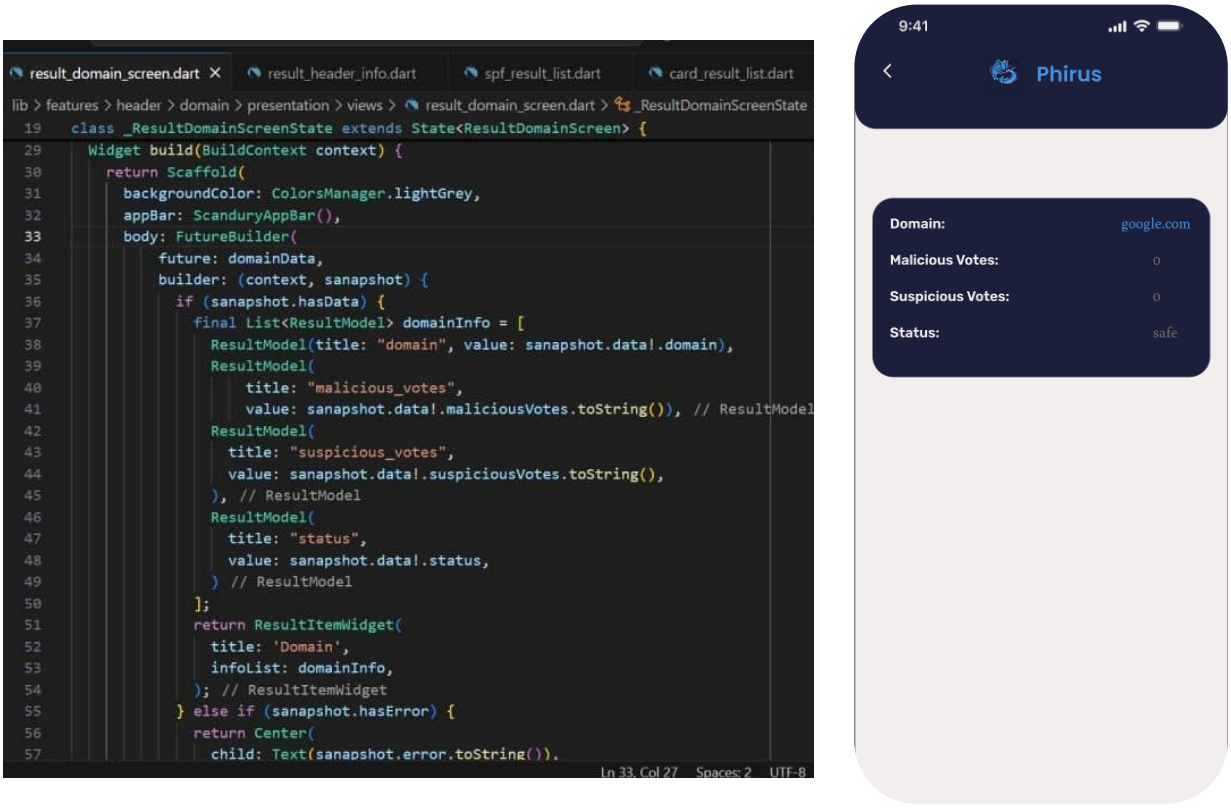
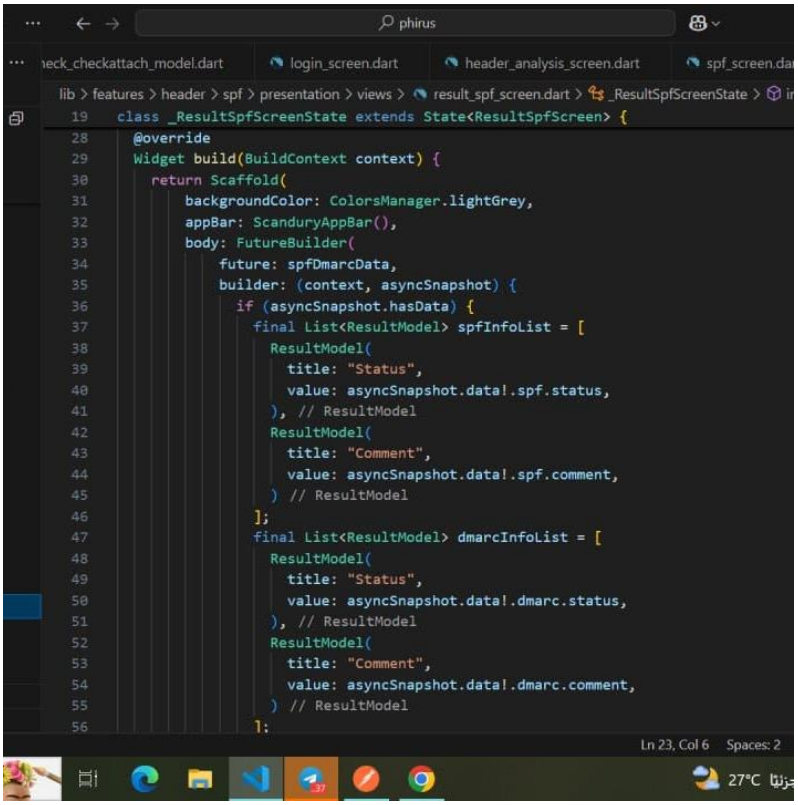


Figure 66 : Flutter: Domain check

6. SPF check screen



```
class _ResultSpfScreenState extends State<ResultSpfScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
        backgroundColor: ColorsManager.lightGrey,
        appBar: ScanduryAppBar(),
        body: FutureBuilder(
            future: spfDmarcData,
            builder: (context, asyncSnapshot) {
              if (asyncSnapshot.hasData) {
                final List<ResultModel> spfInfoList = [
                  ResultModel(
                    title: "Status",
                    value: asyncSnapshot.data!.spf.status,
                  ), // ResultModel
                  ResultModel(
                    title: "Comment",
                    value: asyncSnapshot.data!.spf.comment,
                  ) // ResultModel
                ];
                final List<ResultModel> dmarcInfoList = [
                  ResultModel(
                    title: "Status",
                    value: asyncSnapshot.data!.dmarc.status,
                  ), // ResultModel
                  ResultModel(
                    title: "Comment",
                    value: asyncSnapshot.data!.dmarc.comment,
                  ) // ResultModel
                ];
```

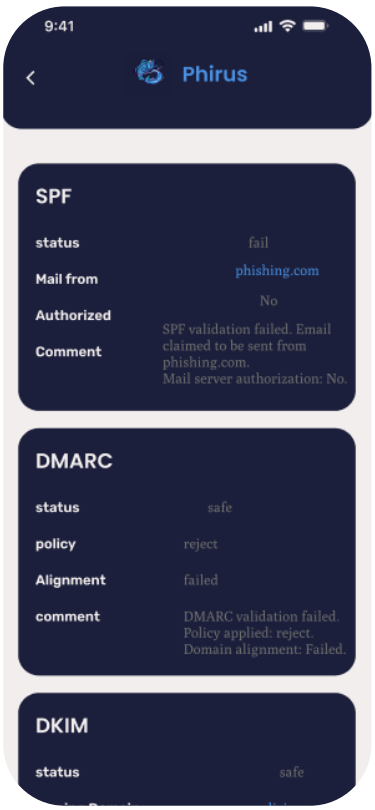
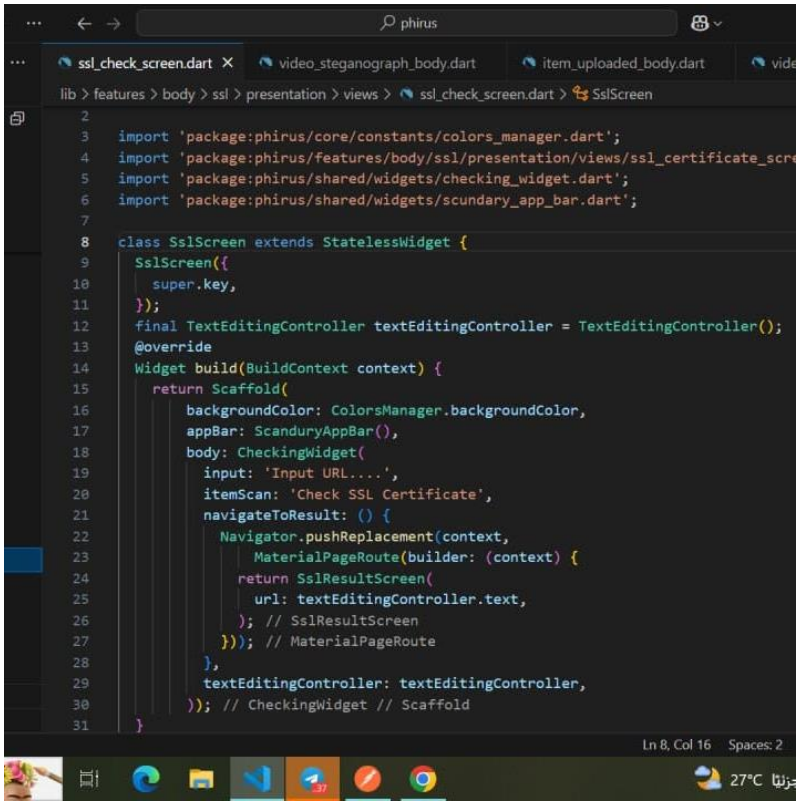


Figure 67 : Flutter: SPF

7. SSL check



```
import 'package:phirus/core/constants/colors_manager.dart';
import 'package:phirus/features/body/ssl/presentation/views/ssl_certificate_scr
import 'package:phirus/shared/widgets/checking_widget.dart';
import 'package:phirus/shared/widgets/scundary_app_bar.dart';

class SslScreen extends StatelessWidget {
  SslScreen({
    super.key,
  });
  final TextEditingController textEditingController = TextEditingController();
  @override
  Widget build(BuildContext context) {
    return Scaffold(
        backgroundColor: ColorsManager.backgroundColor,
        appBar: ScanduryAppBar(),
        body: CheckingWidget(
          input: 'Input URL....',
          itemScan: 'Check SSL Certificate',
          navigateToResult: () {
            Navigator.pushReplacement(context,
                MaterialPageRoute(builder: (context) {
              return SslResultScreen(
                url: textEditingController.text,
              ); // SslResultScreen
            })); // MaterialPageRoute
          },
          textEditingController: textEditingController,
        )); // CheckingWidget // Scaffold
  }
```

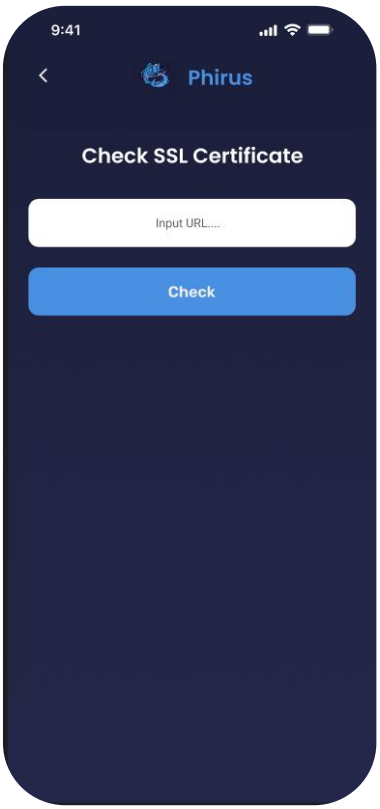


Figure 68 : Flutter: SSL

8. Whois screen

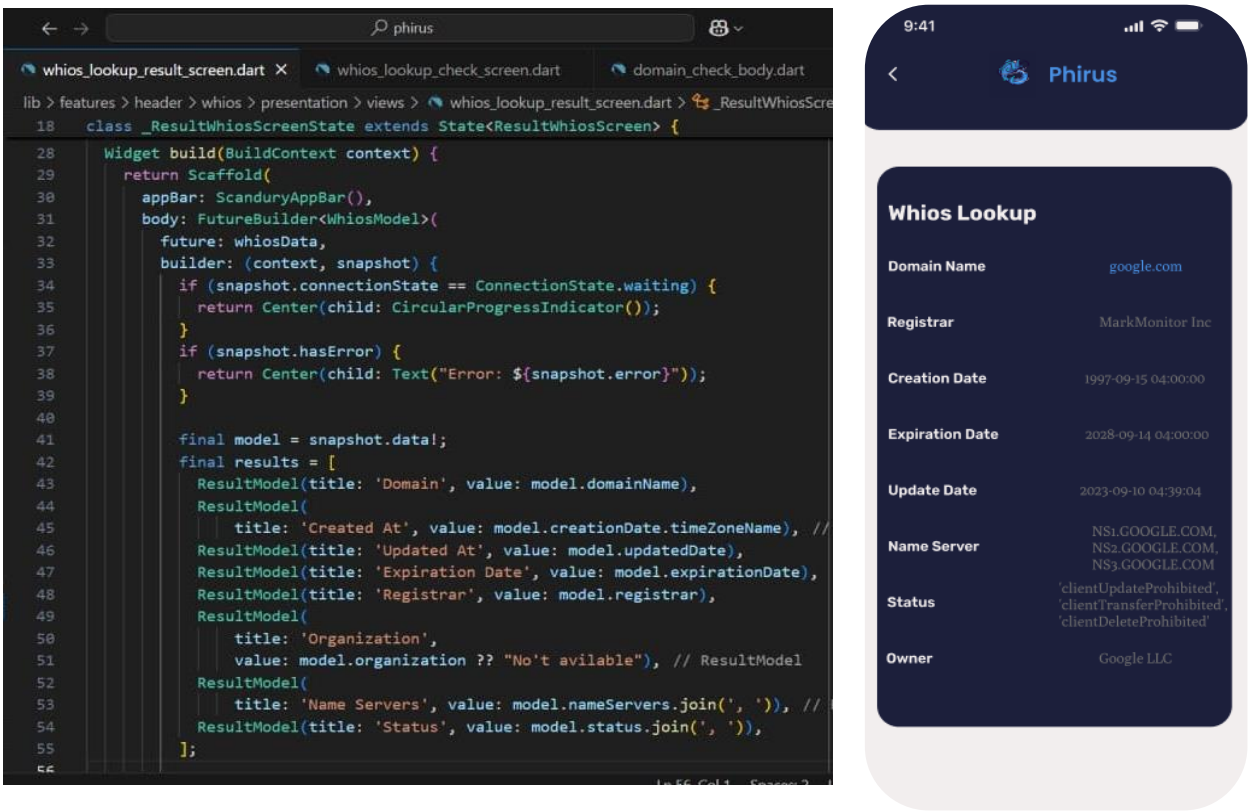


Figure 69 : Flutter: Whois

9. Body analysis screen

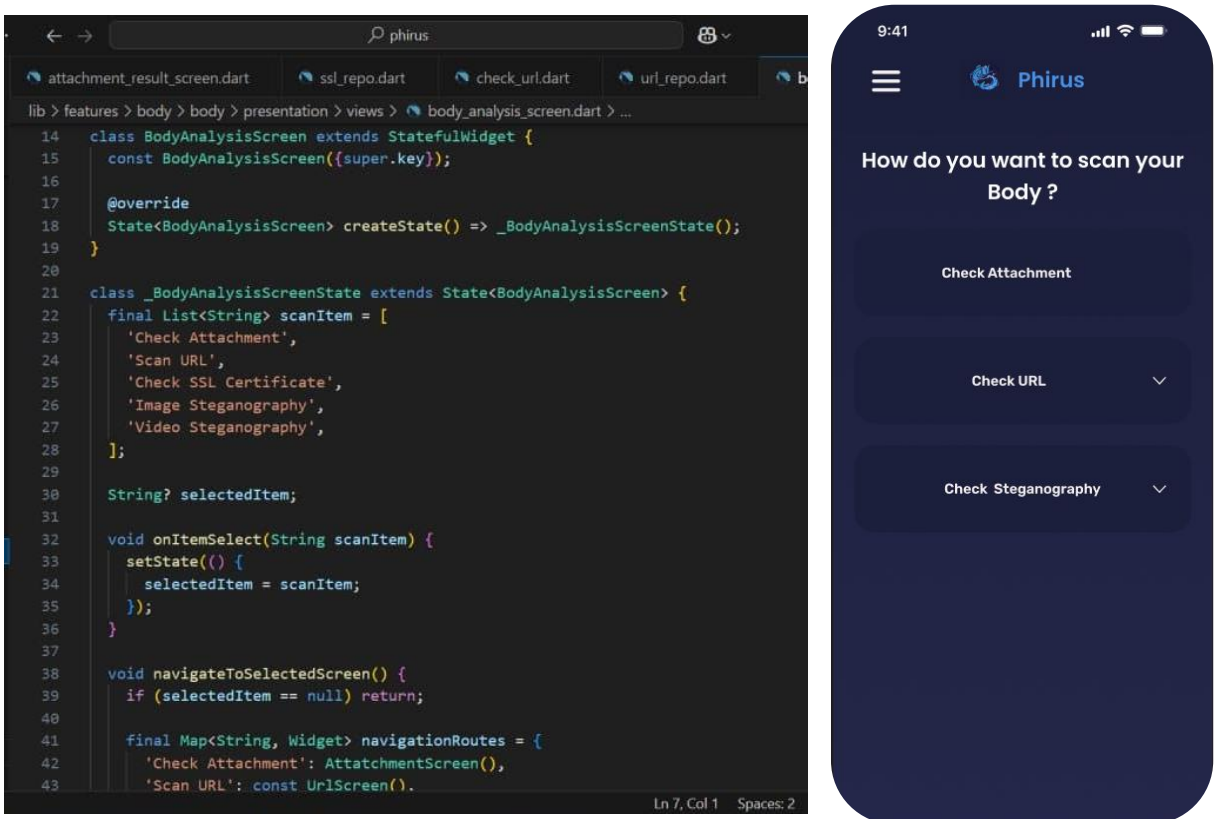


Figure 70 : Flutter: Body analysis

10. Home screen

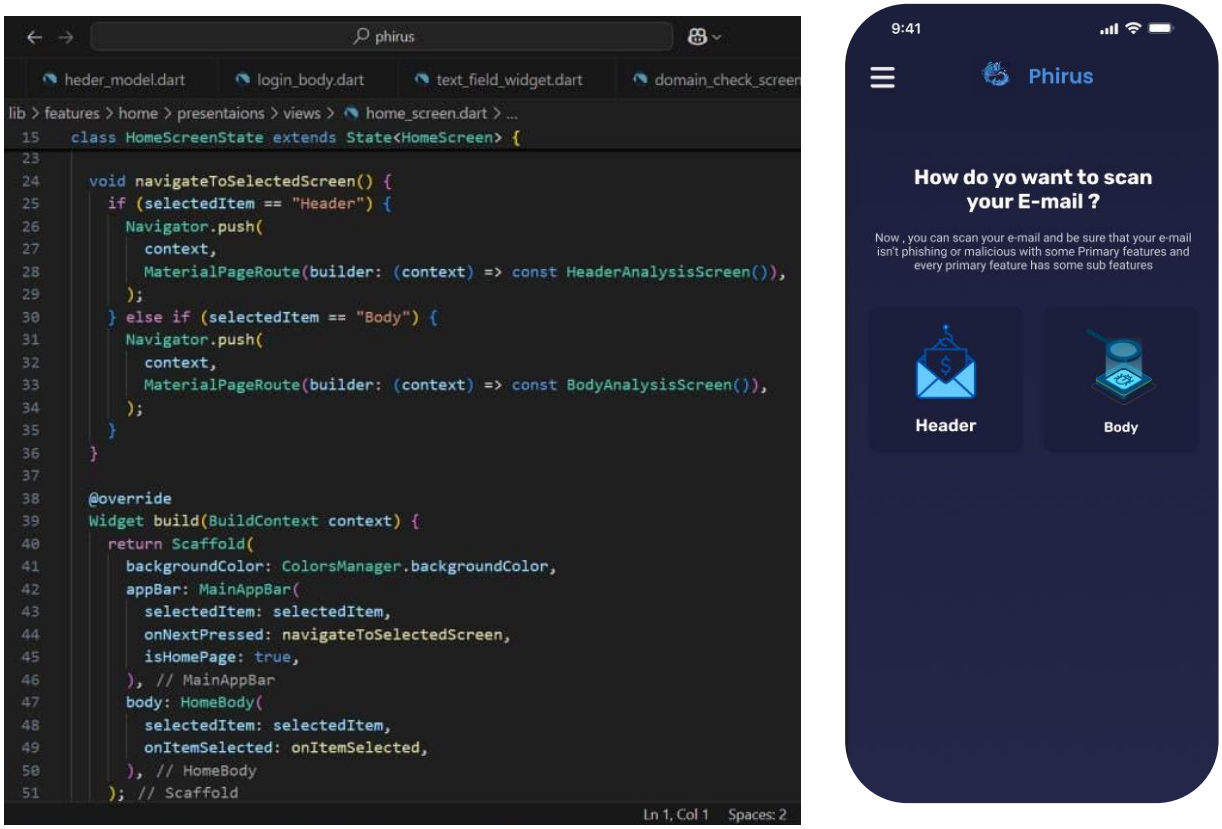


Figure 71 : Flutter: Home screen

5.4 Security Codes

We used Python as a programming language to write codes of our features

5.4.1 Backlist

```python
import dns.resolver
def spf_analysis(domain):
    try:
        result = dns.resolver.resolve(domain, 'TXT')
        spf_records = []
        for rdata in result:
            record = str(rdata).strip('"')
            if record.startswith('v=spf1'):
                spf_records.append(record)

        if spf_records:
            status = "fail"
            mail_from = domain  # Simulating a scenario with phishing.com as the sending domain
            authorized = "No"
            comment = "SPF validation failed. Email claimed to be sent from phishing.com. Mail server authorization: No."
        else:
            status = "pass"
            mail_from = domain
            authorized = "Yes"
            comment = "SPF validation passed."

        return {
            "status": status,
            "mail_from": mail_from,
            "authorized": authorized,
            "comment": comment
        }
```

```
    except dns.resolver.NoAnswer:
        return {"status": "error", "comment": "No SPF record found."}
    except dns.resolver.NXDOMAIN:
        return {"status": "error", "comment": f"Domain {domain} not found."}
    except Exception as e:
        return {"status": "error", "comment": str(e)}
def dkim_analysis(domain):
    try:
        selectors = ['default', 'selector1', 'selector2']
        dkim_records = {}
        for selector in selectors:
            try:
                result = dns.resolver.resolve(f"{selector}._domainkey.{domain}", 'TXT')
                for rdata in result:
                    record = str(rdata).strip('"')
                    if "v=DKIM1" in record:
                        dkim_records[selector] = record
            except (dns.resolver.NoAnswer, dns.resolver.NXDOMAIN):
                dkim_records[selector] = "No DKIM record found."

        if dkim_records:
            status = "fail"
            signing_domain = domain  # Simulating a case where malicious.com is the signing domain
            header_integrity = "Possibly Altered"
            comment = "DKIM validation failed. Email signed by malicious.com. Header integrity: Possibly Altered."
        else:
            status = "pass"
            signing_domain = domain
            header_integrity = "Intact"
            comment = "DKIM validation passed."
```

```
        return {
            "status": status,
            "signing_domain": signing_domain,
            "header_integrity": header_integrity,
            "comment": comment
        }
    except Exception as e:
        return {"status": "error", "comment": str(e)}

def dmarc_analysis(domain):
    try:
        result = dns.resolver.resolve(f"_dmarc.{domain}", 'TXT')
        dmarc_records = []
        for rdata in result:
            record = str(rdata).strip('"')
            if record.startswith('v=DMARC1'):
                dmarc_records.append(record)

        if dmarc_records:
            status = "fail"
            policy = "reject"  # Example: reject policy found
            alignment = "Failed"  # Simulating alignment failure
            comment = f"DMARC validation failed. Policy applied: {policy}. Domain alignment: {alignment}."
        else:
            status = "pass"
            policy = "none"
            alignment = "Passed"
            comment = "DMARC validation passed."

```

```
        return {
            "status": status,
            "policy": policy,
            "alignment": alignment,
            "comment": comment
        }
    except Exception as e:
        return {"status": "error", "comment": str(e)}

def main():
    # Get domain from user input
    domain = input("Enter domain to check SPF, DMARC, DKIM: ")

    print(f"\nSPF Analysis for {domain}:")
    spf_result = spf_analysis(domain)
    print(f"  Status: {spf_result['status']}")
    print(f"  Mail From: {spf_result['mail_from']}")
    print(f"  Authorized: {spf_result['authorized']}")
    print(f"  Comment: {spf_result['comment']}")
    print("-" * 50)

    print(f"DKIM Analysis for {domain}:")
    dkim_result = dkim_analysis(domain)
    print(f"  Status: {dkim_result['status']}")
    print(f"  Signing Domain: {dkim_result['signing_domain']}")
    print(f"  Header Integrity: {dkim_result['header_integrity']}")
    print(f"  Comment: {dkim_result['comment']}")
    print("-" * 50)

    print(f"DMARC Analysis for {domain}:")
    dmarc_result = dmarc_analysis(domain)
    print(f"  Status: {dmarc_result['status']}")
    print(f"  Policy: {dmarc_result['policy']}")
    print(f"  Alignment: {dmarc_result['alignment']}")
    print(f"  Comment: {dmarc_result['comment']}")
    print("-" * 50)

if __name__ == "__main__":
    main()
```

Figure 72 : Security: blacklist

5.4.2. Check Attachment

```python
from flask import Flask, request, jsonify
import requests
import time

app = Flask(__name__)
API_KEY = '7019e4123a3e38c9ed8f8afd087ace44d8a02cb686b5f0227d60b59d8cc8a3eb'

def upload_file(file_obj, filename):
    url = 'https://www.virustotal.com/api/v3/files'
    headers = {
        'x-apikey': API_KEY
    }
    files = {
        'file': (filename, file_obj)
    }

    response = requests.post(url, headers=headers, files=files)

    if response.status_code == 200:
        return response.json()['data']['id']
    else:
        return None, response.json()

def check_file_status(analysis_id):
    url = f'https://www.virustotal.com/api/v3/analyses/{analysis_id}'
    headers = {
        'x-apikey': API_KEY
    }

    response = requests.get(url, headers=headers)

    if response.status_code == 200:
        json_response = response.json()
        data = json_response.get('data', {})
        attributes = data.get('attributes', {})
        status = attributes.get('status')

        if status == 'completed':
            stats = attributes.get('stats', {})
            return {
                'file_id': analysis_id,
                'malicious': stats.get('malicious', 0),
                'undetected': stats.get('undetected', 0)
            }

        return {'status': status}
    else:
        return None

@app.route('/check_attachment', methods=['POST'])
def check_file():
    if 'file' not in request.files:
        return jsonify({'error': 'No file uploaded'}), 400

    file = request.files['file']
    filename = file.filename
```

```
57
58      analysis_id_or_error = upload_file(file, filename)
59      if isinstance(analysis_id_or_error, tuple):
60          _, error = analysis_id_or_error
61          return jsonify({'error': 'File upload failed', 'details': error}), 500
62      else:
63          analysis_id = analysis_id_or_error
64
65
66      attempts = 0
67      result = None
68      while attempts < 12:
69          time.sleep(20)
70          status_result = check_file_status(analysis_id)
71
72          if isinstance(status_result, dict):
73              if status_result.get('status') != 'queued' and 'malicious' in status_result:
74                  result = status_result
75                  break
76          attempts += 1
77
78      if result:
79          return jsonify(result)
80      else:
81          return jsonify({'error': 'File analysis did not complete in time'}), 504
82
83  if __name__ == '__main__':
84      app.run(debug=False, port=5005, use_reloader=False)
```

Figure 73 : Security: check attachment

5.4.3. Extract Email Header

```
1   from email import message_from_file
2   from email.utils import parsedate_to_datetime
3
4   def extract_email_headers_from_file(file_path):
5       try:
6           # Open the email file and parse the headers
7           with open(file_path, 'r') as email_file:
8               msg = message_from_file(email_file)
9
10          # Extract key header details
11          from_ = msg.get("From", "N/A")
12          to = msg.get("To", "N/A")
13          subject = msg.get("Subject", "N/A")
14          date = msg.get("Date", "N/A")
15          message_id = msg.get("Message-ID", "N/A")
16          reply_to = msg.get("Reply-To", "N/A")
17
18          # Convert date to a more readable format if possible
19          if date != "N/A":
20              try:
21                  date = parsedate_to_datetime(date).isoformat()
22              except:
23                  pass  # Keep the original format if parsing fails
24
```

```
        # Display extracted header information
        print("----- Extracted Email Header Information -----")
        print(f"From: {from_}")
        print(f"To: {to}")
        print(f"Subject: {subject}")
        print(f"Date: {date}")
        print(f"Message-ID: {message_id}")
        print(f"Reply-To: {reply_to}")

    except Exception as e:
        print(f"Error reading or parsing the email file: {e}")

# Example usage
if __name__ == "__main__":
    email_file_path = input("Enter the path to the email file (.eml): ").strip()

    if email_file_path:
        extract_email_headers_from_file(email_file_path)
    else:
        print("Error: The file path cannot be empty.")

```

Figure 74 : Security: extract email header

5.4.4. SSl & TLS

```
import socket
import ssl
from urllib.parse import urlparse

def get_ssl_certificate_details(url):
    try:
        # Parse the URL to extract hostname
        parsed_url = urlparse(url)
        hostname = parsed_url.netloc if parsed_url.netloc else parsed_url.path

        # Connect to the server and retrieve the certificate
        context = ssl.create_default_context()
        with socket.create_connection((hostname, 443)) as sock:
            with context.wrap_socket(sock, server_hostname=hostname) as ssock:
                cert = ssock.getpeercert()

        # Extract relevant details
        subject = dict(x[0] for x in cert['subject'])
        issued_to = subject.get('commonName', 'Unknown')
        issuer = dict(x[0] for x in cert['issuer']).get('commonName', 'Unknown')
        valid_from = cert.get('notBefore', 'Unknown')
        valid_to = cert.get('notAfter', 'Unknown')

        return {
            'Issued To': issued_to,
            'Issuer': issuer,
            'Valid From': valid_from,
            'Valid To': valid_to,
        }
```

```
    except Exception as e:
        return f"Error: Unable to retrieve SSL/TLS certificate details - {str(e)}"


if __name__ == "__main__":
    # Prompt user for input URL
    url = input("Enter the URL (e.g., https://example.com): ").strip()
    if not url.startswith("https://"):
        url = "https://" + url  # Ensure it's HTTPS for SSL

    cert_details = get_ssl_certificate_details(url)
    if isinstance(cert_details, dict):
        print("\nSSL/TLS Certificate Details:")
        for key, value in cert_details.items():
            print(f"{key}: {value}")
    else:
        print(cert_details)
```

Figure 75 : Security: SSL & TLS

5.4.5. Steganography (Photo)

```
from PIL import Image

def check_and_extract_message_from_image():
    """Prompts user for an image path, checks if there is a hidden message, and extracts it."""
    image_path = input("Enter the path to the image file: ")

    try:
        image = Image.open(image_path)
    except Exception as e:
        print(f"Error opening image: {e}")
        return None

    width, height = image.size
    bits = ""

    for y in range(height):
        for x in range(width):
            r, g, b = image.getpixel((x, y))
            bits += str(r & 1)  # Extract LSB from the red channel

    # Search for the end signal in the bits
    end_signal = '11111110'
    if end_signal not in bits:
        print("No hidden message found in the image.")
        return None
```

```
26
27      # Decode the message
28      message = ""
29      for i in range(0, len(bits), 8):
30          byte = bits[i:i+8]
31          if byte == end_signal:  # Stop if end signal is found
32              break
33          message += chr(int(byte, 2))
34
35      print("Hidden message found in the image:", message)
36      return message
37
38  # Example usage
39  check_and_extract_message_from_image()
```

Figure 76 : Security: stegano-photo

5.4.6. Steganography (Video)

```
1   # -- coding: utf-8 --
2   import cv2
3   import numpy as np
4   from scipy.stats import chisquare
5   import scipy.fftpack as fftpack
6   from skimage.measure import shannon_entropy
7
8   def detect_steganography(video_path):
9       cap = cv2.VideoCapture(video_path)
10      frame_count = 0
11      lsb_distribution = []
12      lsb_anomaly_detected = False
13      dct_anomaly_detected = False
14      entropy_anomaly_detected = False
15
16      while cap.isOpened():
17          ret, frame = cap.read()
18          if not ret:
19              break
20
21          frame_count += 1
22          gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
23          lsb_frame = np.bitwise_and(gray_frame, 1)
24
25          unique, counts = np.unique(lsb_frame, return_counts=True)
26          lsb_distribution.append(counts)
27
```

```
        dct_transform = fftpack.dct(fftpack.dct(np.float32(gray_frame), axis=0, norm='ortho'), axis=1, norm='ortho')
        dct_mean = np.mean(dct_transform)
        if dct_mean > 50:
            dct_anomaly_detected = True

        entropy_value = shannon_entropy(gray_frame)
        if entropy_value > 7.5:
            entropy_anomaly_detected = True

    cap.release()

    # تحليل LSB
    if lsb_distribution:
        observed = np.sum(lsb_distribution, axis=0)
        expected = np.full_like(observed, np.mean(observed))
        chi_stat, p_value = chisquare(observed, expected)

        if p_value < 0.05:
            lsb_anomaly_detected = True

    # دمج النتائج
    if lsb_anomaly_detected or dct_anomaly_detected or entropy_anomaly_detected:
        print("⚠️ Potential steganography detected")
    else:
        print("✅ No steganography detected")

if __name__ == "__main__":
    video_path = input("Enter the path to the video file: ")
    detect_steganography(video_path)
```

Figure 77 : Security: stegano-video

5.4.7. URL

```
import requests
from urllib.parse import urlparse
import base64
# Function to extract URL components
def extract_url_info(url):
    parsed_url = urlparse(url)
    url_info = {
        "Domain": parsed_url.netloc,
        "Path": parsed_url.path,
        "Query": parsed_url.query,
        "Scheme": parsed_url.scheme.upper(),
        "Port": parsed_url.port if parsed_url.port else ("80" if parsed_url.scheme == "http" else "443")
    }
    return url_info
# Function to encode URL for VirusTotal
def encode_url(url):
    url_bytes = url.encode("utf-8")
    base64_url = base64.urlsafe_b64encode(url_bytes).decode("utf-8").strip("=")
    return base64_url
# Function to check URL in VirusTotal
def check_virustotal(api_key, url):
    base_url = "https://www.virustotal.com/api/v3/urls"
    # Encode the URL
    encoded_url = encode_url(url)
    # Corrected Headers
    headers = {"x-apikey": api_key}
    # Corrected Data Submission Format
    response = requests.get(f"{base_url}/{encoded_url}", headers=headers)

```

```
    if response.status_code == 200:
        return response.json()
    else:
        return {"error": f"VirusTotal submission failed with status code {response.status_code}"}

def main():
    # API Key for VirusTotal (Replace with your actual API key)
    api_key = "80622f5e7e38f3b81381aec0be8aa528bf6186411b8dfa92fe9d8084a89b3ddd"

    # Get URL input from user
    url = input("Enter the URL to analyze: ")

    # Extract URL Information
    url_info = extract_url_info(url)

    # Print extracted URL info
    print("\nExtracted URL Information:")
    for key, value in url_info.items():
        print(f"{key}: {value}")

    # Get VirusTotal results
    vt_result = check_virustotal(api_key, url)
    print("\nVirusTotal Analysis Result:")

    if "error" in vt_result:
        print(vt_result["error"])
    else:
        # Display all available details
        if "data" in vt_result:
            data = vt_result["data"]
```

```
            # Display more detailed information from the response
            print("\nFull Analysis Data:")
            print("ID: ", data["id"])
            print("Type: ", data["type"])

            if "attributes" in data:
                attributes = data["attributes"]
                print("Last Analysis Date: ", attributes.get("last_analysis_date"))
                print("First Submission Date: ", attributes.get("first_submission_date"))
                print("Total Analysis Count: ", attributes.get("total_votes"))
                print("Harmless Count: ", attributes["last_analysis_stats"].get("harmless", "N/A"))
                print("Malicious Count: ", attributes["last_analysis_stats"].get("malicious", "N/A"))
                print("Suspicious Count: ", attributes["last_analysis_stats"].get("suspicious", "N/A"))
                print("Undetected Count: ", attributes["last_analysis_stats"].get("undetected", "N/A"))

                # Display engine details
                print("\nAnalysis Engines:")
                for engine, result in attributes["last_analysis_results"].items():
                    print(f"Engine: {engine}, Verdict: {result.get('category', 'No verdict')}, Result: {result.get('result', 'N/A')}")

                # URL Information
                print("\nURL Information:")
                print("URL: ", url)
                print("Domain: ", url_info["Domain"])
                print("Path: ", url_info["Path"])
                print("Scheme: ", url_info["Scheme"])
                print("Port: ", url_info["Port"])

if __name__ == "__main__":
    main()
```

Figure 78 : Security: URL

5.4.8. Whois Lookup

```python
import whois

def whois_lookup(domain):
    try:
        # Perform WHOIS lookup
        domain_info = whois.whois(domain)

        # Display relevant information
        print(f"Domain Name: {domain_info.domain_name}")
        print(f"Registrar: {domain_info.registrar}")
        print(f"Creation Date: {domain_info.creation_date}")
        print(f"Expiration Date: {domain_info.expiration_date}"
        print(f"Updated Date: {domain_info.updated_date}")
        print(f"Name Servers: {domain_info.name_servers}")
        print(f"Status: {domain_info.status}")
        print(f"Owner:{ domain_info.org}")
    except Exception as e:
        print(f"Error: {e}")

# Get domain input from user
domain = input("Enter the domain you want to look up: ")
whois_lookup(domain)
```

Figure 79 : Security: whois lookup

5.4.9. Full

```python
import os
import re
import requests
import hashlib
from email import policy
from email.parser import BytesParser
from email.utils import getaddresses
import whois
from os import system


RED, WHITE, YELLOW, CIANO, GREEN, END = '\033[91m', '\033[46m', '\033[93m', '\033[100m', '\033[1;32m', '\033[0m'


def runPEnv():
    system('clear')
    print('''
     __  __       _ _        _____                          _
    |  \/  |     (_) |      |_   _|                        | |
    | \  / | __ _ _| | ___    | |  _ __  ___  _ __ ___   __ _| |_
    | |\/| |/ _` | | |/ _ \   | | | '__/ _ \| '_ ` _ \ / _` | __|
    | |  | | (_| | | |  __/  _| |_| | | (_) | | | | | | (_| | |_
    |_|  |_|\__,_|_|_|\___| |_____|_|  \___/|_| |_| |_|\__,_|\__|
                                                                {1}

                        [ {0} MailInspector  {1}|{0}   Unmasking phishing  {1}]\n\n'''.format(RED, END))


def colorize_value(value):
    if isinstance(value, str) and any(s in value.lower() for s in ['none', 'fail', 'not found']):
```

```
        return f"{RED}{value}{END}"
    else:
        return value


def print_green(message):
    print(f"{GREEN}{message}{END}")


def check_virustotal_domain(domain):
    api_key = "7019e4123a3e38c9ed8f8afd087ace44d8a02cb686b5f0227d60b59d8cc8a3eb"
    url = f"https://www.virustotal.com/api/v3/domains/{domain}"
    headers = {"x-apikey": api_key}
    try:
        response = requests.get(url, headers=headers)
        if response.status_code == 200:
            data = response.json()
            return data.get("data", {}).get("attributes", {}).get("last_analysis_stats", {})
        else:
            return {"error": f"Failed to retrieve data, status code: {response.status_code}"}
    except Exception as e:
        return {"error": str(e)}


def check_virustotal_url(url_to_check):
    api_key = "7019e4123a3e38c9ed8f8afd087ace44d8a02cb686b5f0227d60b59d8cc8a3eb"
    url = "https://www.virustotal.com/api/v3/urls"
    headers = {"x-apikey": api_key}
    try:
        data = {"url": url_to_check}
        response = requests.post(url, headers=headers, data=data)
        if response.status_code == 200:
            json_response = response.json()
            url_id = json_response["data"]["id"]
            analysis_url = f"https://www.virustotal.com/api/v3/analyses/{url_id}"
            analysis_response = requests.get(analysis_url, headers=headers)
            if analysis_response.status_code == 200:
                analysis_data = analysis_response.json()
                stats = analysis_data.get("data", {}).get("attributes", {}).get("stats", {})
                return stats
            else:
                return {"error": f"Failed to retrieve analysis, status code: {analysis_response.status_code}"}
        else:
            return {"error": f"Failed to submit URL, status code: {response.status_code}"}
    except Exception as e:
        return {"error": str(e)}


def check_virustotal_file_hash(file_hash):
    api_key = "7019e4123a3e38c9ed8f8afd087ace44d8a02cb686b5f0227d60b59d8cc8a3eb"
    url = f"https://www.virustotal.com/api/v3/files/{file_hash}"
    headers = {"x-apikey": api_key}
    try:
        response = requests.get(url, headers=headers)
        if response.status_code == 200:
            data = response.json()
            return data.get("data", {}).get("attributes", {}).get("last_analysis_stats", {})
        elif response.status_code == 404:
            return {"error": "File not found in VirusTotal database."}
        else:
```

```
            return {"error": f"Failed to retrieve data, status code: {response.status_code}"}
    except Exception as e:
        return {"error": str(e)}


def get_whois_info(domain):
    try:
        domain_info = whois.whois(domain)
        return {
            "creation_date": domain_info.creation_date,
            "expiration_date": domain_info.expiration_date,
            "organization": domain_info.org,
        }
    except Exception as e:
        return {"error": str(e)}

def read_email_file(file_path):
    try:
        with open(file_path, 'rb') as f:
            msg = BytesParser(policy=policy.default).parse(f)
        return msg
    except Exception as e:
        print(f"An error occurred while reading the email file: {e}")
        return None


def extract_basic_email_details(msg):
    sender = msg['From']
    recipient = msg['To']
    reply_to = msg['Reply-To']
```

```
    return_path = msg['Return-Path']
    date = msg['Date']
    subject = msg['Subject']
    sender_email, sender_name, sender_domain, sender_ip = None, None, None, "Not found"

    if sender:
        match = re.match(r'(.*)<(.*)>', sender)
        if match:
            sender_name = match.group(1).strip()
            sender_email = match.group(2).strip()
        else:
            sender_email = sender.strip()

    if sender_email:
        domain_match = re.search(r'@([a-zA-Z0-9.-]+)$', sender_email)
        sender_domain = domain_match.group(1) if domain_match else None

    spf, dmarc, dkim = "Not found in headers", "Not found", "Not found"
    for header in msg.keys():
        if header.lower() == "received-spf":
            spf = msg[header]
        elif header.lower().startswith("authentication-results"):
            auth_results = msg[header]
            if "dmarc=" in auth_results:
                dmarc_match = re.search(r"dmarc=(\w+)", auth_results)
                dmarc = dmarc_match.group(1) if dmarc_match else "Not found"
            if "dkim=" in auth_results:
                dkim_match = re.search(r"dkim=(\w+)", auth_results)
                dkim = dkim_match.group(1) if dkim_match else "Not found"

```

```
    return {
        "date": date,
        "sender_name": sender_name,
        "sender_email": sender_email,
        "sender_domain": sender_domain,
        "sender_ip": sender_ip,
        "reply_to": reply_to,
        "return_path": return_path,
        "spf": spf,
        "dmarc": dmarc,
        "dkim": dkim,
        "recipient": recipient,
        "subject": subject,
    }


def print_email_details(details):
    print("\n--- Email Details ---")
    print(f"{GREEN}Date:{END} {colorize_value(details.get('date', 'N/A'))}")
    print(f"{GREEN}Sender Name:{END} {colorize_value(details.get('sender_name', 'N/A'))}")
    print(f"{GREEN}Sender Email:{END} {colorize_value(details.get('sender_email', 'N/A'))}")
    print(f"{GREEN}Sender Domain:{END} {colorize_value(details.get('sender_domain', 'N/A'))}")
    print(f"{GREEN}Sender IP:{END} {colorize_value(details.get('sender_ip', 'Not found'))}")
    print(f"{GREEN}Reply-To:{END} {colorize_value(details.get('reply_to', 'N/A'))}")
    print(f"{GREEN}Return-Path:{END} {colorize_value(details.get('return_path', 'N/A'))}")
    print(f"{GREEN}SPF:{END} {colorize_value(details.get('spf', 'Not found in headers'))}")
    print(f"{GREEN}DMARC:{END} {colorize_value(details.get('dmarc', 'Not found'))}")
    print(f"{GREEN}DKIM:{END} {colorize_value(details.get('dkim', 'Not found'))}")
    print(f"{GREEN}Recipient Email:{END} {colorize_value(details.get('recipient', 'N/A'))}")
    print(f"{GREEN}Subject:{END} {colorize_value(details.get('subject', 'N/A'))}")
```

```
    print("----------------------")


def analyze_sender_domain_and_ip(details):
    sender_domain = details.get('sender_domain')

    virustotal_domain_results = check_virustotal_domain(sender_domain) if sender_domain else None
    whois_info = get_whois_info(sender_domain) if sender_domain else None

    print("\n--- Domain Analysis ---")
    if virustotal_domain_results:
        if "error" in virustotal_domain_results:
            print(f"{GREEN}VirusTotal Check:{END} {colorize_value(virustotal_domain_results['error'])}")
        else:
            print("VirusTotal Domain Last Analysis Stats:")
            for key, value in virustotal_domain_results.items():
                print(f"  {GREEN}{key.capitalize()}:{END} {colorize_value(value)}")
    else:
        print("VirusTotal Check: No domain to analyze.")

    if whois_info:
        if "error" in whois_info:
            print(f"{GREEN}WHOIS Info:{END} {colorize_value(whois_info['error'])}")
        else:
            print("WHOIS Information:")
            print(f"  {GREEN}Creation Date:{END} {colorize_value(whois_info.get('creation_date', 'N/A'))}")
            print(f"  {GREEN}Expiration Date:{END} {colorize_value(whois_info.get('expiration_date', 'N/A'))}")
            print(f"  {GREEN}Organization:{END} {colorize_value(whois_info.get('organization', 'N/A'))}")
    else:
        print("WHOIS Info: No domain to analyze.")
```

```
    print("----------------------")


def check_suspicious_subject(msg, suspicious_words):
    try:
        subject = msg['Subject']
        if not subject:
            print("No subject found.")
            return

        for word in suspicious_words:
            if word.lower() in subject.lower():
                print(f"{GREEN}Suspicious subject detected:{END} '{subject}' contains the word '{word}'.")
                return

        print("No suspicious words found in the subject.")
    except Exception as e:
        print(f"An error occurred while checking the subject: {e}")


def extract_urls_from_email(msg):
    urls = []
    try:
        if msg.is_multipart():
            parts = msg.walk()
            body = ""
            for part in parts:
                if part.get_content_type() == 'text/plain':
                    body += part.get_content()
        else:
            body = msg.get_content()

        url_regex = re.compile(r'((?:http|ftp)s?://[^\s/$.?#].[^\s]*)', re.IGNORECASE)
        urls = url_regex.findall(body)

        for url in urls:
            print(f"\n{CIANO}Analyzing URL: {url}{END}")
            stats = check_virustotal_url(url)
            if "error" in stats:
                print(f"{RED}VirusTotal Error:{END} {stats['error']}")
            else:
                print("VirusTotal URL Last Analysis Stats:")
                for key, value in stats.items():
                    print(f"  {GREEN}{key.capitalize()}:{END} {colorize_value(value)}")

        return urls
    except Exception as e:
        print(f"An error occurred while extracting and analyzing URLs: {e}")
        return []


def extract_attachments_from_email(msg, output_dir='attachments'):
    attachments = []
    try:
        os.makedirs(output_dir, exist_ok=True)
        for part in msg.walk():
            if part.get_content_maintype() == 'multipart':
                continue
            filename = part.get_filename()
            if filename:
```

```
                filepath = os.path.join(output_dir, filename)
                with open(filepath, 'wb') as fp:
                    content = part.get_payload(decode=True)
                    fp.write(content)
                attachments.append(filepath)

                sha256_hash = hashlib.sha256(content).hexdigest()
                print(f"\n{YELLOW}Analyzing attachment: {filename}{END}")
                analysis = check_virustotal_file_hash(sha256_hash)
                if "error" in analysis:
                    print(f"{RED}VirusTotal Error:{END} {analysis['error']}")
                else:
                    print("VirusTotal Attachment Analysis Stats:")
                    for key, value in analysis.items():
                        print(f"  {GREEN}{key.capitalize()}:{END} {colorize_value(value)}")

        return attachments
    except Exception as e:
        print(f"An error occurred while extracting attachments: {e}")
        return []


def main():
    runPEnv()
    print_green("Welcome to the Email Analysis Tool!")
    file_path = input("Please enter the path to the email file (eml format): ").strip()

    if not os.path.isfile(file_path):
        print("Error: File not found. Please ensure the path is correct.")
        return

    try:
        msg = read_email_file(file_path)
        if not msg:
            print("Failed to read the email file.")
            return
        print_green("\nExtracting basic email details...")
        email_details = extract_basic_email_details(msg)
        print_email_details(email_details)

        print_green("\nAnalyzing sender domain...")
        analyze_sender_domain_and_ip(email_details)

        print_green("\nChecking for suspicious subject keywords...")
        suspicious_words = ["urgent", "invoice", "payment", "sensitive", "action required"]
        check_suspicious_subject(msg, suspicious_words)

        print_green("\nExtracting URLs from the email...")
        urls = extract_urls_from_email(msg)
        if not urls:
            print("No URLs found in the email.")

        print_green("\nExtracting attachments from the email...")
        attachments = extract_attachments_from_email(msg)
        if not attachments:
            print("No attachments found.")

    except Exception as e:
        print(f"An error occurred during the analysis: {e}")

if __name__ == '__main__':
    main()
```

Figure 80 : Security: full email

5.5 Machine Learning

5.5.1 Dataset Definition

This dataset contains 48 features extracted from 5000 phishing webpages and 5000 legitimate webpages, which were downloaded from January to May 2015 and from May to June 2017. An improved feature extraction technique is employed by leveraging the browser automation framework (i.e., Selenium WebDriver), which is more precise and robust compared to the parsing approach based on regular expressions.

Anti-phishing researchers and experts may find this dataset useful for phishing features analysis, conducting rapid proof of concept experiments or benchmarking phishing classification models.

5.5.2. Dealing With Data

Importing Libraries 🔽

[6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm
from scipy import stats
from scipy.stats import yeojohnson
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score
import optuna
import warnings
warnings.filterwarnings('ignore', category=FutureWarning, module='seaborn._oldcore')
import time
```

Reading Data and Exploration 📄 📊

[8]:
```
df = pd.read_csv(r"D:\Graduation Project\Phishing_Legitimate_full.csv")
```

[9]:
```
df.head(20)
```

[9]:

	id	NumDots	SubdomainLevel	PathLevel	UrlLength	NumDash	NumDashInHostname	AtSymbol	TildeSymbol	NumUnderscore	...	IframeOrFrame	MissingTitle
0	1	3	1	5	72	0	0	0	0	0	...	0	0
1	2	3	1	3	144	0	0	0	0	2	...	0	0
2	3	3	1	2	58	0	0	0	0	0	...	0	0
3	4	3	1	6	79	1	0	0	0	0	...	0	0
4	5	3	0	4	46	0	0	0	0	0	...	1	0
5	6	3	1	1	42	1	0	0	0	0	...	1	1
6	7	2	0	5	60	0	0	0	0	0	...	0	0
7	8	1	0	3	30	0	0	0	0	0	...	0	0
8	9	8	7	2	76	1	1	0	0	0	...	0	0
9	10	2	0	2	46	0	0	0	0	0	...	0	0
10	11	5	4	2	64	1	1	0	0	0	...	0	0
11	12	2	0	2	47	0	0	0	0	0	...	0	0

[14]:
```
print(f"{df.dtypes}\n")
print(f"Dimension: {df.shape[0]} x {df.shape[1]}\n")

datatype_counts = df.dtypes.value_counts()
for dtype, count in datatype_counts.items():
    print(f"{dtype}: {count} columns")
```

```
NumDots                         int64
SubdomainLevel                  int64
PathLevel                       int64
UrlLength                       int64
NumDash                         int64
NumDashInHostname               int64
NumUnderscore                   int64
NumPercent                      int64
NumQueryComponents              int64
NumAmpersand                    int64
NumNumericChars                 int64
RandomString                    int64
DomainInPaths                   int64
HostnameLength                  int64
PathLength                      int64
QueryLength                     int64
NumSensitiveWords               int64
PctExtHyperlinks              float64
```

```
[20]: sns.pairplot(df[continuous_columns], size = 2.5)
      plt.show()
```

```
D:\Anaconda\Lib\site-packages\seaborn\axisgrid.py:2100: UserWarning: The `size` parameter has been renamed to `height`; please update your code.
  warnings.warn(msg, UserWarning)
```

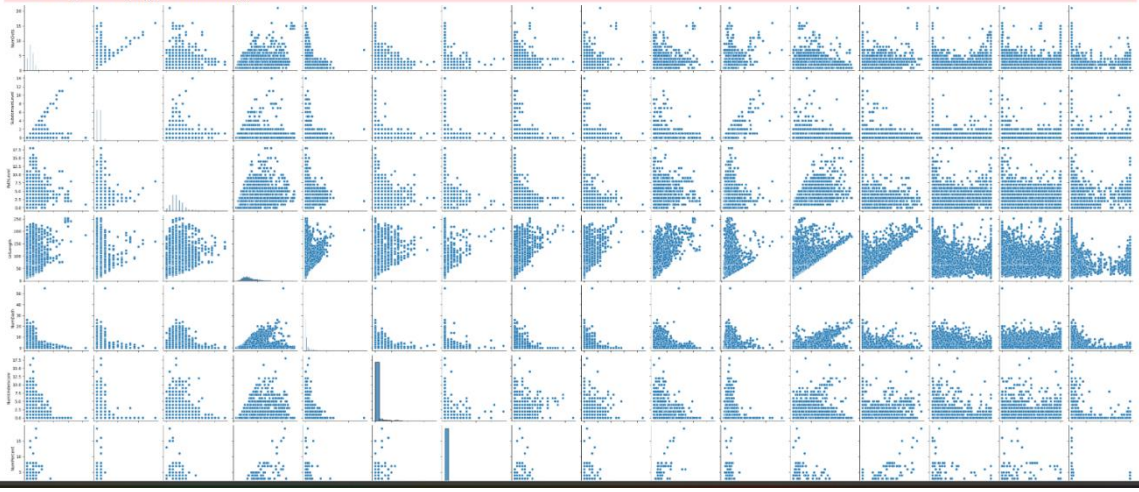


Figure 81 : Machine Learning 1

5.5.3. Data Cleaning

- See Null Values

```
[17]: null = df.isnull().sum()
      for i in range(len(df.columns)):
          print(f"{df.columns[i]}: {null[i]} ({(null[i]/len(df))*100}%)")
      total_cells = np.prod(df.shape)
      total_missing = null.sum()
      print(f"\nTotal missing values: {total_missing} ({(total_missing/total_cells) * 100}%)\n")
```

```
NumDots: 0 (0.0%)
SubdomainLevel: 0 (0.0%)
PathLevel: 0 (0.0%)
UrlLength: 0 (0.0%)
NumDash: 0 (0.0%)
NumDashInHostname: 0 (0.0%)
NumUnderscore: 0 (0.0%)
NumPercent: 0 (0.0%)
NumQueryComponents: 0 (0.0%)
NumAmpersand: 0 (0.0%)
NumNumericChars: 0 (0.0%)
RandomString: 0 (0.0%)
DomainInPaths: 0 (0.0%)
HostnameLength: 0 (0.0%)
PathLength: 0 (0.0%)
QueryLength: 0 (0.0%)
NumSensitiveWords: 0 (0.0%)
PctExtHyperlinks: 0 (0.0%)
```

The data is clear of nulls, we got 0 nulls for all columns!. Next we will check for outliers in the data.

Data Cleaning

There are few task that we have to accompolish in this process:

- Removing Outliers
- Applying transformation to achieve normal distribution

Firstly, we will remove the outliers with the condition below:

- NumDash > 40
- NumDots > 20

```
32]: df = df[df['NumDots'] < 20]
     df = df[df['NumDash'] < 40]
```

```
33]: plt.scatter(x=df['NumDots'], y=df['NumDash'])
     plt.xlabel('NumDots')
     plt.ylabel('NumDash')
     plt.show()
```

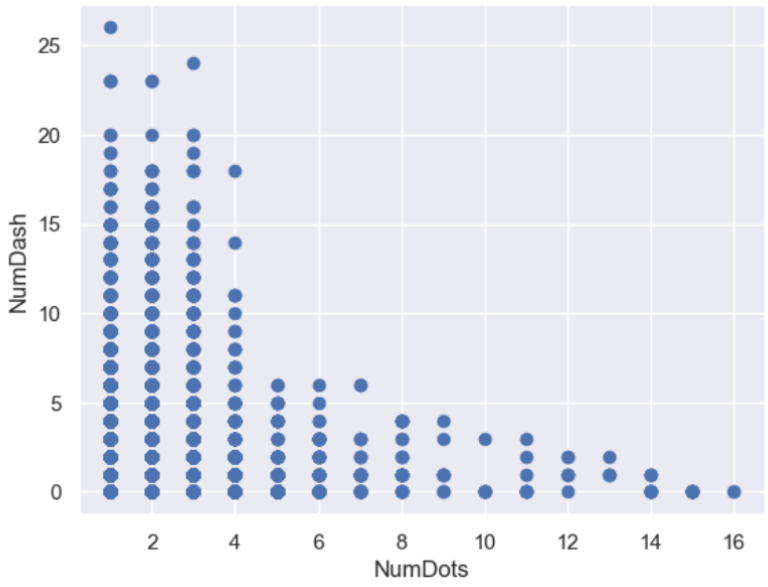


Figure 82 : Machine Learning 2

- Remove Outlier

The outliers have been removed, now we will do transformation to achieve normal distribution within the continuous data.

```
[35]: for col in continuous_columns:
          df[col], _ = yeojohnson(df[col])
```

Let's check the distribution after applying Yeo-Johnson transformation to our data

```
[37]: for col_name in continuous_columns:
          fig1, ax1 = plt.subplots()
          sns.histplot(x=df[col_name], ax=ax1)
          normal(df[col_name].mean(), df[col_name].std())

          fig2, ax2 = plt.subplots()
          stats.probplot(df[col_name], plot=ax2)

          plt.show()
```

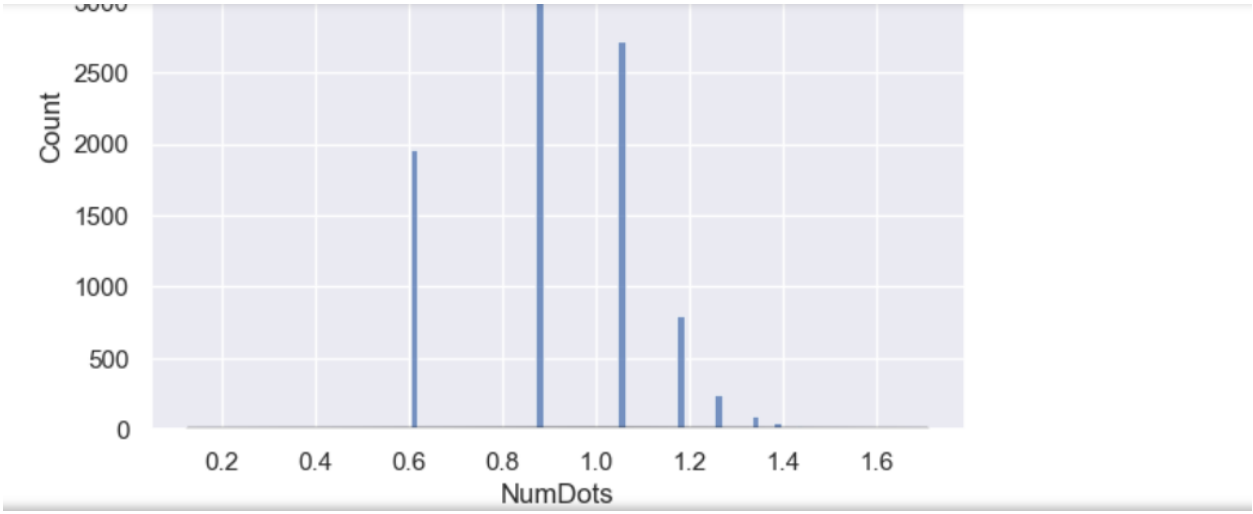


Figure 83 : Machine Learning 3

5.5.4. Training & Accuracy

Figure 84 : Machine Learning 4

- Model Algorithm

```
[43]: def objective(trial):
          n_estimators = trial.suggest_int('n_estimators', 10, 300)
```

Now we will train the model with the best hyperparameter that we got from the tuning before. We will utilize random forest model for this task.

```
[45]: start_time = time.time()

[46]: model = RandomForestClassifier(n_estimators=results['n_estimators'], max_depth=results['max_depth'], min_samples_split=results['min_samples_split'], min_
      model.fit(X_train, y_train)

[46]: RandomForestClassifier
      RandomForestClassifier(max_depth=25, n_estimators=223, n_jobs=-1,
                             random_state=42)
```

```
    )

    clf.fit(X_train, y_train)

    y_pred = clf.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    return accuracy

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=100)

best_trial = study.best_trial
results = best_trial.params
```

Figure 85 : Machine Learning 5

- Accuracy Confusion Matrix

Lastly, we will check the model performance

```
[48]: y_pred = model.predict(X_test)
      print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      1000
           1       0.99      0.99      0.99      1000

    accuracy                           0.99      2000
   macro avg       0.99      0.99      0.99      2000
weighted avg       0.99      0.99      0.99      2000
```

```
[49]: from sklearn.metrics import confusion_matrix

[50]: confusion_matrix(y_test, y_pred  )

[50]: array([[989,  11],
             [ 10, 990]], dtype=int64)

[51]:
      end_time = time.time()
      training_time = end_time - start_time

[52]: print(f"Time taken to train the model: {training_time:.2f} seconds")

      Time taken to train the model: 0.65 seconds
```

Great!, we got 99% accuracy for all class which is balanced!.

Figure 86 : Machine Learning 6

5.5.5. Extract Features

1 -

```
]: import re
from urllib.parse import urlparse, parse_qs
import tldextract
import requests

def contains_sensitive_words(text):
    sensitive_words = ['login', 'password', 'admin', 'secure', 'login', 'signin', 'account', 'user']
    return sum(1 for word in sensitive_words if word in text.lower())

def is_external_url(url, domain):
    return domain not in url

def analyze_url(url):
    features = []

    num_dots = url.count('.')
    features.append(num_dots)

    ext = tldextract.extract(url)
    subdomain_level = len(ext.subdomain.split('.')) if ext.subdomain else 0
    features.append(subdomain_level)

    path_level = len(urlparse(url).path.strip('/').split('/')) if urlparse(url).path else 0
    features.append(path_level)

    url_length = len(url)
    features.append(url_length)

    num_dash = url.count('-')
    features.append(num_dash)
```

2 -

```
    hostname = urlparse(url).hostname
    num_dash_in_hostname = hostname.count('-') if hostname else 0
    features.append(num_dash_in_hostname)

    num_underscore = url.count('_')
    features.append(num_underscore)

    num_percent = url.count('%')
    features.append(num_percent)

    query = urlparse(url).query
    num_query_components = len(parse_qs(query)) if query else 0
    features.append(num_query_components)

    num_ampersand = url.count('&')
    features.append(num_ampersand)

    num_numeric_chars = len(re.findall(r'\d', url))
    features.append(num_numeric_chars)

    random_string = 1 if len(url) > 50 else 0
    features.append(random_string)

    path = urlparse(url).path
    domain_in_paths = 1 if ext.domain in path else 0
    features.append(domain_in_paths)

    hostname_length = len(hostname) if hostname else 0
    features.append(hostname_length)
```

3 -

```
path_length = len(urlparse(url).path)
features.append(path_length)

query_length = len(urlparse(url).query)
features.append(query_length)

sensitive_words_count = contains_sensitive_words(url) + contains_sensitive_words(urlparse(url).path)
features.append(sensitive_words_count)

pct_ext_hyperlinks = 0
features.append(pct_ext_hyperlinks)

pct_ext_resource_urls = 0
features.append(pct_ext_resource_urls)

ext_favicon = 1 if "favicon" in url else 0
features.append(ext_favicon)

insecure_forms = 0
features.append(insecure_forms)

relative_form_action = 1 if 'action' in url else 0
features.append(relative_form_action)

ext_form_action = 1 if is_external_url(urlparse(url).geturl(), ext.domain) else 0
features.append(ext_form_action)

abnormal_form_action = 0
features.append(abnormal_form_action)
```

4 -

```
    pct_ext_resource_urls_rt = 0
    features.append(pct_ext_resource_urls_rt)

    abnormal_ext_form_action_r = 0
    features.append(abnormal_ext_form_action_r)

    ext_meta_script_link_rt = 0
    features.append(ext_meta_script_link_rt)

    pct_ext_null_self_redirect_hyperlinks_rt = 0
    features.append(pct_ext_null_self_redirect_hyperlinks_rt)

    return features
```

Figure 87 : Machine Learning 7

5.5.6. Test Model

```
    pct_ext_resource_urls_rt = 0
    features.append(pct_ext_resource_urls_rt)

    abnormal_ext_form_action_r = 0
    features.append(abnormal_ext_form_action_r)

    ext_meta_script_link_rt = 0
    features.append(ext_meta_script_link_rt)

    pct_ext_null_self_redirect_hyperlinks_rt = 0
    features.append(pct_ext_null_self_redirect_hyperlinks_rt)

    return features

url = "https://example.com/path/to/resource-with-dashes?query=value&param=2"
features = analyze_url(url)
print(features)
```

```
[1, 0, 3, 68, 2, 0, 0, 0, 2, 1, 1, 1, 0, 11, 29, 19, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

```
02]: phishing="http://secure-login.paypall-security.com/account/verify"
p=analyze_url(phishing)
prediction1 = model.predict([p])
print(prediction1 )
```

Figure 88 : Machine Learning 8

5.5.7. API Code

```
from flask import Flask, request, jsonify
from joblib import load
import numpy as np
import threading
import re
from urllib.parse import urlparse, parse_qs
import tldextract
import requests

def contains_sensitive_words(text):
    sensitive_words = ['login', 'password', 'admin', 'secure', 'login', 'signin', 'account', 'user']
    return sum(1 for word in sensitive_words if word in text.lower())

def is_external_url(url, domain):
    return domain not in url

def analyze_url(url):
    features = []

    num_dots = url.count('.')
    features.append(num_dots)

    ext = tldextract.extract(url)
    subdomain_level = len(ext.subdomain.split('.')) if ext.subdomain else 0
    features.append(subdomain_level)

    path_level = len(urlparse(url).path.strip('/').split('/')) if urlparse(url).path else 0
    features.append(path_level)

    url_length = len(url)
    features.append(url_length)

    num_dash = url.count('-')
    features.append(num_dash)

    hostname = urlparse(url).hostname
    num_dash_in_hostname = hostname.count('-') if hostname else 0
```

```
num_dash_in_hostname = hostname.count('-') if hostname else 0
features.append(num_dash_in_hostname)

num_underscore = url.count('_')
features.append(num_underscore)

num_percent = url.count('%')
features.append(num_percent)

query = urlparse(url).query
num_query_components = len(parse_qs(query)) if query else 0
features.append(num_query_components)

num_ampersand = url.count('&')
features.append(num_ampersand)

num_numeric_chars = len(re.findall(r'\d', url))
features.append(num_numeric_chars)

random_string = 1 if len(url) > 50 else 0
features.append(random_string)

path = urlparse(url).path
domain_in_paths = 1 if ext.domain in path else 0
features.append(domain_in_paths)

hostname_length = len(hostname) if hostname else 0
features.append(hostname_length)

path_length = len(urlparse(url).path)
features.append(path_length)

query_length = len(urlparse(url).query)
features.append(query_length)
```

```
    features.append(pct_ext_null_self_redirect_hyperlinks_rt)

    return features



model = load("model.joblib")
app = Flask(__name__)

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    url=data['url']
    features = analyze_url(url)
    input_array = np.array(features).reshape(1, -1)
    prediction = model.predict(input_array)
    return jsonify({'prediction': prediction.tolist()})

# تشغيل السيرفر في Thread منفصل
def run_flask():
    app.run(debug=True, use_reloader=False)

thread = threading.Thread(target=run_flask)
thread.start()
```

Figure 89 : Machine Learning 9

Chapter (6)

Chapter 6: Conclusion & Future Work

6.1 Conclusion.

Our graduation project introduces an innovative and comprehensive approach to addressing the growing threat of phishing attacks through email communication.

Phishing remains one of the most prevalent and dangerous cybersecurity threats, aiming to deceive users into revealing sensitive information or compromising their systems. Traditional email filters and manual inspection methods, while useful, often fall short of effectively detecting sophisticated phishing attempts. Our project aims to go beyond these limitations by leveraging cutting-edge technologies to create an intelligent and automated phishing detection system.

We have seamlessly integrated Artificial Intelligence (AI), machine learning algorithms, threat intelligence APIs, and advanced email parsing techniques to develop a system that accurately analyzes email contents, attachments, headers, and embedded links. By combining multiple layers of analysis, our system significantly enhances detection capabilities and offers a robust solution to help both users and cybersecurity analysts identify malicious emails early and effectively.

The core of our system is the intelligent analysis engine that extracts critical features from emails, evaluates them using pre-trained AI models, and cross-verifies threats through integration with external security services. This multi-faceted approach ensures a high detection accuracy, reduces false positives, and provides comprehensive reporting for informed decision-making.

Our user-friendly interface and API-driven backend allow seamless submission and evaluation of suspicious emails, making the system accessible to both technical and non-technical users. This design ensures that the protective capabilities of advanced phishing detection are available to a broader audience, beyond traditional security teams.

While implementing robust feature extraction, integrating external APIs, and training reliable machine learning models presented notable challenges, we remained committed to refining our system to ensure scalability, accuracy, and efficiency. Handling various email formats, ensuring secure communication with APIs, and optimizing AI performance required extensive testing and continuous enhancement.

We believe that our project has the potential to significantly improve the early detection of phishing threats, thereby reducing risks, protecting user data, and strengthening organizational cybersecurity defenses.

By integrating automation, intelligence, and user-centered design, we pave the way for a more secure digital environment. Our project not only addresses immediate cybersecurity needs but also contributes to the broader mission of raising phishing awareness and empowering users to defend themselves proactively.

We are committed to advancing this work, exploring new techniques in threat detection, and enhancing the system's capabilities to meet the ever-evolving landscape of cybersecurity threats.

6.2 Future Works.

While our graduation project has laid the foundation for a comprehensive and intelligent phishing email detection system, several avenues for future work and improvement remain to be explored:

1. Integration of Advanced Threat Intelligence Sources:

In the future, we aim to expand our system's capabilities by integrating additional threat intelligence platforms, such as dark web monitoring, domain reputation services, and real-time threat feeds. This would provide broader coverage and enable even earlier detection of emerging phishing campaigns.

2. Enhancement of Machine Learning Models:

Continuous refinement of our AI models is crucial to keep up with evolving phishing tactics. This involves training on larger, more diverse datasets, applying advanced deep learning techniques like transformers or graph-based learning, and implementing ensemble models to boost detection accuracy and reduce false positives.

3. Development of a Browser Extension:

Extending our system to offer a real-time browser extension could provide immediate phishing warnings to users as they interact with suspicious emails or links. Such an extension would enhance user protection by seamlessly integrating into daily browsing activities without requiring manual email submission.

4. Enhancing User Interface and Reporting:

Future work includes improving the system's user interface by offering more detailed, user-friendly reports that highlight key phishing indicators. Adding visual dashboards for SOC analysts to monitor phishing trends and receive actionable insights would further empower effective decision-making.

5. Expansion to Multilingual Phishing Detection:

Since phishing attacks are increasingly targeting users in various languages, extending the system to support multilingual analysis would be a critical enhancement. This requires training

language-specific models and developing intelligent text processing pipelines for non-English phishing content.

6. Collaboration with Cybersecurity Experts and Organizations:

Collaborating with cybersecurity professionals and organizations will allow us to align our system with real-world security operations practices. Such partnerships will help validate our system's effectiveness, keep it updated with the latest phishing trends, and ensure it meets professional standards.

7. Scalability and Cloud Deployment:

To accommodate large-scale enterprise environments, future work includes optimizing the system for cloud deployment. Leveraging cloud infrastructure will enhance scalability, availability, and performance, allowing organizations to integrate phishing detection into their cybersecurity ecosystems seamlessly.

Through these future enhancements, we envision a continuously evolving platform that stays ahead of cyber threats and offers a robust, intelligent shield against phishing attacks, contributing meaningfully to the security and resilience of digital communication environments.

References

- The Latest Phishing Statistics (updated June 2025) | AAG IT Support
- Phishing Exploration & Classification 99% 💻(dataset)
- Characteristics of Understanding URLs and Domain Names Features: The Detection of Phishing Websites With Machine Learning Methods | IEEE Journals & Magazine | IEEE Xplore
- Phishing detection: A recent intelligent machine learning comparison based on models content and features | IEEE Conference Publication | IEEE Xplore
- Detection of Phishing Attacks: A Machine Learning Approach | SpringerLink
- (PDF) Phishing Detection: Analysis of Visual Similarity Based Approaches
- Bergholz, A., De Beer, J., Glahn, S., & Moens, M. (2010). A real-life study of phishing attacks. *CEAS 2010*.
- Mohammad, R. M., Thabtah, F., & McCluskey, L. (2015). Predicting phishing websites using classification techniques. *Applied Computing and Informatics*, 11(1), 1–12.
- Marchal, S., Saari, K., Singh, N., & Asokan, N. (2016). Know your phish: Novel techniques for detecting phishing sites and their targets. *IEEE Internet Computing*, 20(1), 54–61.
- Ma, J., Saul, L. K., Savage, S., & Voelker, G. M. (2009). Beyond blacklists: Learning to detect malicious Web sites from suspicious URLs. *KDD '09*, 1245–1254.
- Fette, I., Sadeh, N., & Tomasic, A. (2007). Learning to detect phishing emails. *WWW '07 Proceedings*, 649–656.
- Xiang, G., et al. (2011). Content-based phishing detection using machine learning. *IEEE Symposium on Security and Privacy*, 583–596.
- Abu-Nimeh, S., Nappa, D., Wang, X., & Nair, S. (2007). A comparison of machine learning techniques for phishing detection. *eCrime Researchers Summit*.
- Dou, W., et al. (2017). Detecting phishing websites using a hybrid model. *Future Generation Computer Systems*, 74, 371–379.
- Ramesh, M. V., & Vaishnavi, R. (2016). Mobile-based phishing detection system using deep learning. *IEEE International Conference on Communications*, 1–6.
- Verma, R., & Hossain, N. (2014). Semantic analysis for phishing detection. *IEEE eCrime Researchers Summit*, 1–12.
- Pan, Y., & Ding, X. (2006). Anomaly-based phishing detection. *IWIA 2006*, 15–22.

- Liu, W., et al. (2005). Detecting phishing web pages with visual similarity assessment. *WWW '05*, 1060–1061.
- Amrutkar, C., et al. (2011). PhishZoo: Detecting phishing websites by looking at them. *ACM Symposium on Applied Computing*, 927–933.
- Herzberg, A., & Gbara, A. (2006). TrustBar: Protecting (even naïve) web users from spoofing and phishing attacks. *Cryptology and Network Security*, 72–87.
- Chiew, K. L., Yong, K. S. C., & Tan, C. L. (2018). A survey of phishing attacks: Their types, vectors and technical approaches. *Expert Systems with Applications*, 106, 1–20.
- Ali, A., & Aydin, N. (2020). A phishing detection system based on machine learning algorithms. *PeerJ Computer Science*, 6, e271.
- Raghavan, S., & Venkatesan, V. P. (2013). PhishGuard: A browser plug-in for protection from phishing attacks. *ICACCI*, 1796–1801.
- Gupta, B. B., Tewari, A., Jain, A. K., & Agrawal, D. P. (2017). Fighting against phishing attacks: state of the art and future challenges. *Neural Computing and Applications*, 28(12), 3629–3654.
- Canali, D., Cova, M., Vigna, G., & Kruegel, C. (2011). Prophiler: a fast filter for the large-scale detection of malicious web pages. *Proceedings of the 20th international conference on World wide web (WWW)*, 197–206.
- Chiew, K. L., Yong, K. S. C., & Tan, C. L. (2019). A survey of phishing attacks and defenses. *ACM Computing Surveys (CSUR)*, 52(4), 1–34.
- Sharma, P., & Kalra, P. (2019). A framework for phishing detection and classification using deep learning. *International Journal of Information Security and Privacy (IJISP)*, 13(3), 1–18.
- Le, A., Markopoulou, A., & Faloutsos, M. (2011). PhishDef: URL names say it all. *IEEE INFOCOM*, 191–195.
- Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117, 345–357.
- Sahoo, D., Liu, C., & Hoi, S. C. H. (2017). Malicious URL detection using machine learning: A survey. *arXiv preprint arXiv:1701.07179*.
- Adebowale, M. A., Lwin, K. T., & Hossain, M. A. (2021). Intelligent phishing detection system using deep learning models. *Information Systems*, 101, 101804.
- Zainal, H., Maarof, M. A., & Shamsuddin, S. M. (2009). Feature selection and classification for phishing email detection using machine learning techniques. *International Symposium on Information Technology*, 1, 1–6.

- Jain, A. K., & Gupta, B. B. (2018). A novel approach to protect against phishing attacks at client side using auto-updated white-list. *EURASIP Journal on Information Security*, 2018(1), 1–11.
- Jang, J., Kim, H., & Kim, J. (2014). Malicious URL detection using URL-based heuristic features and comparison of machine learning classifiers. *Proceedings of the 5th International Conference on Computer Science and its Applications*, 31–38.
- Rao, R. S., & Pais, A. R. (2019). Detection of phishing websites using an efficient feature-based machine learning framework. *Neural Computing and Applications*, 31(8), 3851–3873.
- Jain, V., & Richariya, V. (2014). A novel approach to detect phishing URLs using Random Forest classifier. *International Journal of Computer Applications*, 85(10), 1–4.
- Jain, A., & Rathee, G. (2020). A novel approach for phishing URLs detection using lexical features. *International Journal of Information Technology*, 12, 135–141.
- Xiang, G., Hong, J., Rose, C., & Cranor, L. (2010). Cantina+: A feature-rich machine learning framework for detecting phishing web sites. *ACM Transactions on Information and System Security (TISSEC)*, 14(2), 1–28.
- Gaurav, A., Singh, S., & Kumar, S. (2021). Detection of phishing URLs using natural language processing techniques. *Procedia Computer Science*, 192, 540–549.
- Zulkifli, A. N., Selamat, S. R., & Yunos, Z. (2019). Detection and prevention of phishing attacks using deep learning models: A survey. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 10(12), 451–458.
- Whittaker, C., Ryner, B., & Nazif, M. (2010). Large-scale automatic classification of phishing pages. *NDSS*, 10(3), 1–13.
- Tewari, A., & Gupta, B. B. (2020). Machine learning-based approach to detect phishing emails. *Security and Privacy*, 3(1), e91.
- Han, S., & Kim, D. (2022). Phishing website detection using hybrid deep learning approach. *IEEE Access*, 10, 12248–12259.
- Parveen, R., & Mehtre, B. M. (2020). A machine learning-based phishing detection model using URL features. *Journal of King Saud University – Computer and Information Sciences*, In Press.
- Wu, M., Miller, R. C., & Garfinkel, S. L. (2006). Do security toolbars actually prevent phishing attacks?. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 601–610.

- Idika, N., & Mathur, A. P. (2007). A survey of malware detection techniques. *Purdue University Technical Report*.
- Alazab, M., & Broadhurst, R. (2016). Malware and cybersecurity: Issues and threats. *Cybercrime and Digital Forensics*, 22–39.
- Garera, S., Provos, N., Chew, M., & Rubin, A. D. (2007). A framework for detection and measurement of phishing attacks. *WORM '07 Proceedings*, 1–8.
- Baki, S., et al. (2017). Phishing email detection using natural language processing techniques. *IEEE International Conference on Big Data*, 602–610.
- Basit, A., Zafar, M., Liu, X., & Javed, A. R. (2021). A comprehensive survey of AI-enabled phishing attacks detection techniques. *Journal of Network and Computer Applications*, 179, 102980.
- Kintis, P., et al. (2017). Hiding in plain sight: A longitudinal study of combosquatting abuse. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 569–586.
- Sheng, S., Holbrook, M., Kumaraguru, P., Cranor, L. F., & Downs, J. (2010). Who falls for phish? A demographic analysis of phishing susceptibility and effectiveness of interventions. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 373–382.
- Thomas, K., et al. (2017). Data breaches, phishing, or malware? Understanding the risks of stolen credentials. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 1421–1434.